



US 20250287017A1

(19) **United States**

(12) **Patent Application Publication**  
**WANG et al.**

(10) **Pub. No.: US 2025/0287017 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **METHOD, APPARATUS, AND MEDIUM FOR VIDEO PROCESSING**

**Publication Classification**

(71) Applicants: **Douyin Vision (Beijing) Co., Ltd.**,  
Beijing (CN); **Bytedance Inc.**, Los  
Angeles, CA (US)

(51) **Int. Cl.**

**H04N 19/159** (2014.01)

**H04N 19/105** (2014.01)

**H04N 19/176** (2014.01)

**H04N 19/196** (2014.01)

(72) Inventors: **Yang WANG**, Beijing (CN); **Kai Zhang**, Los Angeles, CA (US); **Na Zhang**, Beijing (CN); **Li Zhang**, Los Angeles, CA (US)

(52) **U.S. Cl.**

CPC ..... **H04N 19/159** (2014.11); **H04N 19/105** (2014.11); **H04N 19/176** (2014.11); **H04N 19/196** (2014.11)

(21) Appl. No.: **19/033,286**

(57)

**ABSTRACT**

(22) Filed: **Jan. 21, 2025**

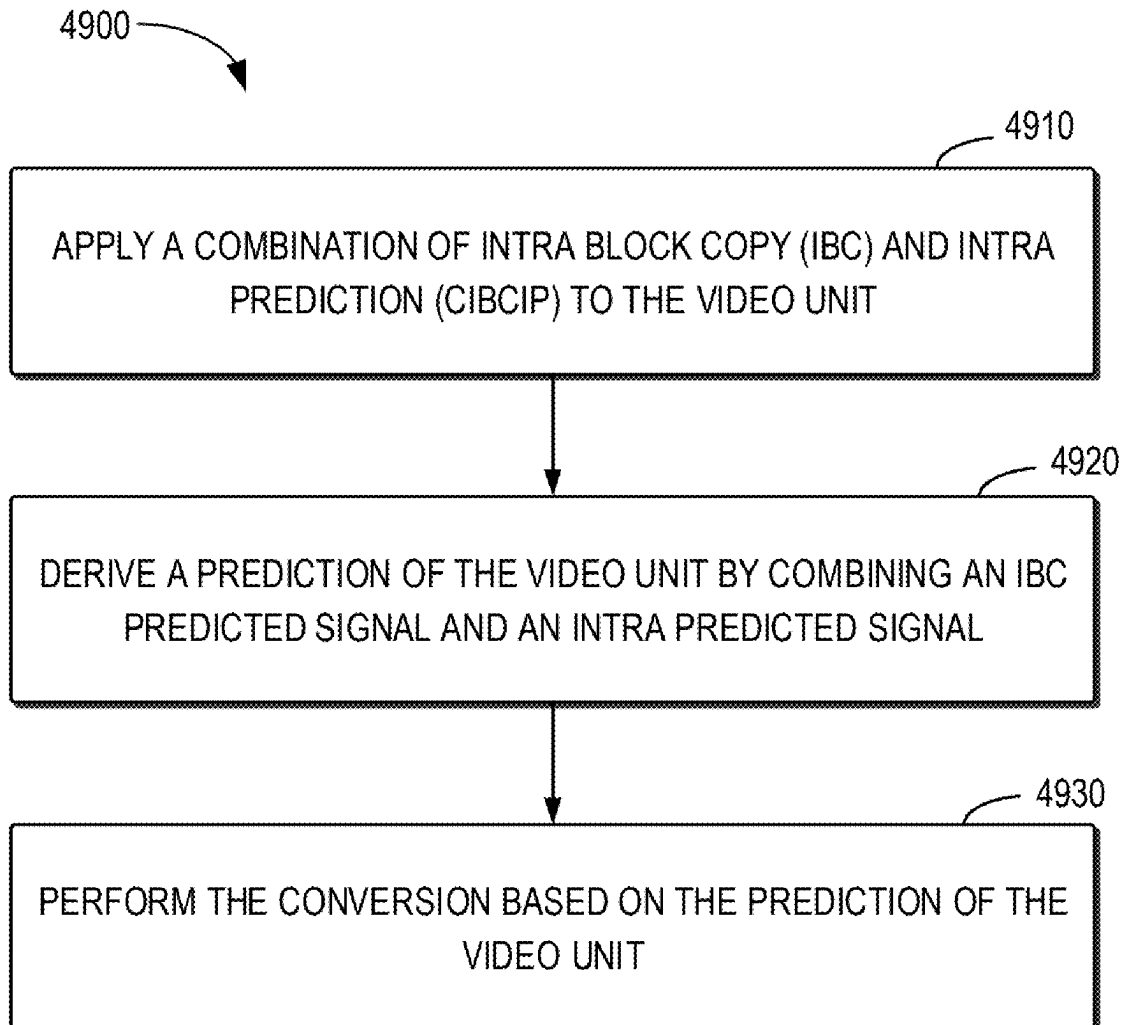
**Related U.S. Application Data**

(63) Continuation of application No. PCT/CN2023/108704, filed on Jul. 21, 2023.

Embodiments of the disclosure provide a solution for video processing. A method for video processing is proposed. The method includes: applying, for a conversion between a video unit of a video and a bitstream of the video unit, a combination of intra block copy (IBC) and intra prediction (CIBCIP) to the video unit; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and performing the conversion based on the prediction of the video unit.

(30) **Foreign Application Priority Data**

Jul. 21, 2022 (WO) ..... PCT/CN2022/107183



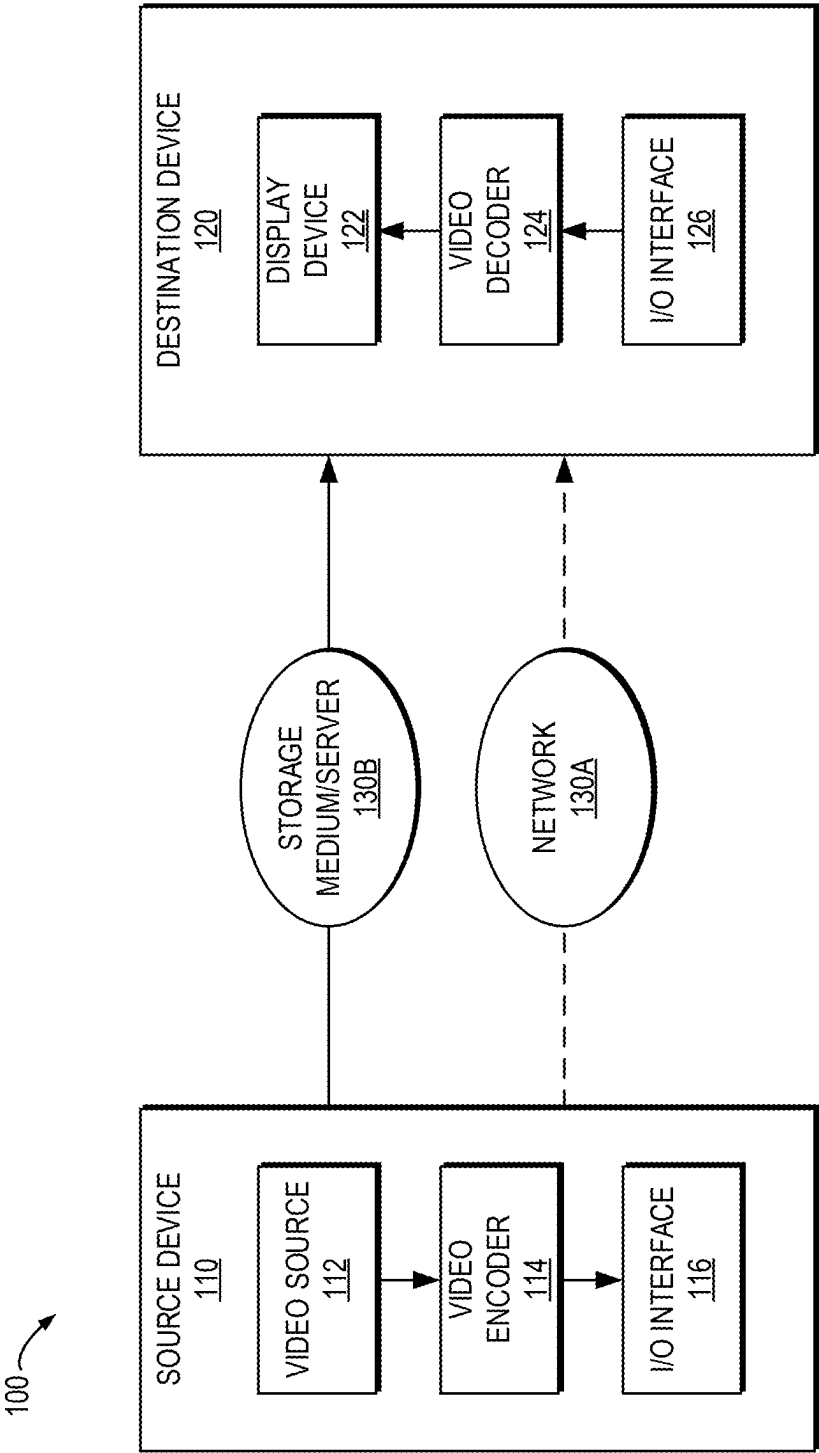


Fig. 1

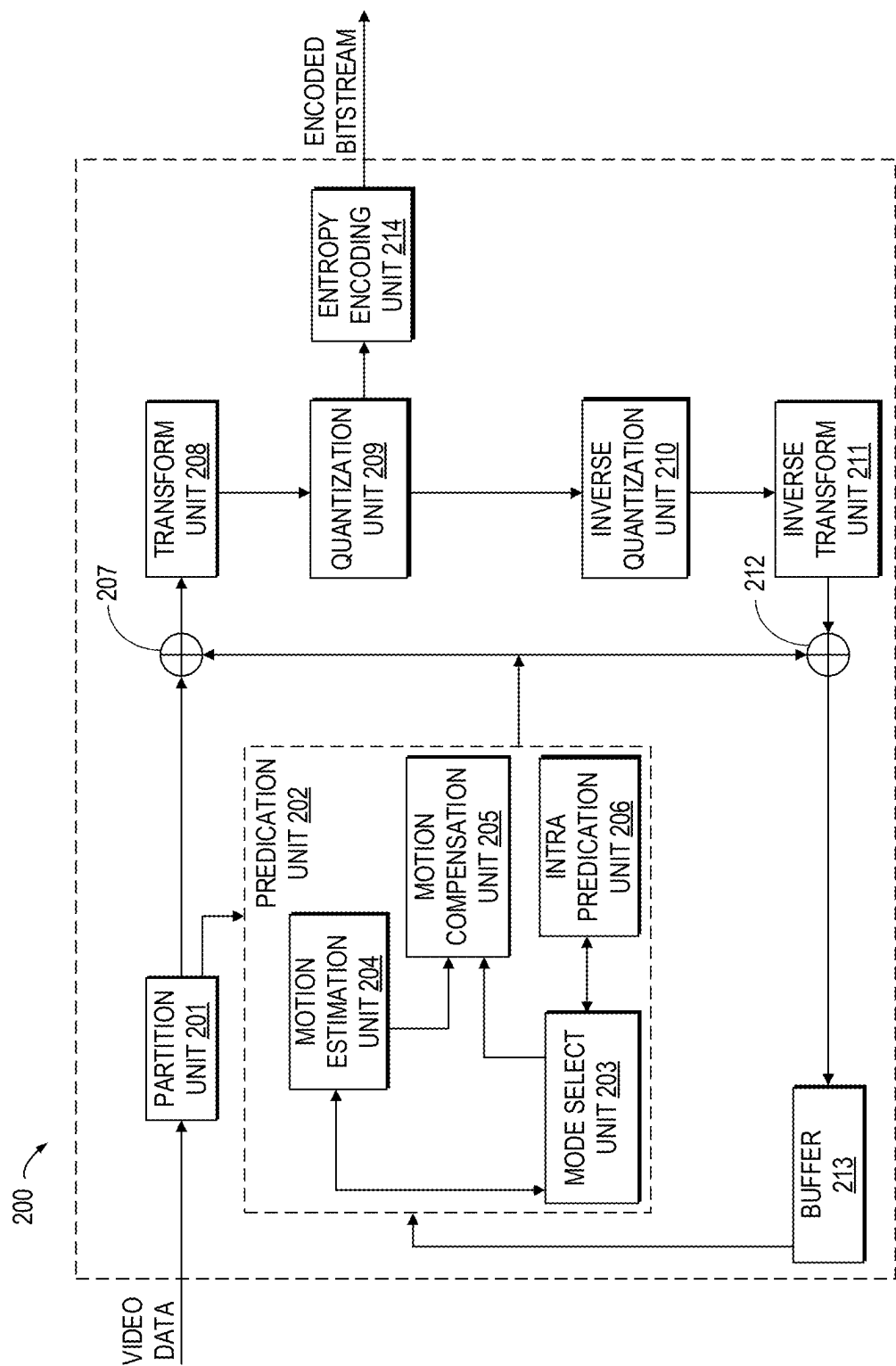


Fig. 2

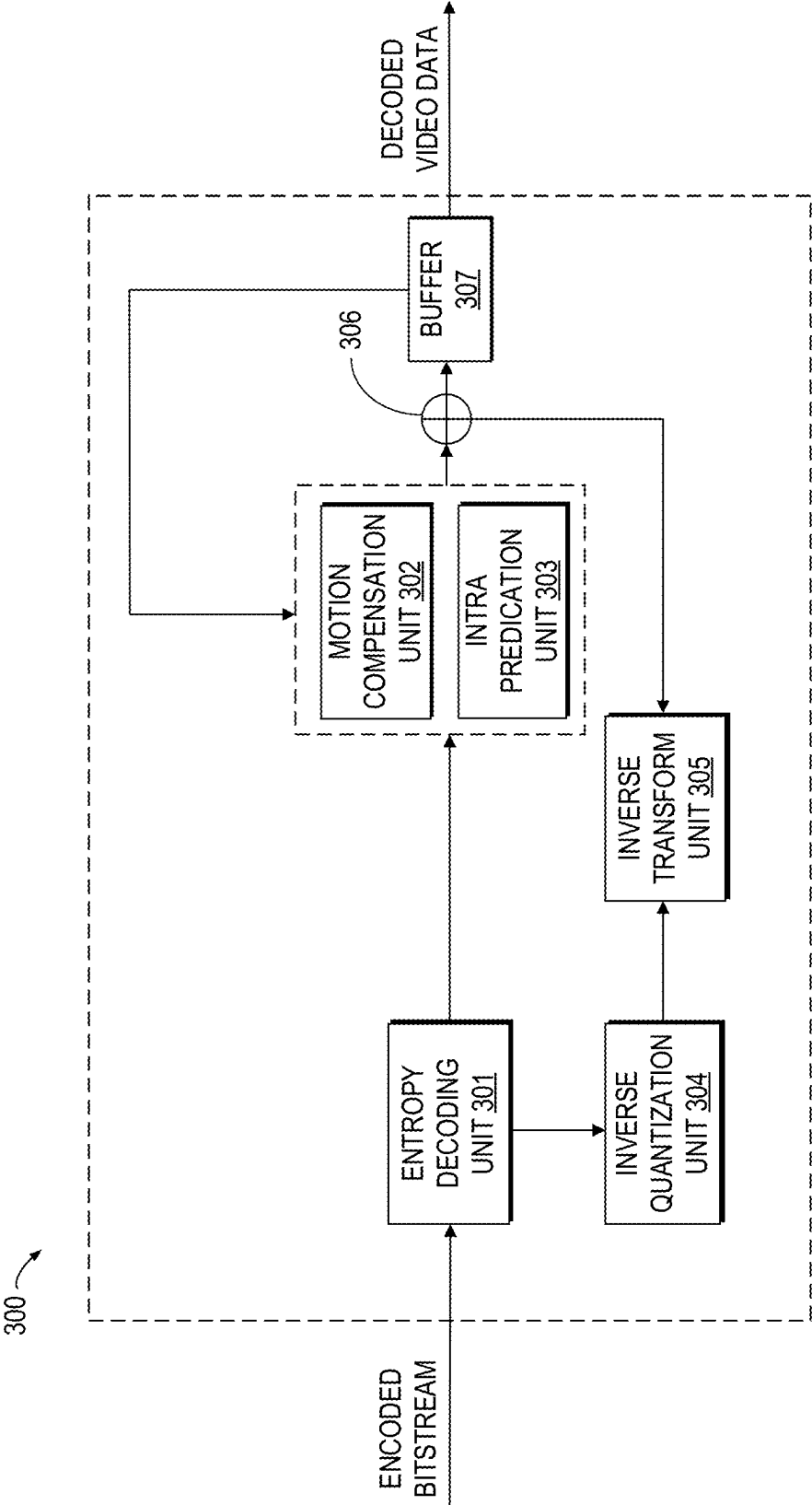


Fig. 3

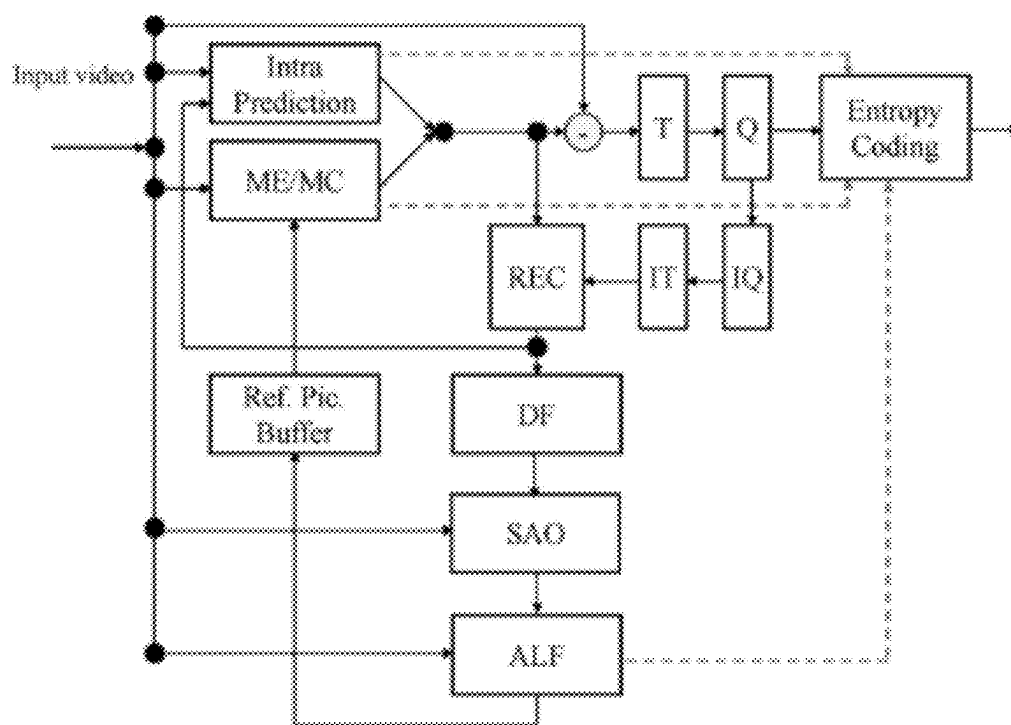


Fig. 4

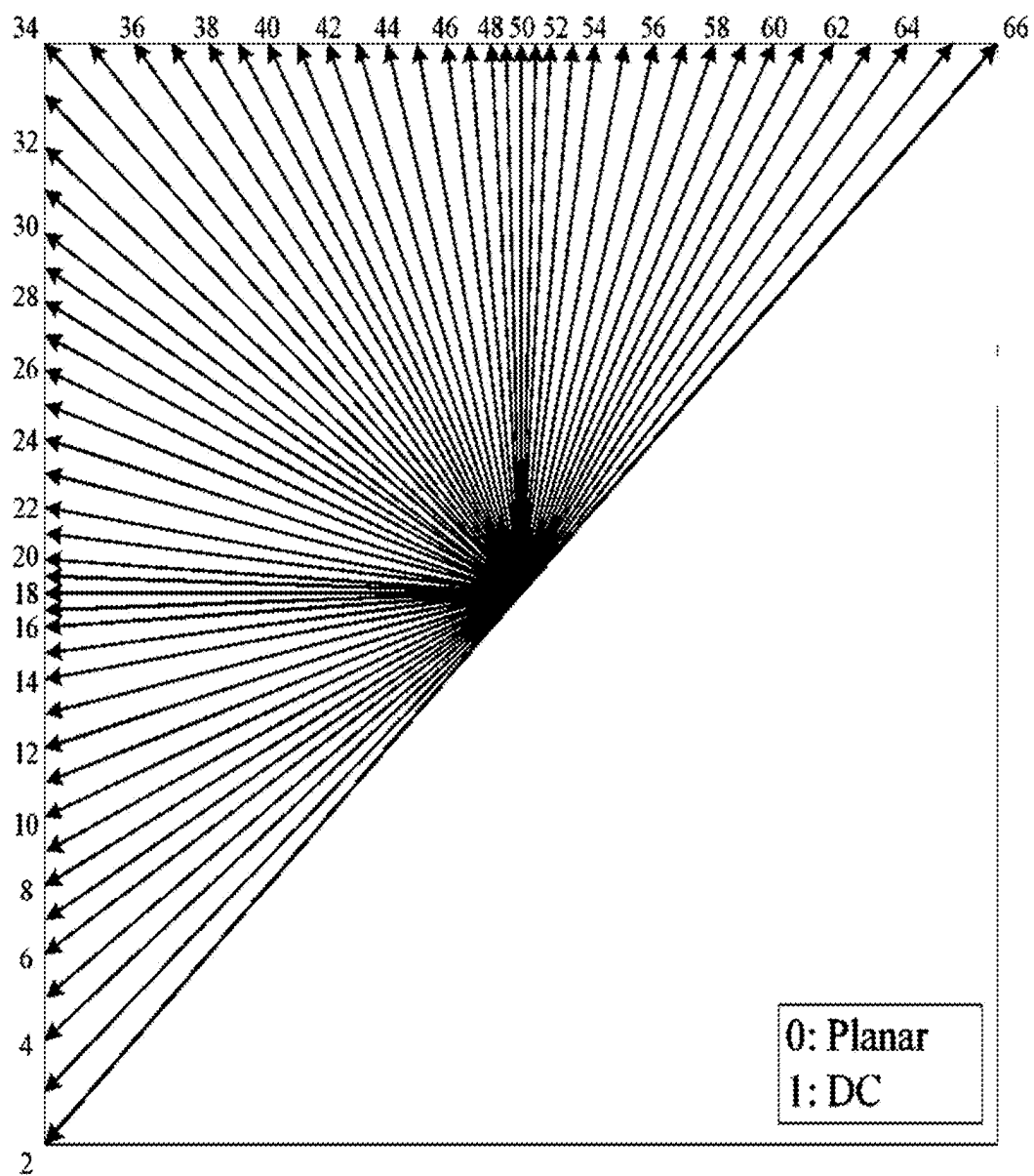


Fig. 5

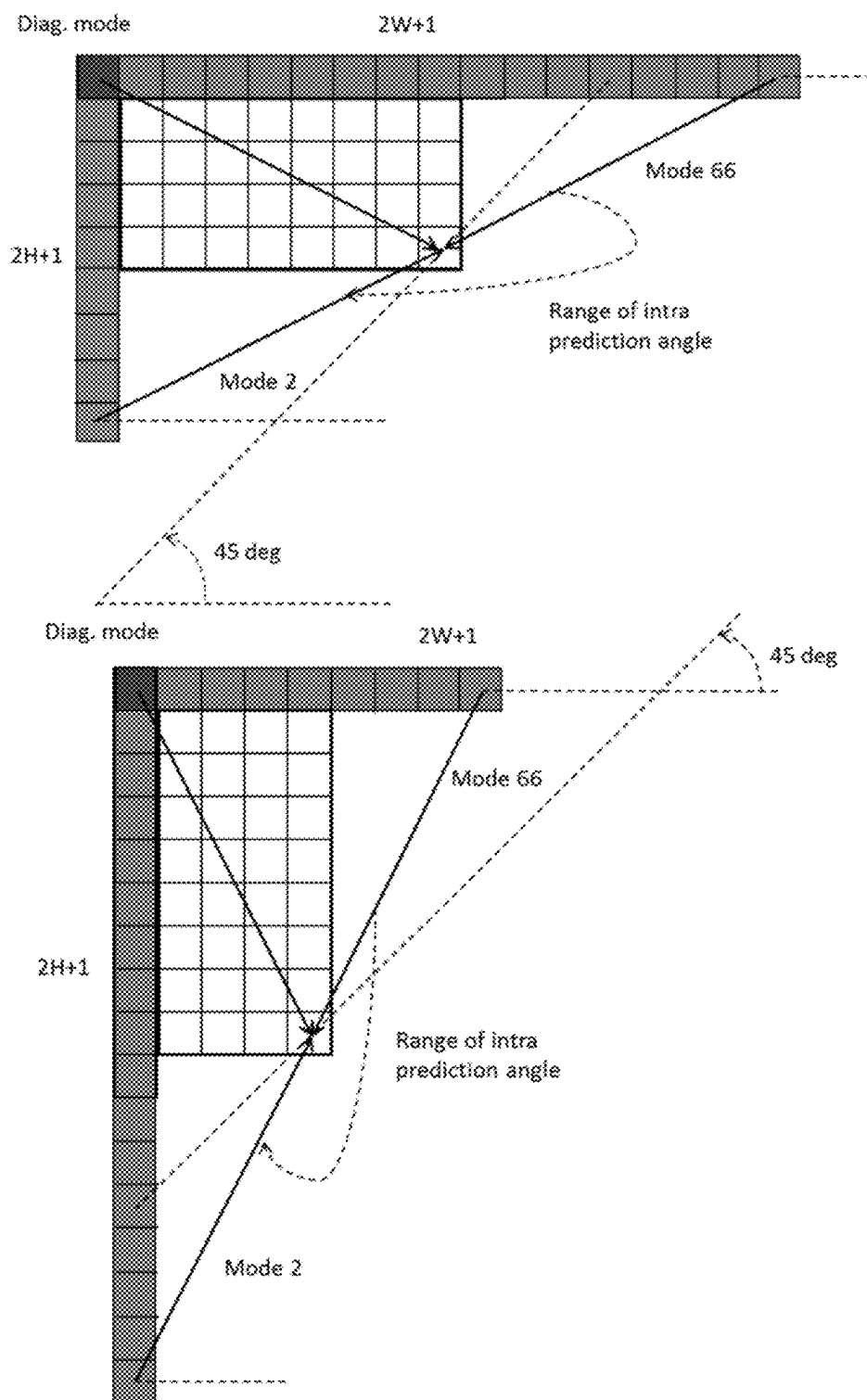
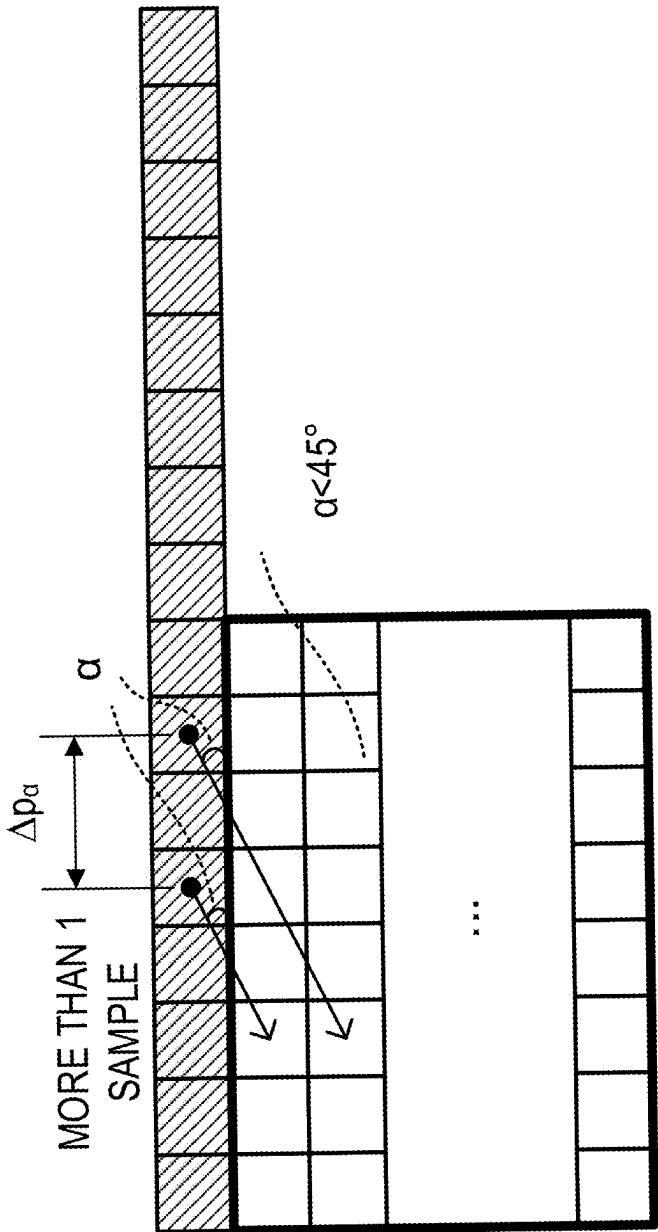
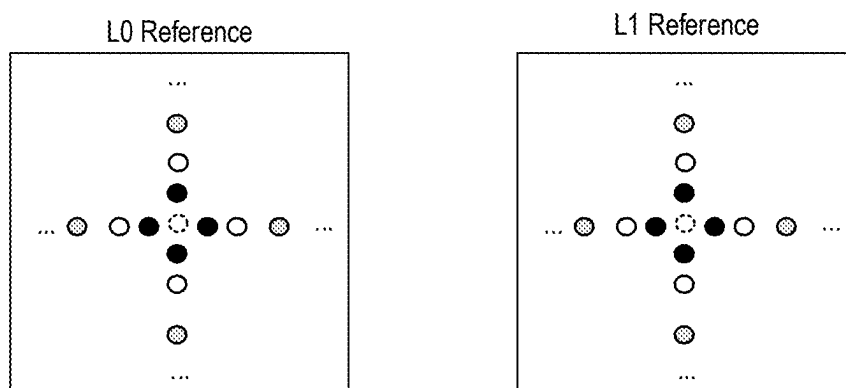


Fig. 6







**Fig. 8**

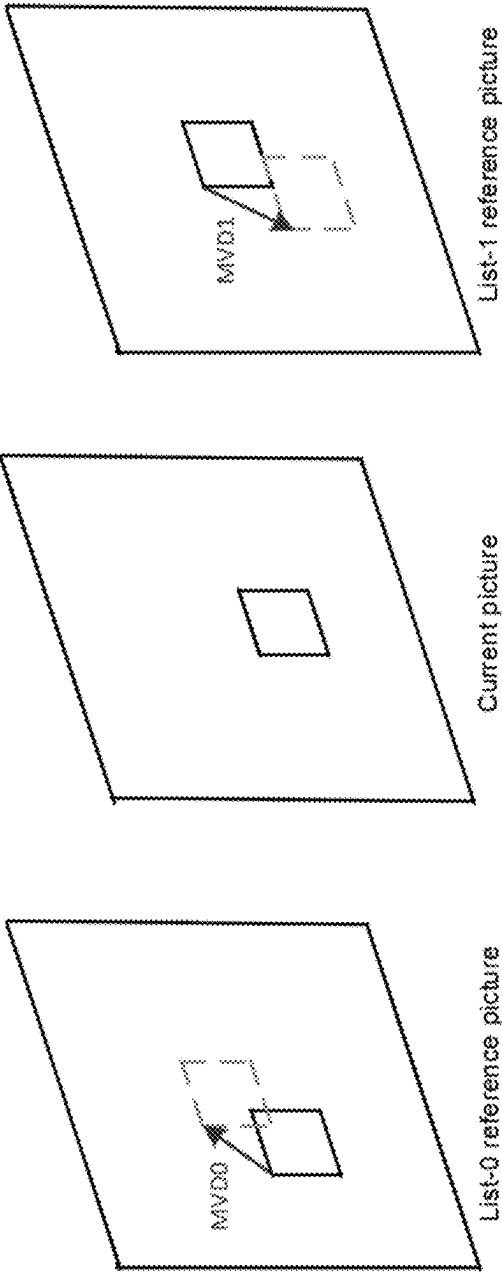


Fig. 9

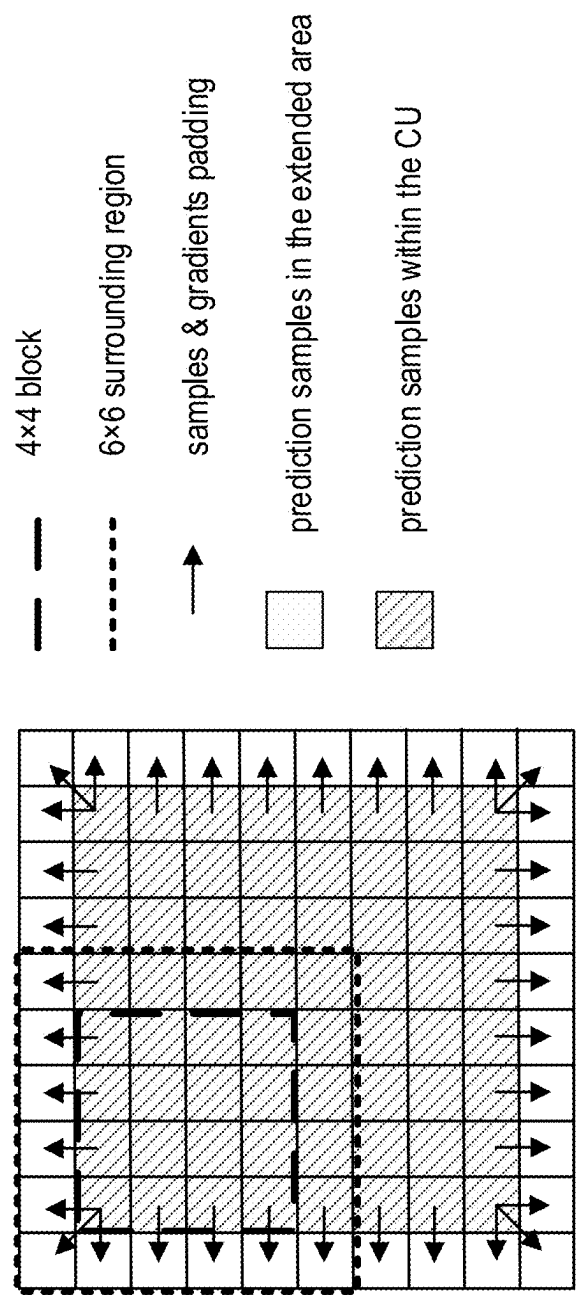


Fig. 10

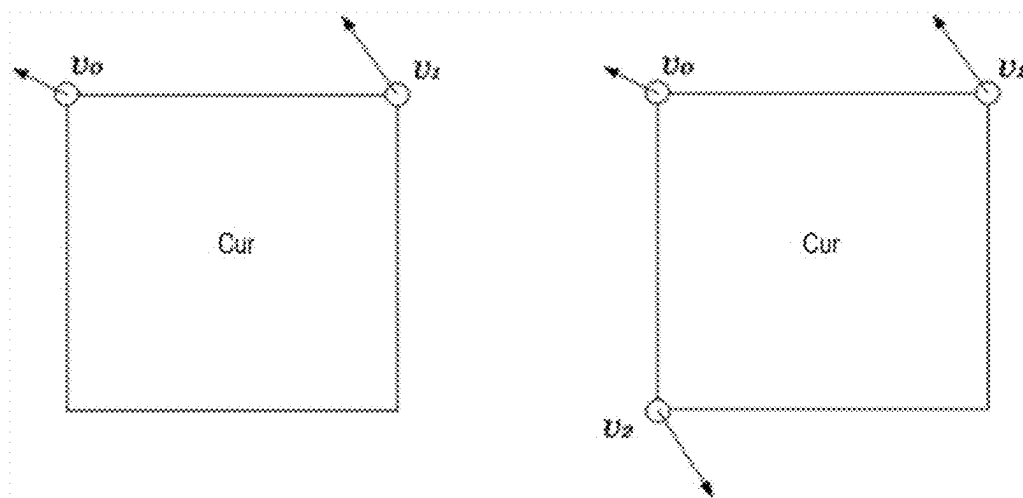


Fig. 11

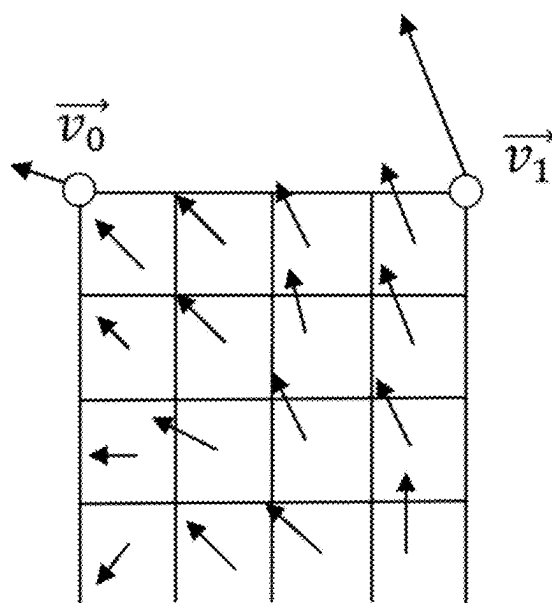


Fig. 12

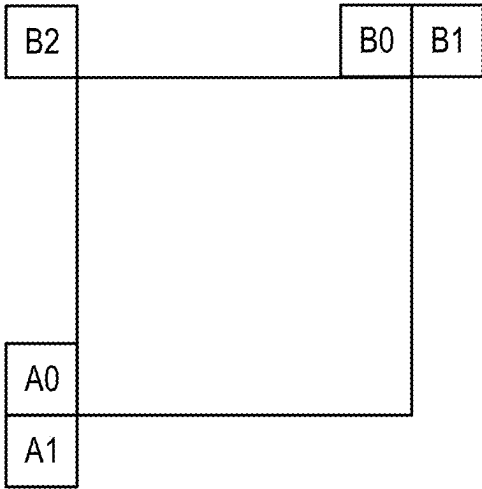


Fig. 13

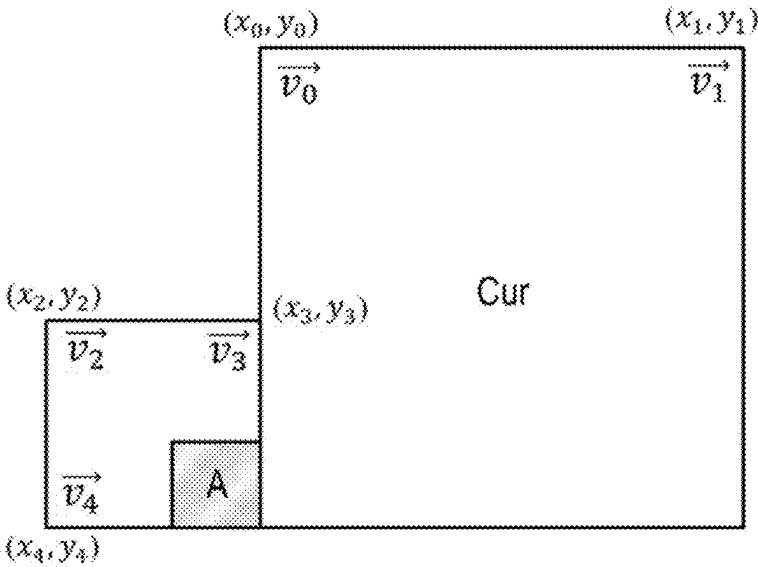


Fig. 14

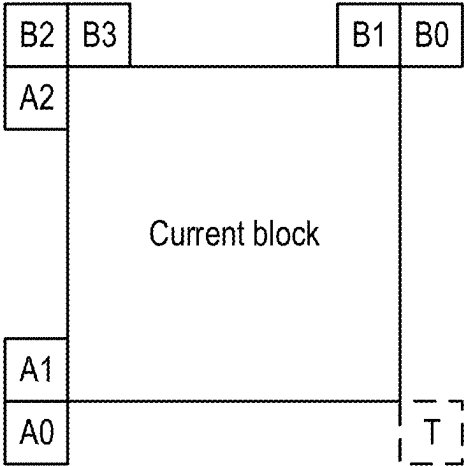


Fig. 15

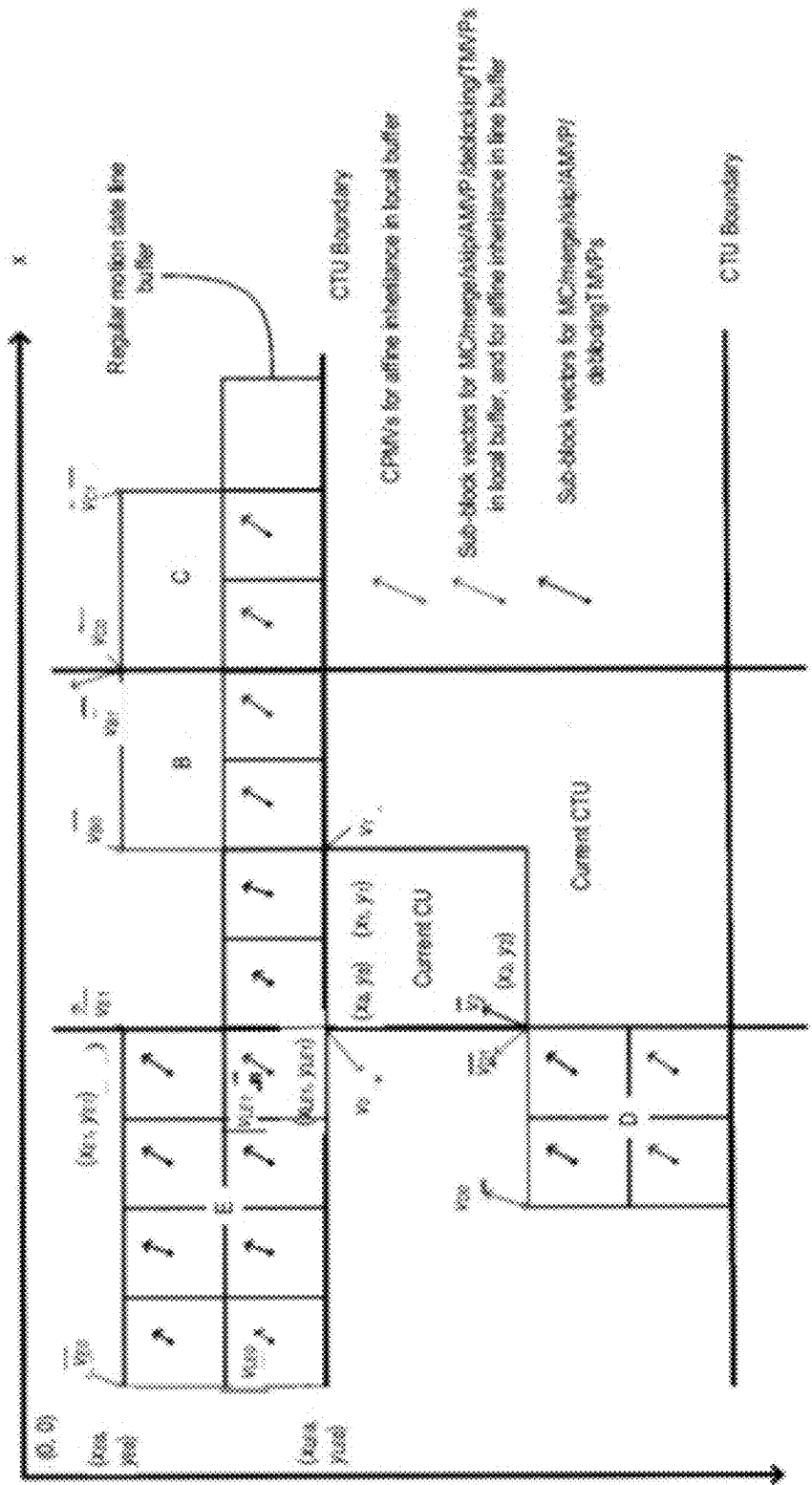


Fig. 16

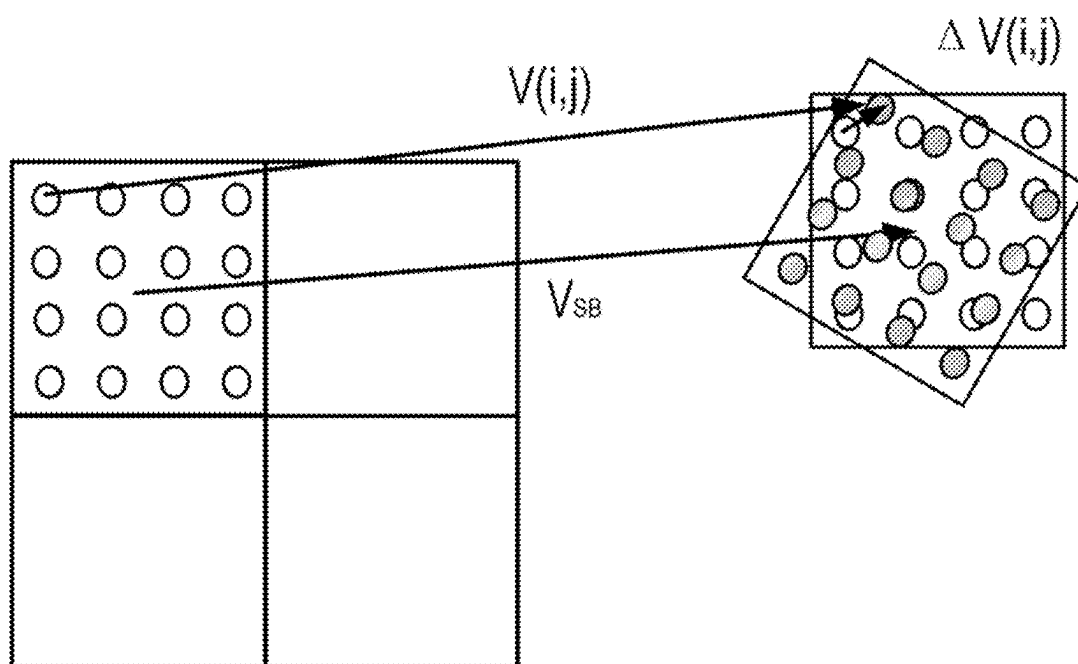


Fig. 17



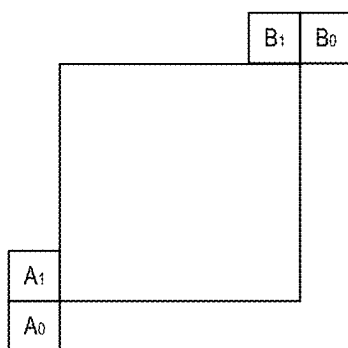


Fig. 18A

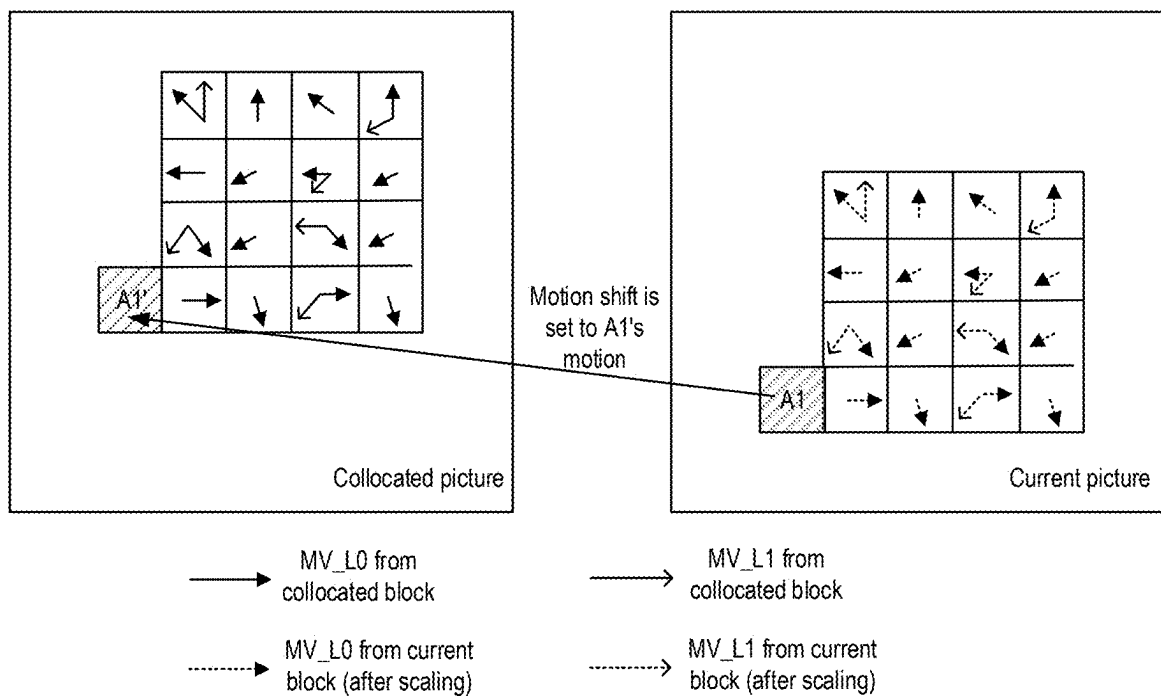


Fig. 18B

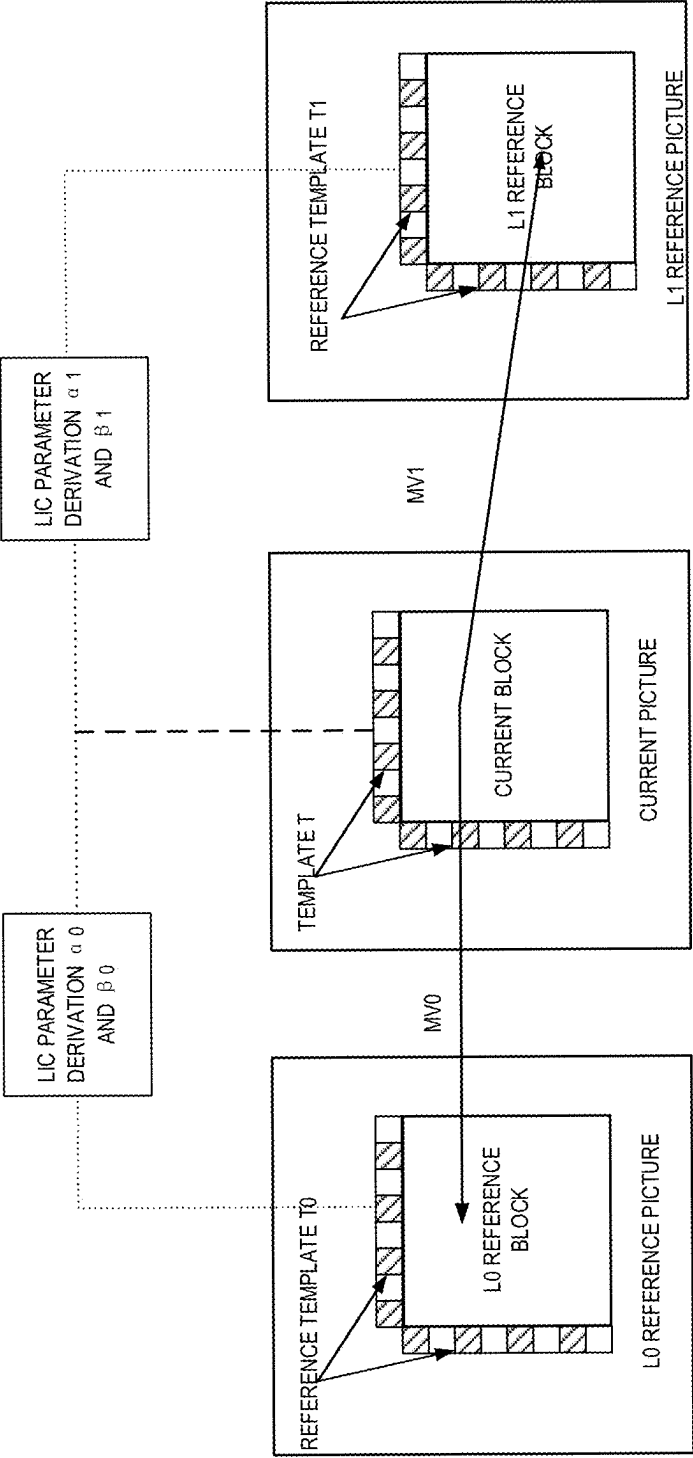


Fig. 19

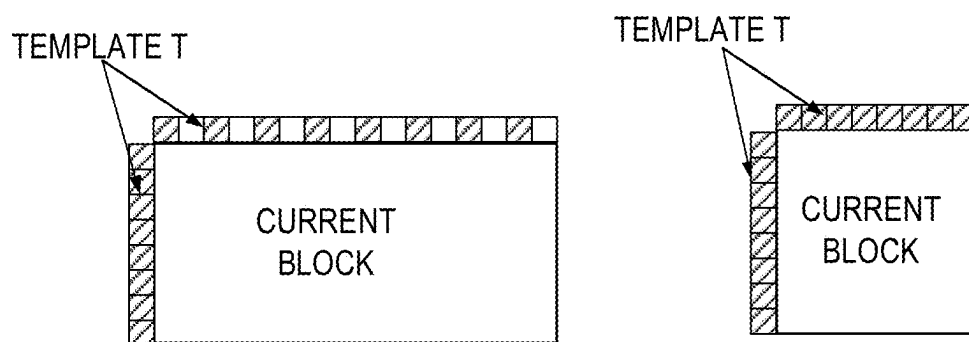


Fig. 20

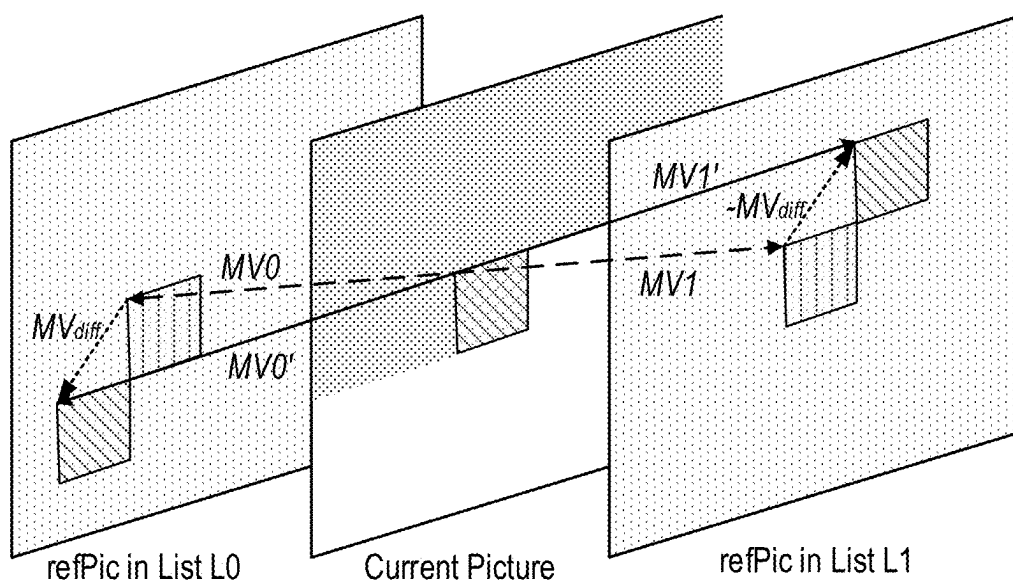


Fig. 21

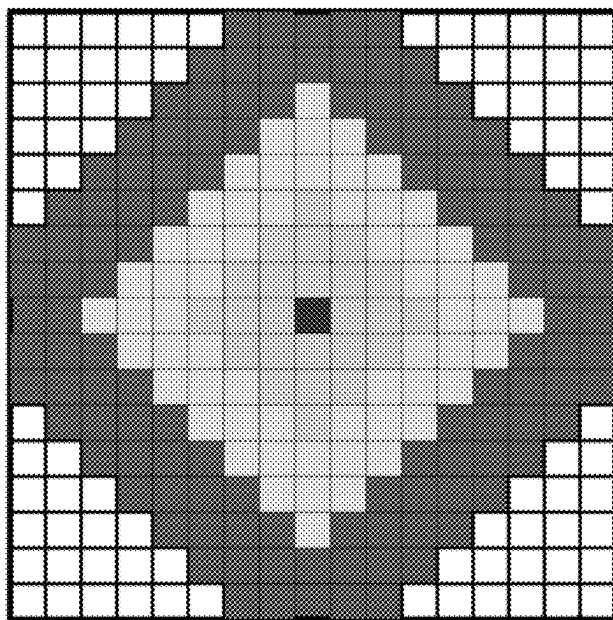


Fig. 22

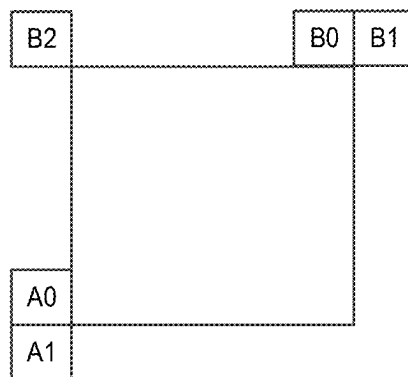


Fig. 23

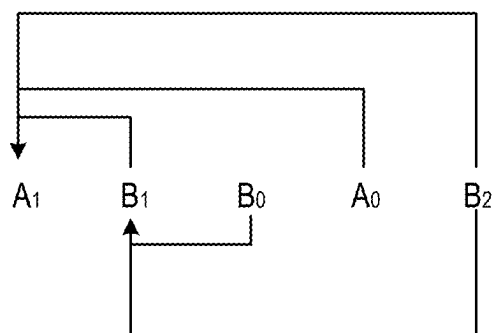


Fig. 24

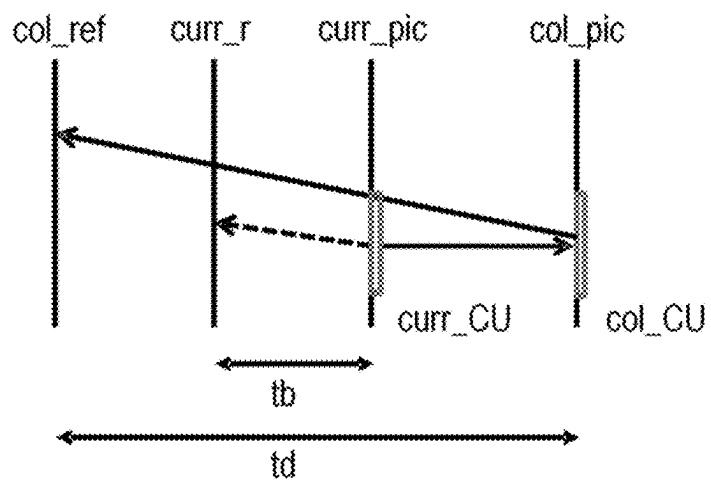


Fig. 25

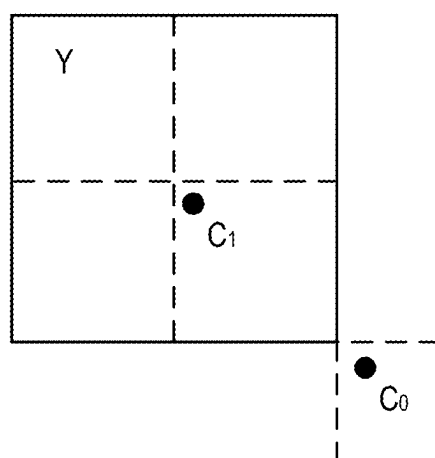


Fig. 26

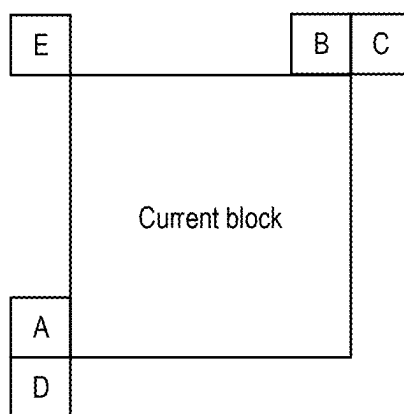


Fig. 27

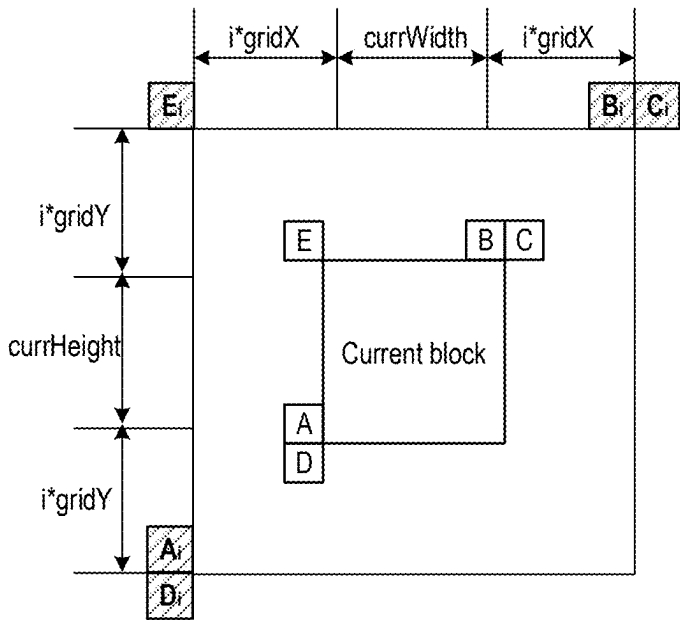


Fig. 28

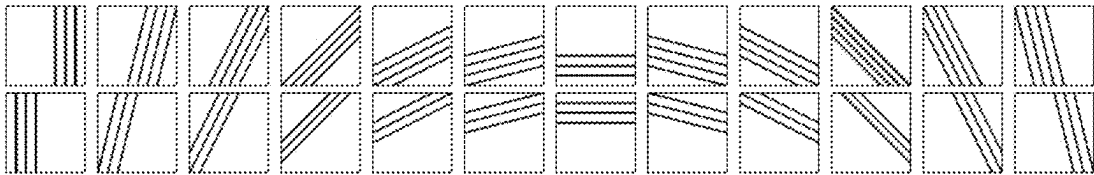


Fig. 29

| Merge index | L0 MV | L1 MV |
|-------------|-------|-------|
| 0           | x     |       |
| 1           |       | x     |
| 2           | x     |       |
| 3           |       | x     |
| 4           | x     |       |

Fig. 30

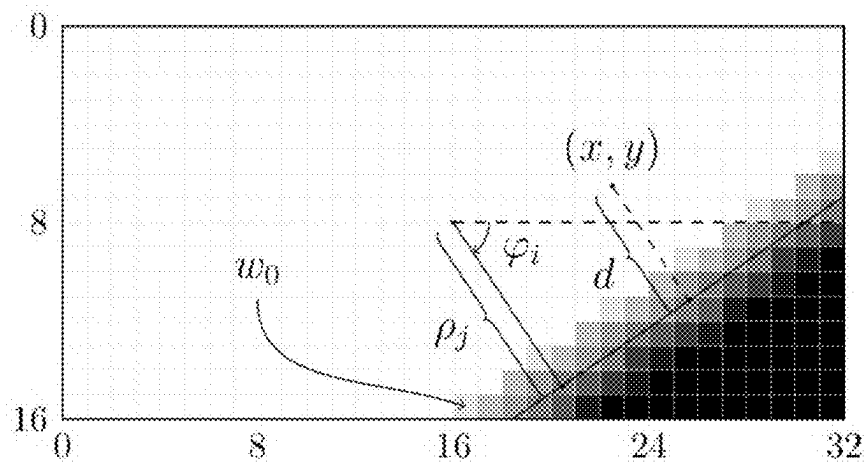


Fig. 31

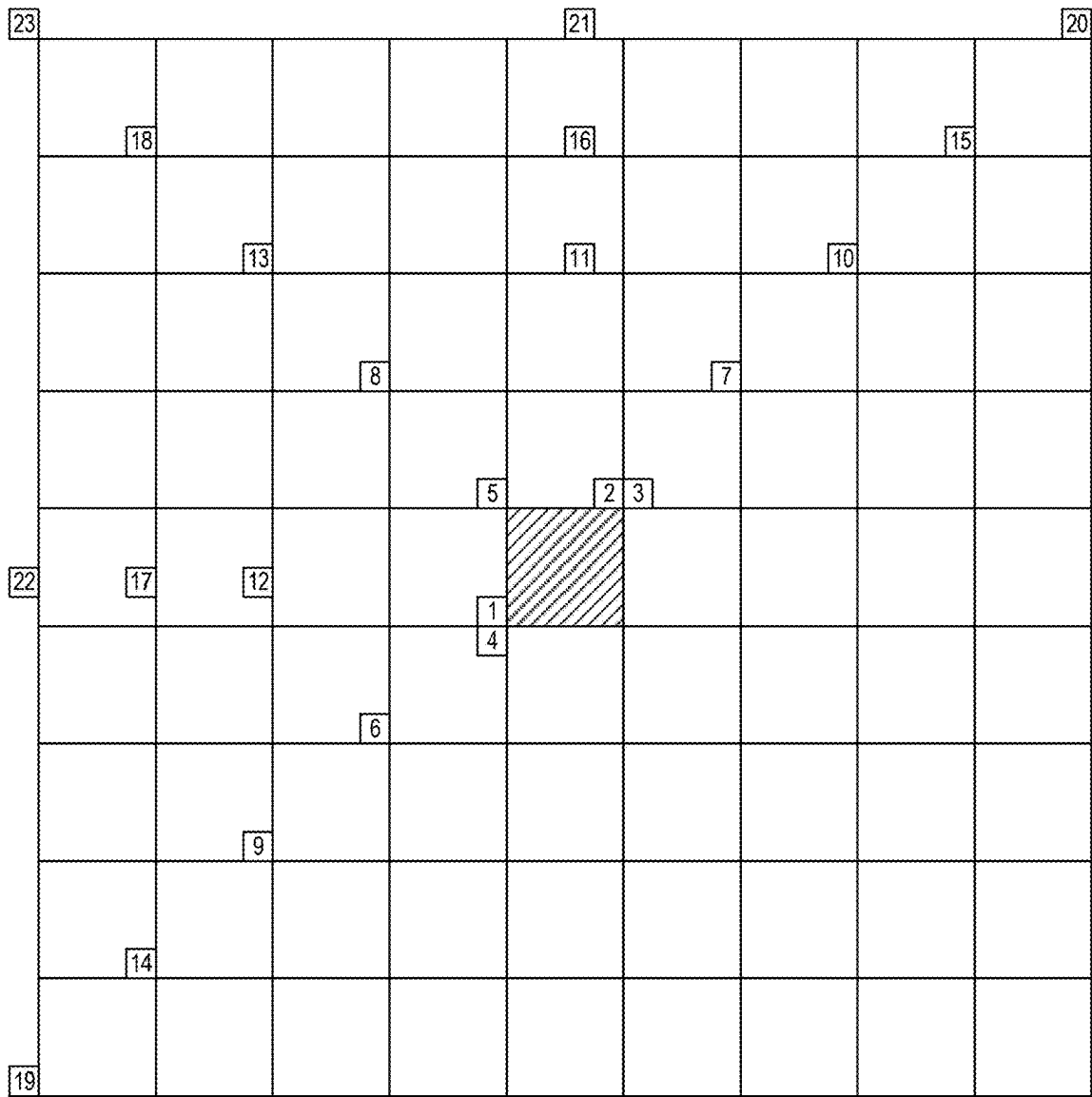


Fig. 32



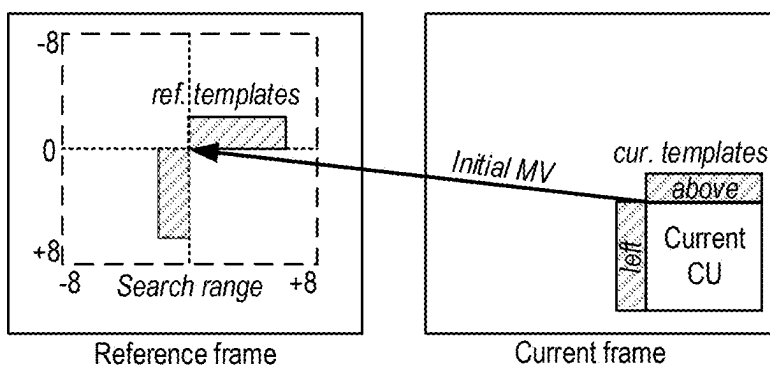


Fig. 33

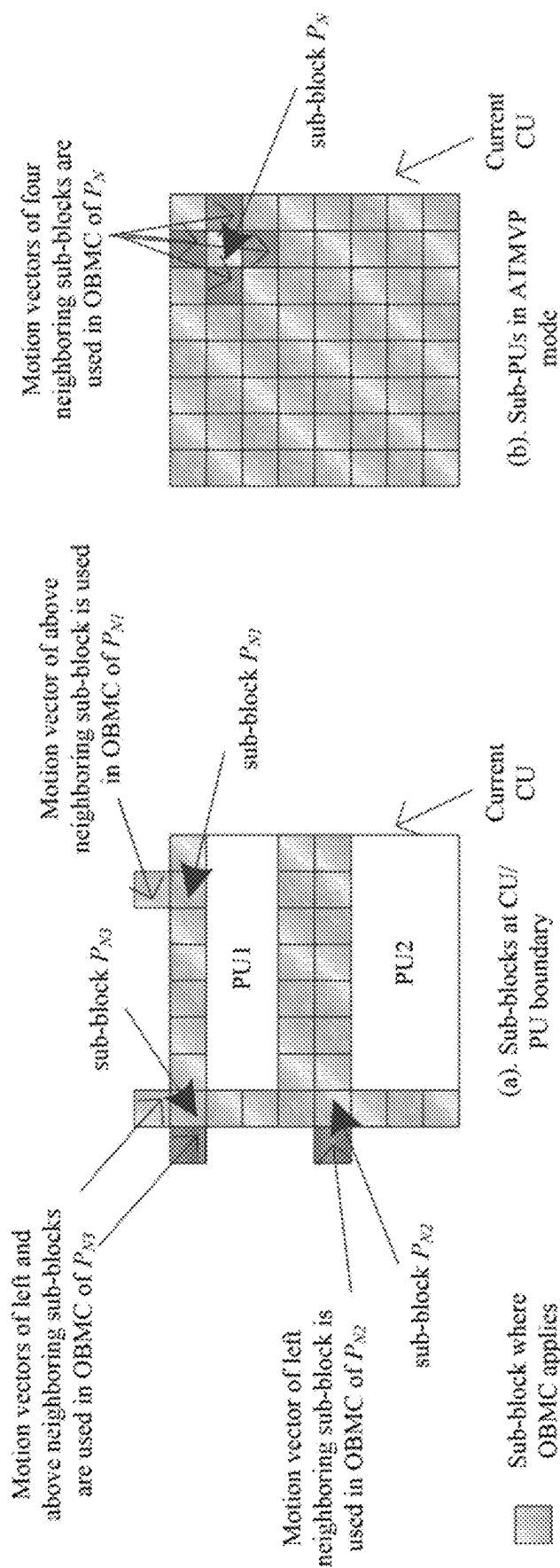


Fig. 34

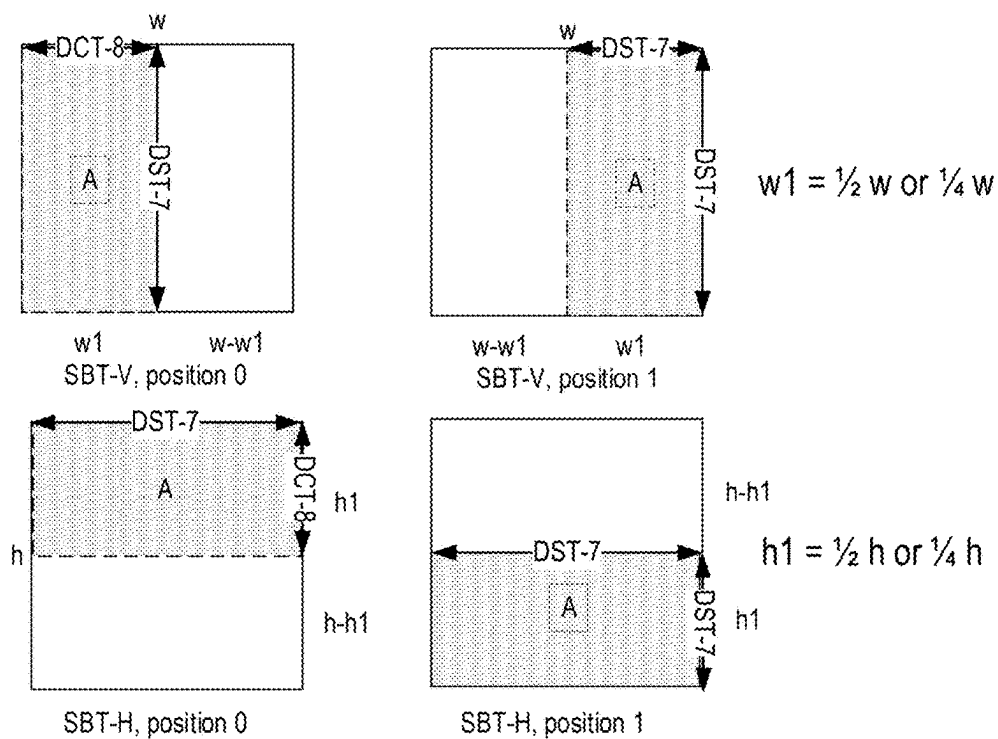


Fig. 35

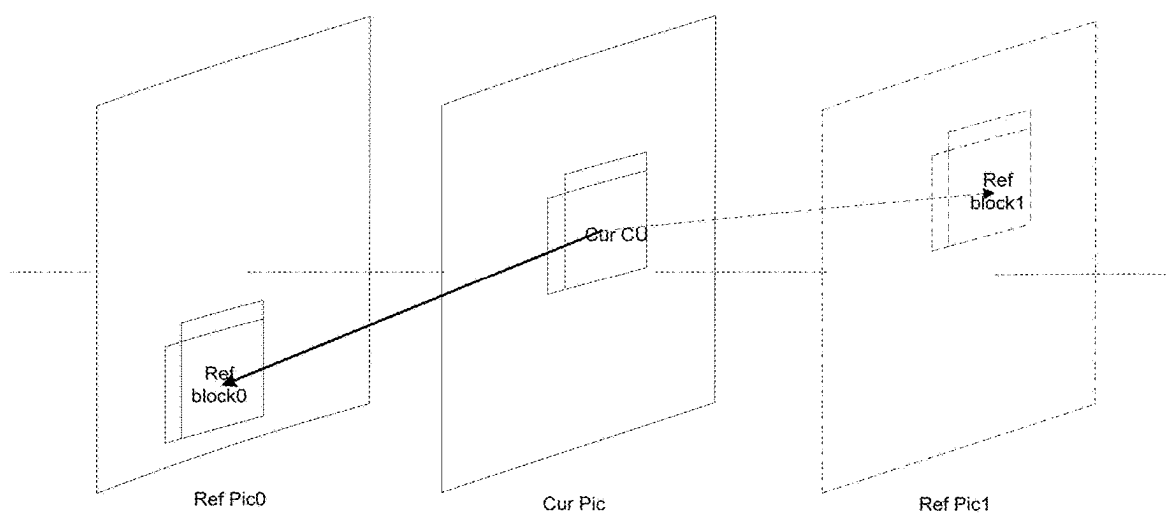


Fig. 36

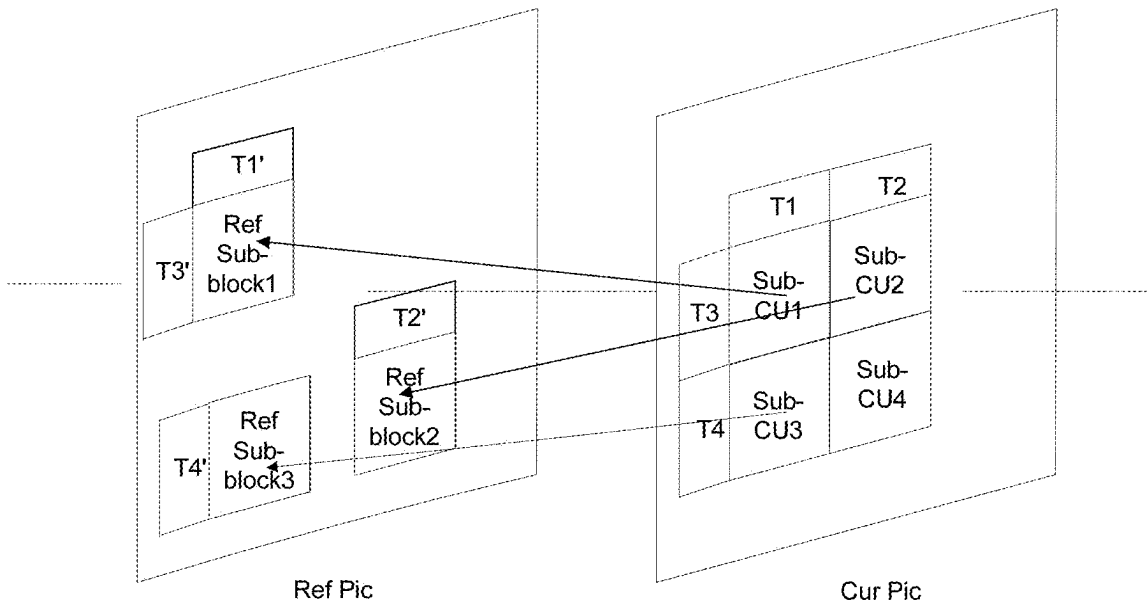


Fig. 37

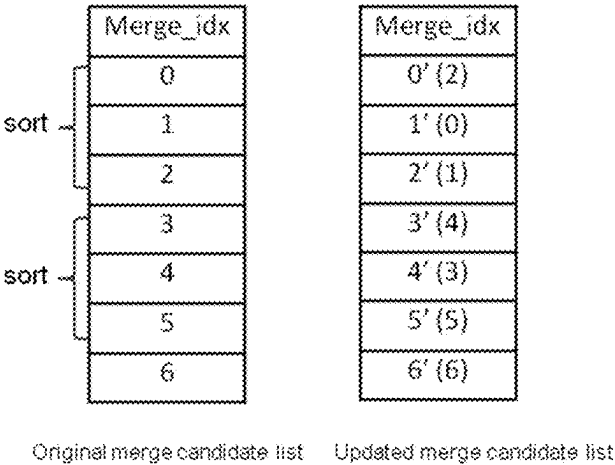


Fig. 38

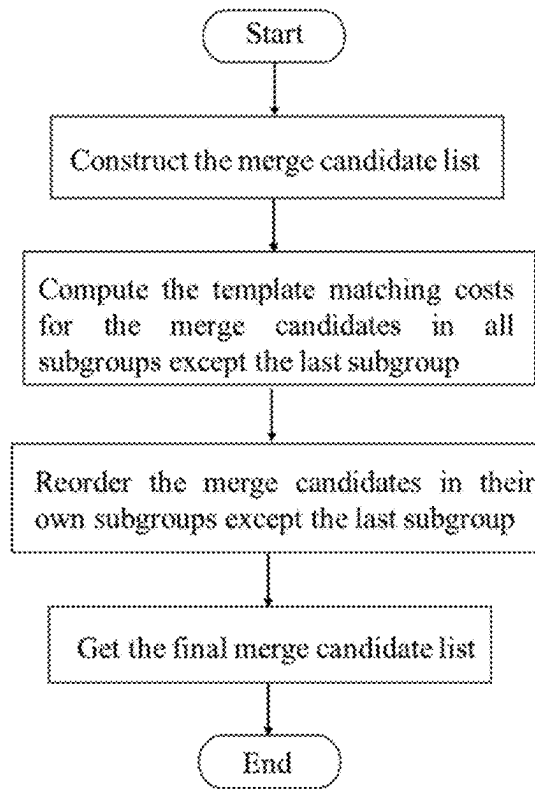


Fig. 39

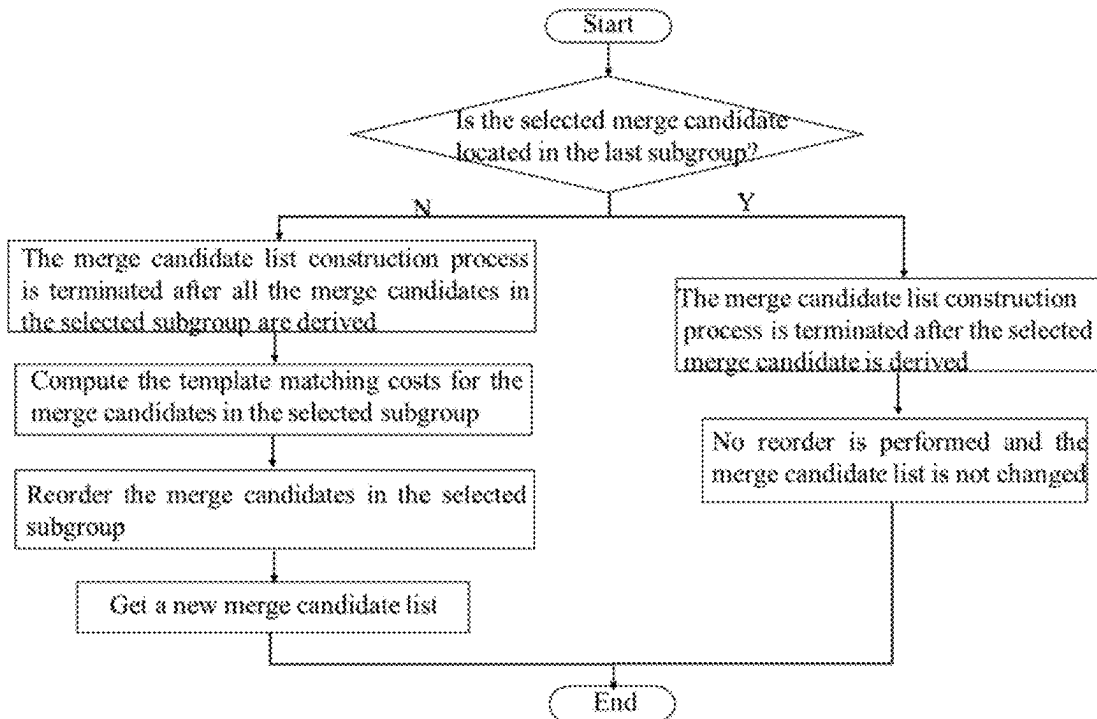


Fig. 40

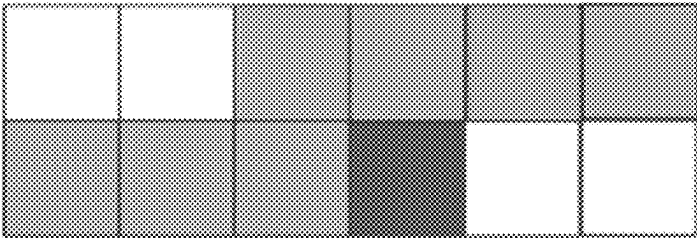


Fig. 41

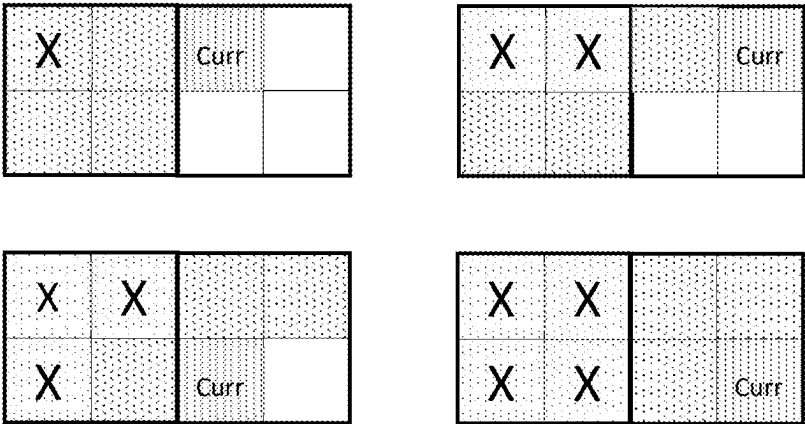


Fig. 42

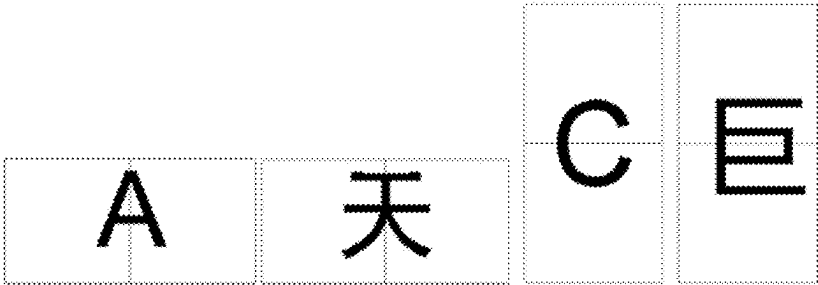


Fig. 43

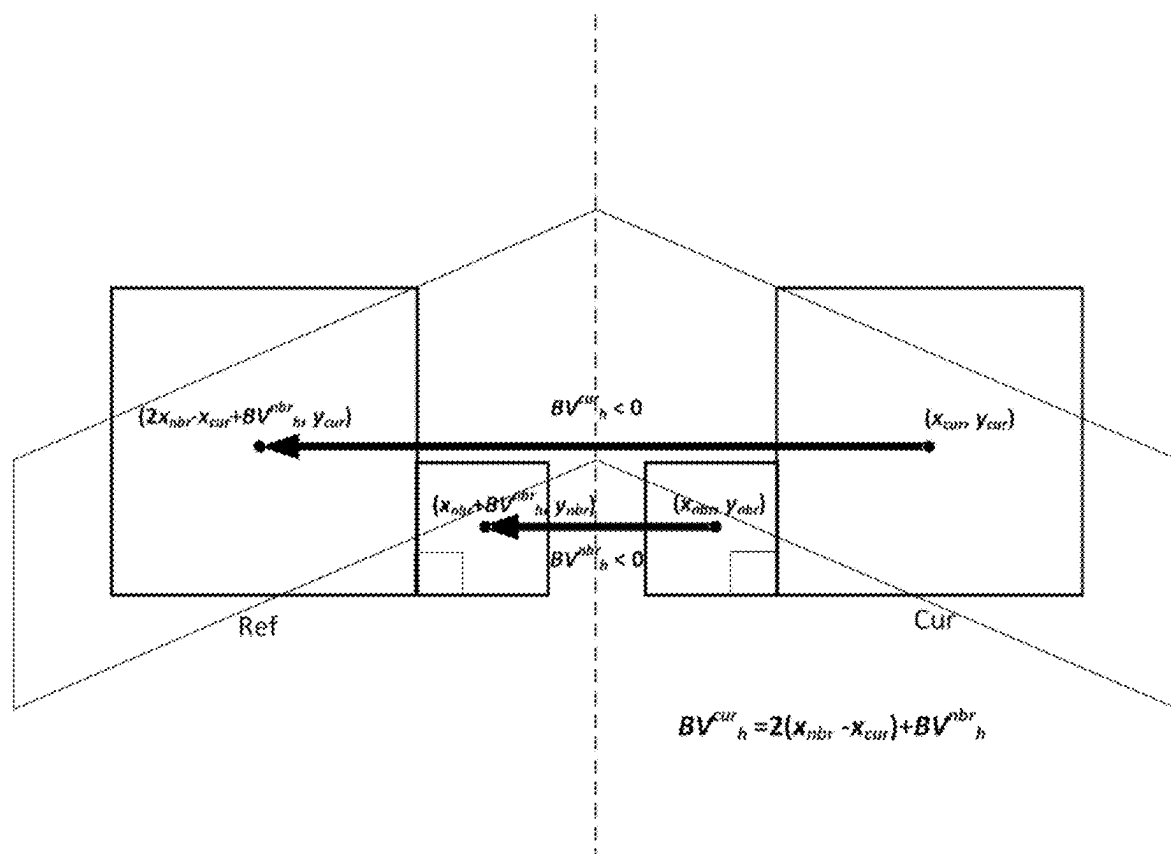


Fig. 44(a)

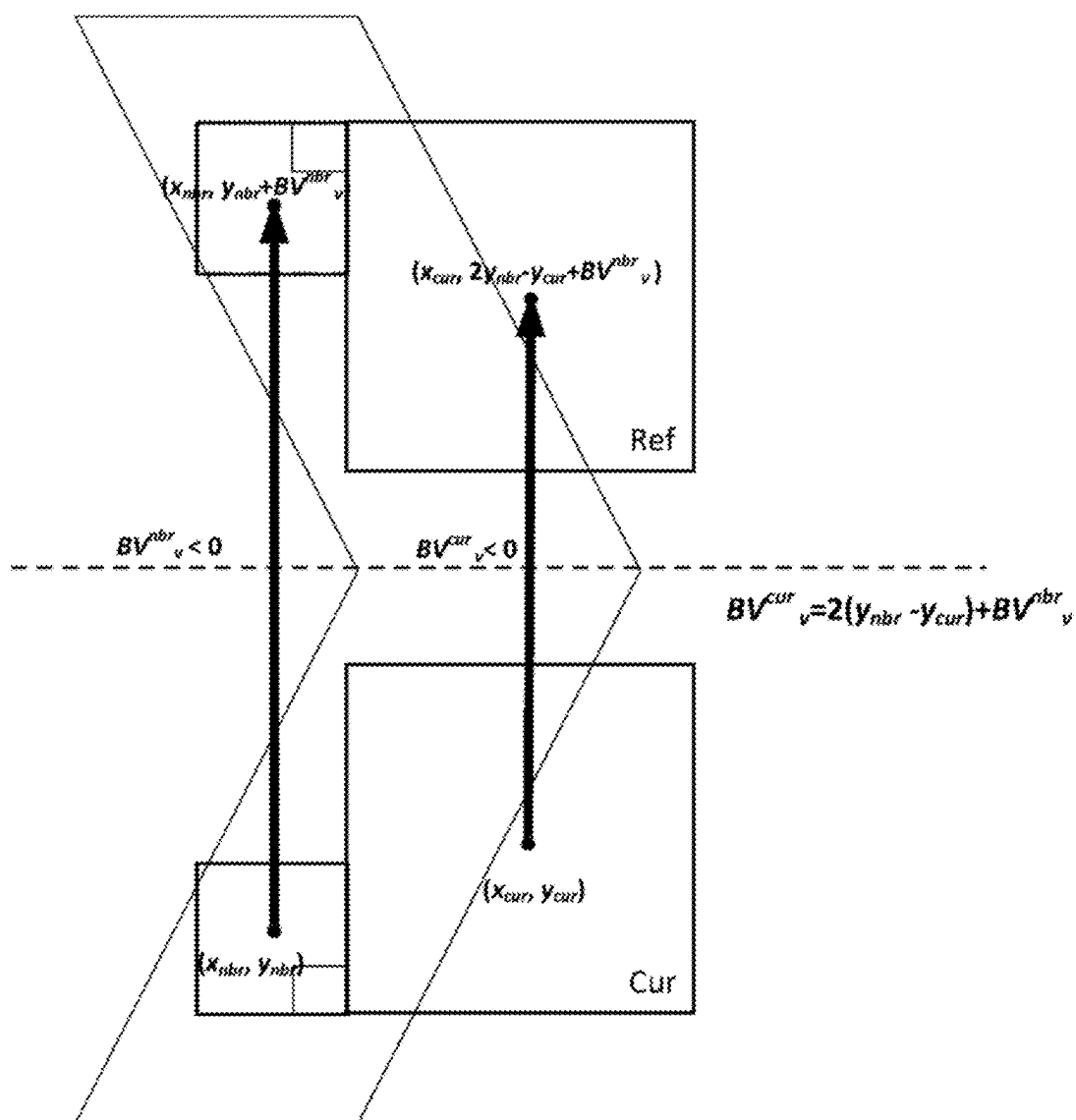


Fig. 44(b)



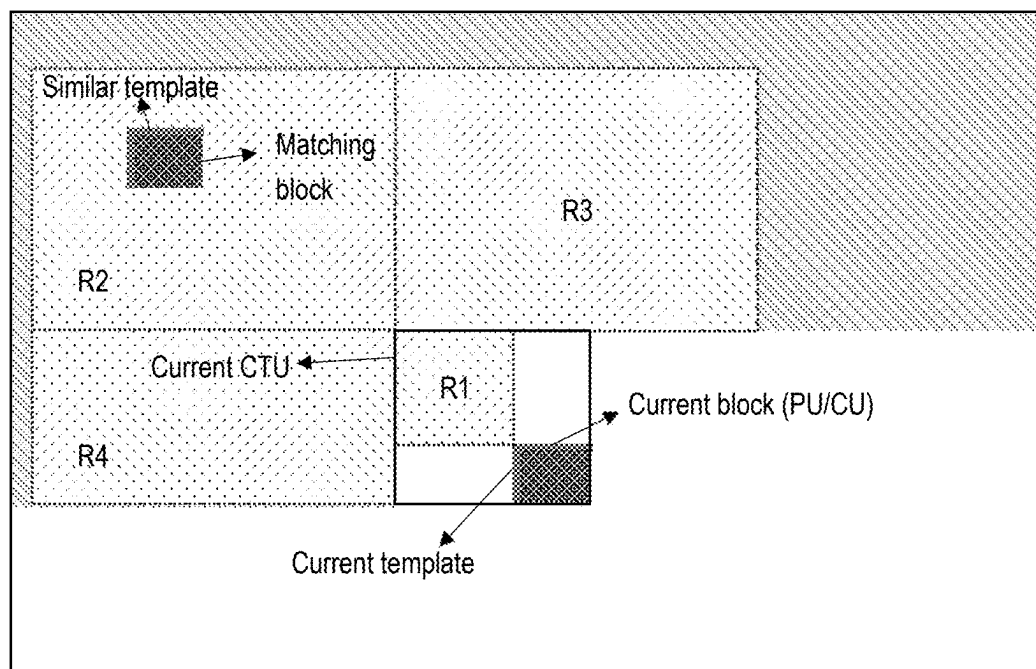


Fig. 45

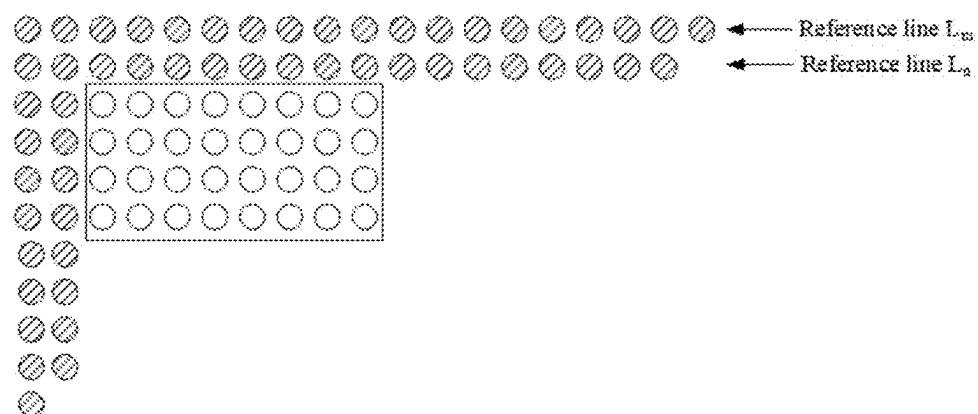


Fig. 46

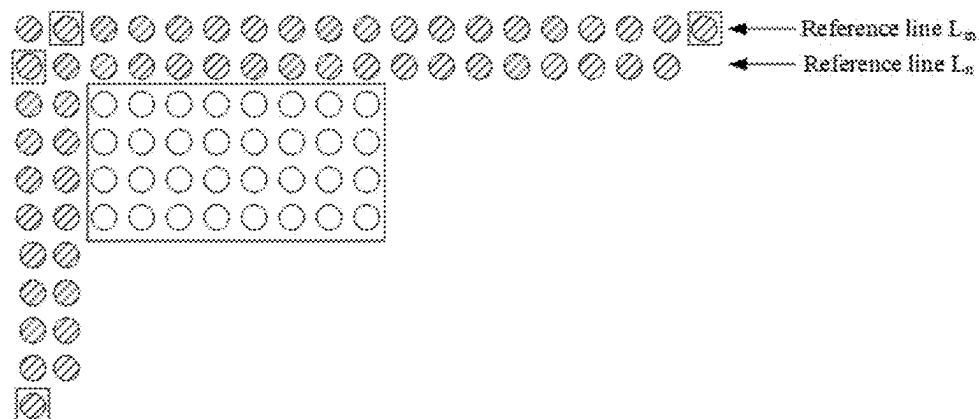


Fig. 47

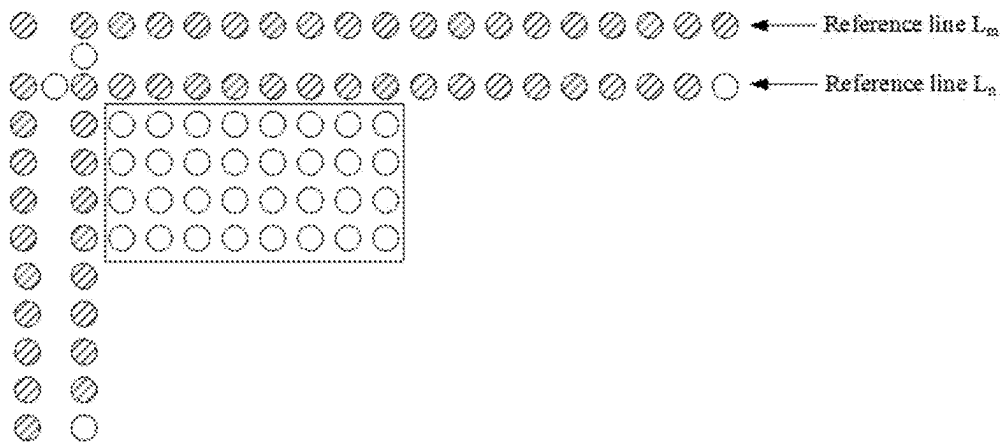


Fig. 48

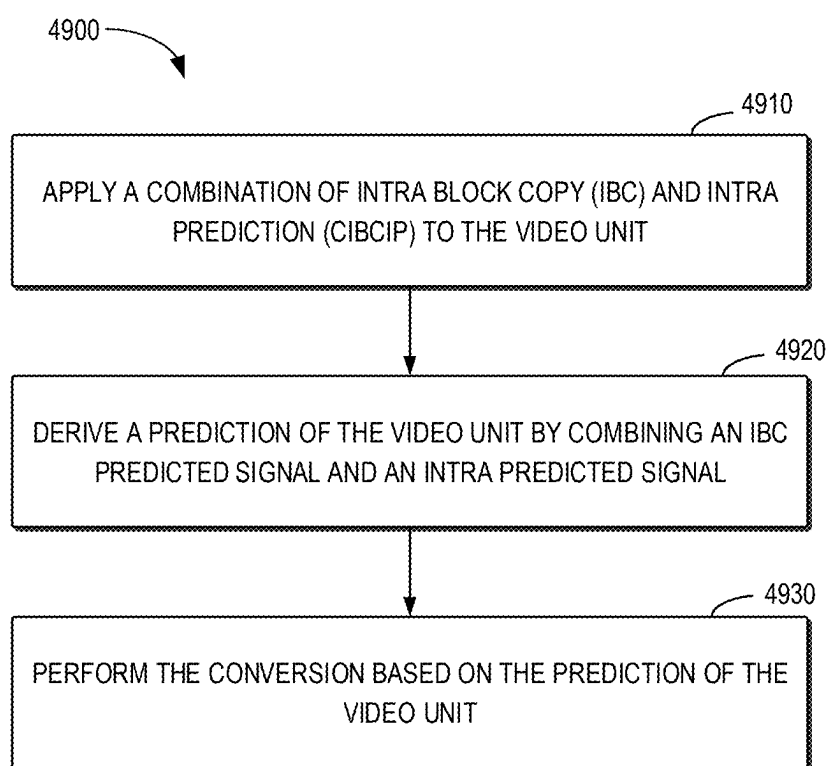


Fig. 49

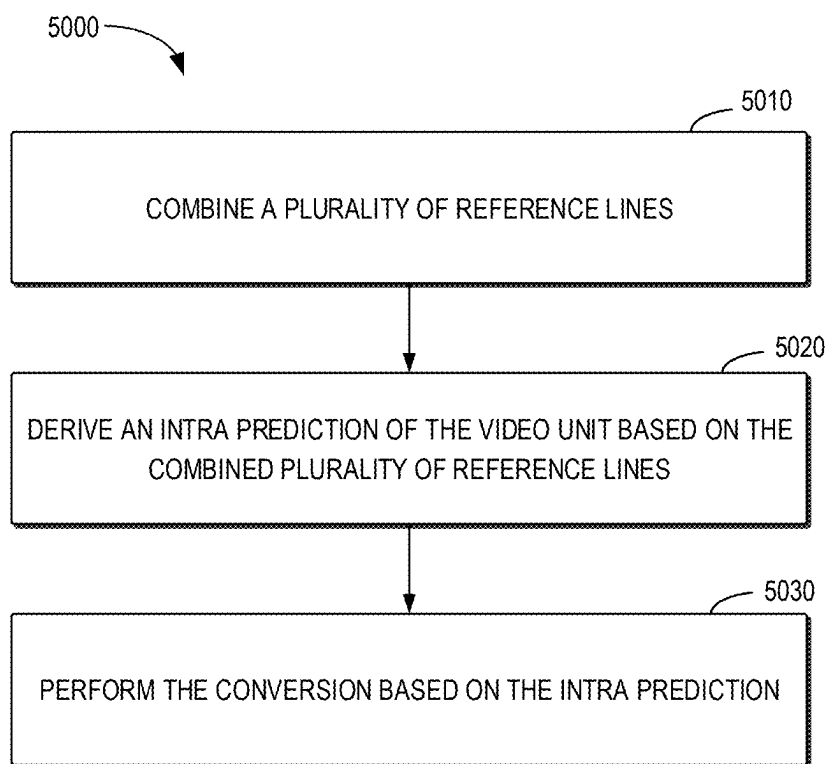


Fig. 50

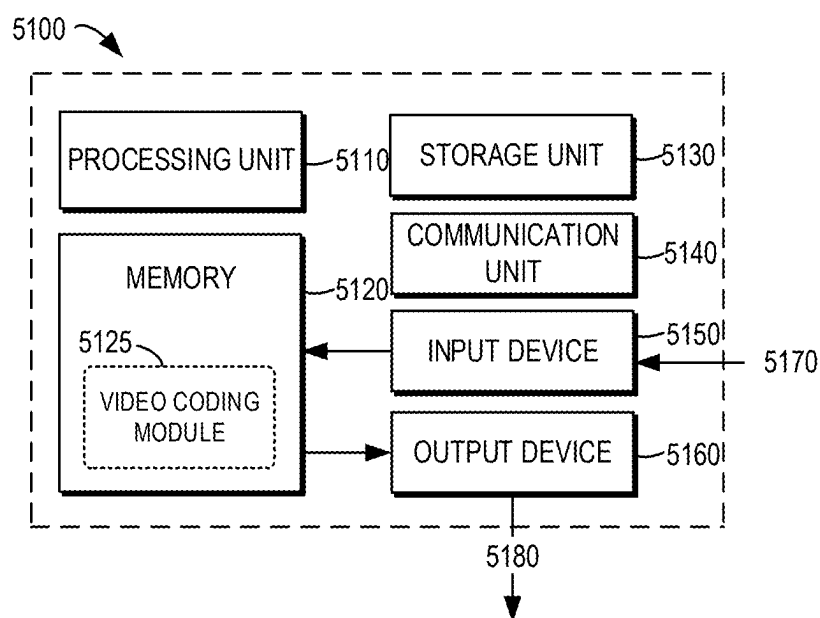


Fig. 51

## METHOD, APPARATUS, AND MEDIUM FOR VIDEO PROCESSING

### CROSS REFERENCE

**[0001]** This application is a continuation of International Application No. PCT/CN2023/108704, filed on Jul. 21, 2023, which claims the benefit of International Application No. PCT/CN2022/107183 filed on Jul. 21, 2022. The entire contents of these applications are hereby incorporated by reference in their entireties.

### FIELDS

**[0002]** Embodiments of the present disclosure relates generally to video processing techniques, and more particularly, to combined intra block copy and intra prediction.

### BACKGROUND

**[0003]** In nowadays, digital video capabilities are being applied in various aspects of peoples' lives. Multiple types of video compression technologies, such as MPEG-2, MPEG-4, ITU-T H.263, ITU-T H.264/MPEG-4 Part 10 Advanced Video Coding (AVC), ITU-T H.265 high efficiency video coding (HEVC) standard, versatile video coding (VVC) standard, have been proposed for video encoding/decoding. However, coding efficiency of video coding techniques is generally expected to be further improved.

### SUMMARY

**[0004]** Embodiments of the present disclosure provide a solution for video processing.

**[0005]** In a first aspect, a method for video processing is proposed. The method comprises: applying, for a conversion between a video unit of a video and a bitstream of the video unit, a combination of intra block copy (IBC) and intra prediction (CIBCIP) to the video unit; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and performing the conversion based on the prediction of the video unit. In this way, it can improve coding efficiency.

**[0006]** In a second aspect, another method for video processing is proposed. The method comprises: combining, for a conversion between a video unit of a video and a bitstream of the video unit, a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; and performing the conversion based on the intra prediction. In this way, it can improve coding efficiency.

**[0007]** In a third aspect, an apparatus for video processing is proposed. The apparatus comprises a processor and a non-transitory memory with instructions thereon. The instructions upon execution by the processor, cause the processor to perform a method in accordance with the first or second aspect of the present disclosure.

**[0008]** In a fourth aspect, a non-transitory computer-readable storage medium is proposed. The non-transitory computer-readable storage medium stores instructions that cause a processor to perform a method in accordance with the first or second aspect of the present disclosure.

**[0009]** In a fifth aspect, another non-transitory computer-readable recording medium is proposed. The non-transitory computer-readable recording medium stores a bitstream of a video which is generated by a method performed by an apparatus for video processing. The method comprises:

applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of the video; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and generating the bitstream based on the prediction of the video unit.

**[0010]** In a sixth aspect, a method for storing a bitstream of a video is proposed. The method comprises: applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of the video; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; generating the bitstream based on the prediction of the video unit; and storing the bitstream in a non-transitory computer-readable recording medium.

**[0011]** In a seventh aspect, another non-transitory computer-readable recording medium is proposed. The non-transitory computer-readable recording medium stores a bitstream of a video which is generated by a method performed by an apparatus for video processing. The method comprises: combining a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; and generating the bitstream based on the prediction of the video unit.

**[0012]** In an eighth aspect, a method for storing a bitstream of a video is proposed. The method comprises: combining a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; generating the bitstream based on the prediction of the video unit; and storing the bitstream in a non-transitory computer-readable recording medium.

**[0013]** This Summary is provided to introduce a selection of concepts in a simplified form that are further described below in the Detailed Description. This Summary is not intended to identify key features or essential features of the claimed subject matter, nor is it intended to be used to limit the scope of the claimed subject matter.

### BRIEF DESCRIPTION OF THE DRAWINGS

**[0014]** Through the following detailed description with reference to the accompanying drawings, the above and other objectives, features, and advantages of example embodiments of the present disclosure will become more apparent. In the example embodiments of the present disclosure, the same reference numerals usually refer to the same components.

**[0015]** FIG. 1 illustrates a block diagram that illustrates an example video coding system, in accordance with some embodiments of the present disclosure;

**[0016]** FIG. 2 illustrates a block diagram that illustrates a first example video encoder, in accordance with some embodiments of the present disclosure;

**[0017]** FIG. 3 illustrates a block diagram that illustrates an example video decoder, in accordance with some embodiments of the present disclosure;

**[0018]** FIG. 4 illustrates an example of encoder block diagram;

**[0019]** FIG. 5 shows 67 intra prediction modes;

**[0020]** FIG. 6 shows reference samples for wide-angular intra prediction;

**[0021]** FIG. 7 shows problem of discontinuity in case of directions beyond 45°;

**[0022]** FIG. 8 shows MMVD search point;

**[0023]** FIG. 9 is illustration for symmetrical MVD mode;

**[0024]** FIG. 10 shows extended CU region used in BDOP;

[0025] FIG. 11 shows control point based affine motion model;

[0026] FIG. 12 shows affine MVF per subblock;

[0027] FIG. 13 shows locations of inherited affine motion predictors;

[0028] FIG. 14 shows control point motion vector inheritance;

[0029] FIG. 15 shows locations of Candidates position for constructed affine merge mode;

[0030] FIG. 16 is an illustration of motion vector usage for proposed combined method;

[0031] FIG. 17 shows subblock MV VSB and pixel  $\Delta v(i, j)$ ;

[0032] FIG. 18A shows spatial neighboring blocks used by ATMVP;

[0033] FIG. 18B shows deriving sub-CU motion field by applying a motion shift from spatial neighbor and scaling the motion information from the corresponding collocated sub-CUs;

[0034] FIG. 19 shows location illumination compensation;

[0035] FIG. 20 shows no subsampling for the short side;

[0036] FIG. 21 shows decoding side motion vector refinement;

[0037] FIG. 22 shows diamond regions in the search area;

[0038] FIG. 23 shows positions of spatial merge candidate;

[0039] FIG. 24 shows candidate pairs considered for redundancy check of spatial merge candidates;

[0040] FIG. 25 is an illustrations of motion vector scaling for temporal merge candidate;

[0041] FIG. 26 shows candidate positions for temporal merge candidate, C0 and C1;

[0042] FIG. 27 shows VVC spatial neighboring blocks of the current block;

[0043] FIG. 28 is an illustration of virtual block in the  $i$ -th search round;

[0044] FIG. 29 shows examples of the GPM splits grouped by identical angles;

[0045] FIG. 30 shows uni-prediction MV selection for geometric partitioning mode;

[0046] FIG. 31 shows exemplified generation of a bending weight  $w_0$  using geometric partitioning mode;

[0047] FIG. 32 shows spatial neighboring blocks used to derive the spatial merge candidates;

[0048] FIG. 33 shows template matching performs on a search area around initial MV;

[0049] FIG. 34 is an illustration of sub-blocks where OBMC applies;

[0050] FIG. 35 shows SBT position, type and transform type;

[0051] FIG. 36 shows neighbouring samples used for calculating SAD;

[0052] FIG. 37 shows neighbouring samples used for calculating SAD for sub-CU level motion information;

[0053] FIG. 38 shows the sorting process;

[0054] FIG. 39 shows recorder process in encoder;

[0055] FIG. 40 shows reorder process in decoder;

[0056] FIG. 41 is an illustrations of the extended reference area;

[0057] FIG. 42 shows IBC reference region depending on current CU position;

[0058] FIG. 43 shows examples of symmetry in screen content pictures;

[0059] FIG. 44 (a) is an illustrations of BV adjustment for horizontal flip;

[0060] FIG. 44 (b) is an illustrations of BV adjustment for vertical flip;

[0061] FIG. 45 shows intra template matching search area used;

[0062] FIG. 46 shows an example of different numbers of samples in different reference lines for fusion;

[0063] FIG. 47 shows an example of different numbers of samples in different reference lines for fusion, and the samples surrounded by square box are discarded and not used for fusion of reference lines;

[0064] FIG. 48 shows an example of different numbers of samples in different reference lines for fusion, and the samples denoted by blank circle in reference  $L_n$  are padded and used for fusion of reference lines;

[0065] FIG. 49 illustrates a flowchart of a method for video processing in accordance with embodiments of the present disclosure;

[0066] FIG. 50 illustrates a flowchart of a method for video processing in accordance with embodiments of the present disclosure;

[0067] FIG. 51 illustrates a block diagram of a computing device in which various embodiments of the present disclosure can be implemented.

[0068] Throughout the drawings, the same or similar reference numerals usually refer to the same or similar elements.

## DETAILED DESCRIPTION

[0069] Principle of the present disclosure will now be described with reference to some embodiments. It is to be understood that these embodiments are described only for the purpose of illustration and help those skilled in the art to understand and implement the present disclosure, without suggesting any limitation as to the scope of the disclosure. The disclosure described herein can be implemented in various manners other than the ones described below.

[0070] In the following description and claims, unless defined otherwise, all technical and scientific terms used herein have the same meaning as commonly understood by one of ordinary skills in the art to which this disclosure belongs.

[0071] References in the present disclosure to “one embodiment,” “an embodiment,” “an example embodiment,” and the like indicate that the embodiment described may include a particular feature, structure, or characteristic, but it is not necessary that every embodiment includes the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an example embodiment, it is submitted that it is within the knowledge of one skilled in the art to affect such feature, structure, or characteristic in connection with other embodiments whether or not explicitly described.

[0072] It shall be understood that although the terms “first” and “second” etc. may be used herein to describe various elements, these elements should not be limited by these terms. These terms are only used to distinguish one element from another. For example, a first element could be termed a second element, and similarly, a second element could be termed a first element, without departing from the

scope of example embodiments. As used herein, the term “and/or” includes any and all combinations of one or more of the listed terms.

**[0073]** The terminology used herein is for the purpose of describing particular embodiments only and is not intended to be limiting of example embodiments. As used herein, the singular forms “a”, “an” and “the” are intended to include the plural forms as well, unless the context clearly indicates otherwise. It will be further understood that the terms “comprises”, “comprising”, “has”, “having”, “includes” and/or “including”, when used herein, specify the presence of stated features, elements, and/or components etc., but do not preclude the presence or addition of one or more other features, elements, components and/or combinations thereof.

#### Example Environment

**[0074]** FIG. 1 is a block diagram that illustrates an example video coding system 100 that may utilize the techniques of this disclosure. As shown, the video coding system 100 may include a source device 110 and a destination device 120. The source device 110 can be also referred to as a video encoding device, and the destination device 120 can be also referred to as a video decoding device. In operation, the source device 110 can be configured to generate encoded video data and the destination device 120 can be configured to decode the encoded video data generated by the source device 110. The source device 110 may include a video source 112, a video encoder 114, and an input/output (I/O) interface 116.

**[0075]** The video source 112 may include a source such as a video capture device. Examples of the video capture device include, but are not limited to, an interface to receive video data from a video content provider, a computer graphics system for generating video data, and/or a combination thereof.

**[0076]** The video data may comprise one or more pictures. The video encoder 114 encodes the video data from the video source 112 to generate a bitstream. The bitstream may include a sequence of bits that form a coded representation of the video data. The bitstream may include coded pictures and associated data. The coded picture is a coded representation of a picture. The associated data may include sequence parameter sets, picture parameter sets, and other syntax structures. The I/O interface 116 may include a modulator/demodulator and/or a transmitter. The encoded video data may be transmitted directly to destination device 120 via the I/O interface 116 through the network 130A. The encoded video data may also be stored onto a storage medium/server 130B for access by destination device 120.

**[0077]** The destination device 120 may include an I/O interface 126, a video decoder 124, and a display device 122. The I/O interface 126 may include a receiver and/or a modem. The I/O interface 126 may acquire encoded video data from the source device 110 or the storage medium/server 130B. The video decoder 124 may decode the encoded video data. The display device 122 may display the decoded video data to a user. The display device 122 may be integrated with the destination device 120, or may be external to the destination device 120 which is configured to interface with an external display device.

**[0078]** The video encoder 114 and the video decoder 124 may operate according to a video compression standard,

such as the High Efficiency Video Coding (HEVC) standard, Versatile Video Coding (VVC) standard and other current and/or further standards.

**[0079]** FIG. 2 is a block diagram illustrating an example of a video encoder 200, which may be an example of the video encoder 114 in the system 100 illustrated in FIG. 1, in accordance with some embodiments of the present disclosure.

**[0080]** The video encoder 200 may be configured to implement any or all of the techniques of this disclosure. In the example of FIG. 2, the video encoder 200 includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of the video encoder 200. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

**[0081]** In some embodiments, the video encoder 200 may include a partition unit 201, a predication unit 202 which may include a mode select unit 203, a motion estimation unit 204, a motion compensation unit 205 and an intra-prediction unit 206, a residual generation unit 207, a transform unit 208, a quantization unit 209, an inverse quantization unit 210, an inverse transform unit 211, a reconstruction unit 212, a buffer 213, and an entropy encoding unit 214.

**[0082]** In other examples, the video encoder 200 may include more, fewer, or different functional components. In an example, the predication unit 202 may include an intra block copy (IBC) unit. The IBC unit may perform predication in an IBC mode in which at least one reference picture is a picture where the current video block is located.

**[0083]** Furthermore, although some components, such as the motion estimation unit 204 and the motion compensation unit 205, may be integrated, but are represented in the example of FIG. 2 separately for purposes of explanation.

**[0084]** The partition unit 201 may partition a picture into one or more video blocks. The video encoder 200 and the video decoder 300 may support various video block sizes.

**[0085]** The mode select unit 203 may select one of the coding modes, intra or inter, e.g., based on error results, and provide the resulting intra-coded or inter-coded block to a residual generation unit 207 to generate residual block data and to a reconstruction unit 212 to reconstruct the encoded block for use as a reference picture. In some examples, the mode select unit 203 may select a combination of intra and inter predication (CIIP) mode in which the predication is based on an inter predication signal and an intra predication signal. The mode select unit 203 may also select a resolution for a motion vector (e.g., a sub-pixel or integer pixel precision) for the block in the case of inter-predication.

**[0086]** To perform inter prediction on a current video block, the motion estimation unit 204 may generate motion information for the current video block by comparing one or more reference frames from buffer 213 to the current video block. The motion compensation unit 205 may determine a predicted video block for the current video block based on the motion information and decoded samples of pictures from the buffer 213 other than the picture associated with the current video block.

**[0087]** The motion estimation unit 204 and the motion compensation unit 205 may perform different operations for a current video block, for example, depending on whether the current video block is in an I-slice, a P-slice, or a B-slice. As used herein, an “I-slice” may refer to a portion of a picture composed of macroblocks, all of which are based

upon macroblocks within the same picture. Further, as used herein, in some aspects, “P-slices” and “B-slices” may refer to portions of a picture composed of macroblocks that are not dependent on macroblocks in the same picture.

[0088] In some examples, the motion estimation unit 204 may perform uni-directional prediction for the current video block, and the motion estimation unit 204 may search reference pictures of list 0 or list 1 for a reference video block for the current video block. The motion estimation unit 204 may then generate a reference index that indicates the reference picture in list 0 or list 1 that contains the reference video block and a motion vector that indicates a spatial displacement between the current video block and the reference video block. The motion estimation unit 204 may output the reference index, a prediction direction indicator, and the motion vector as the motion information of the current video block. The motion compensation unit 205 may generate the predicted video block of the current video block based on the reference video block indicated by the motion information of the current video block.

[0089] Alternatively, in other examples, the motion estimation unit 204 may perform bi-directional prediction for the current video block. The motion estimation unit 204 may search the reference pictures in list 0 for a reference video block for the current video block and may also search the reference pictures in list 1 for another reference video block for the current video block. The motion estimation unit 204 may then generate reference indexes that indicate the reference pictures in list 0 and list 1 containing the reference video blocks and motion vectors that indicate spatial displacements between the reference video blocks and the current video block. The motion estimation unit 204 may output the reference indexes and the motion vectors of the current video block as the motion information of the current video block. The motion compensation unit 205 may generate the predicted video block of the current video block based on the reference video blocks indicated by the motion information of the current video block.

[0090] In some examples, the motion estimation unit 204 may output a full set of motion information for decoding processing of a decoder. Alternatively, in some embodiments, the motion estimation unit 204 may signal the motion information of the current video block with reference to the motion information of another video block. For example, the motion estimation unit 204 may determine that the motion information of the current video block is sufficiently similar to the motion information of a neighboring video block.

[0091] In one example, the motion estimation unit 204 may indicate, in a syntax structure associated with the current video block, a value that indicates to the video decoder 300 that the current video block has the same motion information as the another video block.

[0092] In another example, the motion estimation unit 204 may identify, in a syntax structure associated with the current video block, another video block and a motion vector difference (MVD). The motion vector difference indicates a difference between the motion vector of the current video block and the motion vector of the indicated video block. The video decoder 300 may use the motion vector of the indicated video block and the motion vector difference to determine the motion vector of the current video block.

[0093] As discussed above, video encoder 200 may predictively signal the motion vector. Two examples of predictive signaling techniques that may be implemented by video

encoder 200 include advanced motion vector predication (AMVP) and merge mode signaling.

[0094] The intra prediction unit 206 may perform intra prediction on the current video block. When the intra prediction unit 206 performs intra prediction on the current video block, the intra prediction unit 206 may generate prediction data for the current video block based on decoded samples of other video blocks in the same picture. The prediction data for the current video block may include a predicted video block and various syntax elements.

[0095] The residual generation unit 207 may generate residual data for the current video block by subtracting (e.g., indicated by the minus sign) the predicted video block(s) of the current video block from the current video block. The residual data of the current video block may include residual video blocks that correspond to different sample components of the samples in the current video block.

[0096] In other examples, there may be no residual data for the current video block for the current video block, for example in a skip mode, and the residual generation unit 207 may not perform the subtracting operation.

[0097] The transform processing unit 208 may generate one or more transform coefficient video blocks for the current video block by applying one or more transforms to a residual video block associated with the current video block.

[0098] After the transform processing unit 208 generates a transform coefficient video block associated with the current video block, the quantization unit 209 may quantize the transform coefficient video block associated with the current video block based on one or more quantization parameter (QP) values associated with the current video block.

[0099] The inverse quantization unit 210 and the inverse transform unit 211 may apply inverse quantization and inverse transforms to the transform coefficient video block, respectively, to reconstruct a residual video block from the transform coefficient video block. The reconstruction unit 212 may add the reconstructed residual video block to corresponding samples from one or more predicted video blocks generated by the predication unit 202 to produce a reconstructed video block associated with the current video block for storage in the buffer 213.

[0100] After the reconstruction unit 212 reconstructs the video block, loop filtering operation may be performed to reduce video blocking artifacts in the video block.

[0101] The entropy encoding unit 214 may receive data from other functional components of the video encoder 200. When the entropy encoding unit 214 receives the data, the entropy encoding unit 214 may perform one or more entropy encoding operations to generate entropy encoded data and output a bitstream that includes the entropy encoded data.

[0102] FIG. 3 is a block diagram illustrating an example of a video decoder 300, which may be an example of the video decoder 124 in the system 100 illustrated in FIG. 1, in accordance with some embodiments of the present disclosure.

[0103] The video decoder 300 may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. 3, the video decoder 300 includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of the video decoder 300. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.



[0104] In the example of FIG. 3, the video decoder 300 includes an entropy decoding unit 301, a motion compensation unit 302, an intra prediction unit 303, an inverse quantization unit 304, an inverse transformation unit 305, and a reconstruction unit 306 and a buffer 307. The video decoder 300 may, in some examples, perform a decoding pass generally reciprocal to the encoding pass described with respect to video encoder 200.

[0105] The entropy decoding unit 301 may retrieve an encoded bitstream. The encoded bitstream may include entropy coded video data (e.g., encoded blocks of video data). The entropy decoding unit 301 may decode the entropy coded video data, and from the entropy decoded video data, the motion compensation unit 302 may determine motion information including motion vectors, motion vector precision, reference picture list indexes, and other motion information. The motion compensation unit 302 may, for example, determine such information by performing the AMVP and merge mode. AMVP is used, including derivation of several most probable candidates based on data from adjacent PBs and the reference picture. Motion information typically includes the horizontal and vertical motion vector displacement values, one or two reference picture indices, and, in the case of prediction regions in B slices, an identification of which reference picture list is associated with each index. As used herein, in some aspects, a “merge mode” may refer to deriving the motion information from spatially or temporally neighboring blocks.

[0106] The motion compensation unit 302 may produce motion compensated blocks, possibly performing interpolation based on interpolation filters. Identifiers for interpolation filters to be used with sub-pixel precision may be included in the syntax elements.

[0107] The motion compensation unit 302 may use the interpolation filters as used by the video encoder 200 during encoding of the video block to calculate interpolated values for sub-integer pixels of a reference block. The motion compensation unit 302 may determine the interpolation filters used by the video encoder 200 according to the received syntax information and use the interpolation filters to produce predictive blocks.

[0108] The motion compensation unit 302 may use at least part of the syntax information to determine sizes of blocks used to encode frame(s) and/or slice(s) of the encoded video sequence, partition information that describes how each macroblock of a picture of the encoded video sequence is partitioned, modes indicating how each partition is encoded, one or more reference frames (and reference frame lists) for each inter-encoded block, and other information to decode the encoded video sequence. As used herein, in some aspects, a “slice” may refer to a data structure that can be decoded independently from other slices of the same picture, in terms of entropy coding, signal prediction, and residual signal reconstruction. A slice can either be an entire picture or a region of a picture.

[0109] The intra prediction unit 303 may use intra prediction modes for example received in the bitstream to form a prediction block from spatially adjacent blocks. The inverse quantization unit 304 inverse quantizes, i.e., de-quantizes, the quantized video block coefficients provided in the bitstream and decoded by entropy decoding unit 301. The inverse transform unit 305 applies an inverse transform.

[0110] The reconstruction unit 306 may obtain the decoded blocks, e.g., by summing the residual blocks with

the corresponding prediction blocks generated by the motion compensation unit 302 or intra-prediction unit 303. If desired, a deblocking filter may also be applied to filter the decoded blocks in order to remove blockiness artifacts. The decoded video blocks are then stored in the buffer 307, which provides reference blocks for subsequent motion compensation/intra prediction and also produces decoded video for presentation on a display device.

[0111] Some exemplary embodiments of the present disclosure will be described in detailed hereinafter. It should be understood that section headings are used in the present document to facilitate ease of understanding and do not limit the embodiments disclosed in a section to only that section. Furthermore, while certain embodiments are described with reference to Versatile Video Coding or other specific video codecs, the disclosed techniques are applicable to other video coding technologies also. Furthermore, while some embodiments describe video coding steps in detail, it will be understood that corresponding steps decoding that undo the coding will be implemented by a decoder. Furthermore, the term video processing encompasses video coding or compression, video decoding or decompression and video transcoding in which video pixels are represented from one compressed format into another compressed format or at a different compressed bitrate.

## 1. Brief Summary

[0112] The present disclosure is related to video coding technologies. Specifically, it is related to combined intra block copy, in which the reference (or prediction) block is obtained with samples in the current picture, and intra prediction, and other coding tools in image/video coding. It may be applied to the existing video coding standard like HEVC, or Versatile Video Coding (VVC). It may be also applicable to future video coding standards or video codec.

## 2. Introduction

[0113] Video coding standards have evolved primarily through the development of the well-known ITU-T and ISO/IEC standards. The ITU-T produced H.261 and H.263, ISO/IEC produced MPEG-1 and MPEG-4 Visual, and the two organizations jointly produced the H.262/MPEG-2 Video and H.264/MPEG-4 Advanced Video Coding (AVC) and H.265/HEVC standards. Since H.262, the video coding standards are based on the hybrid video coding structure wherein temporal prediction plus transform coding are utilized. To explore the future video coding technologies beyond HEVC, Joint Video Exploration Team (JVET) was founded by VCEG and MPEG jointly in 2015. Since then, many new methods have been adopted by JVET and put into the reference software named Joint Exploration Model (JEM). In April 2018, the Joint Video Expert Team (JVET) between VCEG (Q6/16) and ISO/IEC JTC1 SC29/WG11 (MPEG) was created to work on the VVC standard targeting at 50% bitrate reduction compared to HEVC. ITU-T VCEG (Q6/16) and ISO/IEC MPEG (JTC 1/SC 29/WG 5) are studying the potential need for standardization of future video coding technology with a compression capability that significantly exceeds that of the current VVC standard. Such future standardization action could either take the form of additional extension(s) of VVC or an entirely new standard. The groups are working together on this exploration activity in a joint collaboration effort known as the Joint Video

Exploration Team (JVET) to evaluate compression technology designs proposed by their experts in this area. New coding features and encoding methods implemented in Enhanced Compression Model (ECM) software that are under coordinated exploration study by the Joint Video Exploration Team (JVET) of ITU-T VCEG and ISO/IEC MPEG as potential enhanced video coding technology beyond the capabilities of VVC.

### 2.1. Coding Flow of a Typical Video Codec

[0114] FIG. 4 shows an example of encoder block diagram of VVC, which contains three in-loop filtering blocks: deblocking filter (DF), sample adaptive offset (SAO) and ALF. Unlike DF, which uses predefined filters, SAO and ALF utilize the original samples of the current picture to reduce the mean square errors between the original samples and the reconstructed samples by adding an offset and by applying a finite impulse response (FIR) filter, respectively, with coded side information signalling the offsets and filter coefficients. ALF is located at the last processing stage of each picture and can be regarded as a tool trying to catch and fix artifacts created by the previous stages.

### 2.2. Intra Mode Coding with 67 Intra Prediction Modes

[0115] To capture the arbitrary edge directions presented in natural video, the number of directional intra modes is extended from 33, as used in HEVC, to 65, as shown in FIG. 5, and the planar and DC modes remain the same. These denser directional intra prediction modes apply for all block sizes and for both luma and chroma intra predictions.

[0116] In the HEVC, every intra-coded block has a square shape and the length of each of its side is a power of 2. Thus, no division operations are required to generate an intra-predictor using DC mode. In VVC, blocks can have a rectangular shape that necessitates the use of a division operation per block in the general case. To avoid division operations for DC prediction, only the longer side is used to compute the average for non-square blocks.

#### 2.2.1. Wide Angle Intra Prediction

[0117] Although 67 modes are defined in the VVC, the exact prediction direction for a given intra prediction mode index is further dependent on the block shape. Conventional angular intra prediction directions are defined from 45 degrees to -135 degrees in clockwise direction. In VVC, several conventional angular intra prediction modes are adaptively replaced with wide-angle intra prediction modes for non-square blocks. The replaced modes are signalled using the original mode indexes, which are remapped to the indexes of wide angular modes after parsing. The total number of intra prediction modes is unchanged, i.e., 67, and the intra mode coding method is unchanged.

[0118] To support these prediction directions, the top reference with length  $2W+1$ , and the left reference with length  $2H+1$ , are defined as shown in FIG. 6.

[0119] The number of replaced modes in wide-angular direction mode depends on the aspect ratio of a block. The replaced intra prediction modes are illustrated in Table 2-1.

TABLE 2-1

| Intra prediction modes replaced by wide-angular modes |                                                              |
|-------------------------------------------------------|--------------------------------------------------------------|
| Aspect ratio                                          | Replaced intra prediction modes                              |
| W/H == 16                                             | Modes 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15         |
| W/H == 8                                              | Modes 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13                 |
| W/H == 4                                              | Modes 2, 3, 4, 5, 6, 7, 8, 9, 10, 11                         |
| W/H == 2                                              | Modes 2, 3, 4, 5, 6, 7, 8, 9                                 |
| W/H == 1                                              | None                                                         |
| W/H == 1/2                                            | Modes 59, 60, 61, 62, 63, 64, 65, 66                         |
| W/H == 1/4                                            | Modes 57, 58, 59, 60, 61, 62, 63, 64, 65, 66                 |
| W/H == 1/8                                            | Modes 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66         |
| W/H == 1/16                                           | Modes 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66 |

[0120] As shown in FIG. 7, two vertically adjacent predicted samples may use two non-adjacent reference samples in the case of wide-angle intra prediction. Hence, low-pass reference samples filter and side smoothing are applied to the wide-angle prediction to reduce the negative effect of the increased gap  $\Delta p_a$ . If a wide-angle mode represents a non-fractional offset. There are 8 modes in the wide-angle modes satisfy this condition, which are [-14, -12, -10, -6, 72, 76, 78, 80]. When a block is predicted by these modes, the samples in the reference buffer are directly copied without applying any interpolation. With this modification, the number of samples needed to be smoothing is reduced. Besides, it aligns the design of non-fractional modes in the conventional prediction modes and wide-angle modes. In VVC, 4:2:2 and 4:4:4 chroma formats are supported as well as 4:2:0. Chroma derived mode (DM) derivation table for 4:2:2 chroma format was initially ported from HEVC extending the number of entries from 35 to 67 to align with the extension of intra prediction modes. Since HEVC specification does not support prediction angle below -135 degree and above 45 degree, luma intra prediction modes ranging from 2 to 5 are mapped to 2. Therefore, chroma DM derivation table for 4:2:2: chroma format is updated by replacing some values of the entries of the mapping table to convert prediction angle more precisely for chroma blocks.

### 2.3. Inter Prediction

[0121] For each inter-predicted CU, motion parameters consisting of motion vectors, reference picture indices and reference picture list usage index, and additional information needed for the new coding feature of VVC to be used for inter-predicted sample generation. The motion parameter can be signalled in an explicit or implicit manner. When a CU is coded with skip mode, the CU is associated with one PU and has no significant residual coefficients, no coded motion vector delta or reference picture index. A merge mode is specified whereby the motion parameters for the current CU are obtained from neighbouring CUs, including spatial and temporal candidates, and additional schedules introduced in VVC. The merge mode can be applied to any inter-predicted CU, not only for skip mode. The alternative to merge mode is the explicit transmission of motion parameters, where motion vector, corresponding reference picture index for each reference picture list and reference picture list usage flag and other needed information are signalled explicitly per each CU.

#### 2.4. Intra Block Copy (IBC)

**[0122]** Intra block copy (IBC) is a tool adopted in HEVC extensions on SCC. It is well known that it significantly improves the coding efficiency of screen content materials. Since IBC mode is implemented as a block level coding mode, block matching (BM) is performed at the encoder to find the optimal block vector (or motion vector) for each CU. Here, a block vector is used to indicate the displacement from the current block to a reference block, which is already reconstructed inside the current picture. The luma block vector of an IBC-coded CU is in integer precision. The chroma block vector rounds to integer precision as well. When combined with AMVR, the IBC mode can switch between 1-pel and 4-pel motion vector precisions. An IBC-coded CU is treated as the third prediction mode other than intra or inter prediction modes. The IBC mode is applicable to the CUs with both width and height smaller than or equal to 64 luma samples.

**[0123]** At the encoder side, hash-based motion estimation is performed for IBC. The encoder performs RD check for blocks with either width or height no larger than 16 luma samples. For non-merge mode, the block vector search is performed using hash-based search first. If hash search does not return valid candidate, block matching based local search will be performed.

**[0124]** In the hash-based search, hash key matching (32-bit CRC) between the current block and a reference block is extended to all allowed block sizes. The hash key calculation for every position in the current picture is based on 4×4 sub-blocks. For the current block of a larger size, a hash key is determined to match that of the reference block when all the hash keys of all 4×4 sub-blocks match the hash keys in the corresponding reference locations. If hash keys of multiple reference blocks are found to match that of the current block, the block vector costs of each matched reference are calculated and the one with the minimum cost is selected.

**[0125]** In block matching search, the search range is set to cover both the previous and current CTUs.

**[0126]** At CU level, IBC mode is signalled with a flag and it can be signalled as IBC AMVP mode or IBC skip/merge mode as follows:

**[0127]** IBC skip/merge mode: a merge candidate index is used to indicate which of the block vectors in the list from neighbouring candidate IBC coded blocks is used to predict the current block. The merge list consists of spatial, HMVP, and pairwise candidates.

**[0128]** IBC AMVP mode: block vector difference is coded in the same way as a motion vector difference. The block vector prediction method uses two candidates as predictors, one from left neighbour and one from above neighbour (if IBC coded). When either neighbour is not available, a default block vector will be used as a predictor. A flag is signalled to indicate the block vector predictor index.

#### 2.5. IBC Motion Candidates

**[0129]** The term ‘block’ may represent a coding tree block (CTB), a coding tree unit (CTU), a coding block (CB), a CU, a PU, a TU, a PB, a TB or a video processing unit comprising multiple samples/pixels. A block may be rectangular or non-rectangular.

**[0130]** For an IBC coded block, a block vector (BV) is used to indicate the displacement from the current block to a reference block, which is already reconstructed inside the current picture.

**[0131]** W and H are the width and height of current block (e.g., luma block).

**[0132]** The non-adjacent spatial candidates of current coding block are adjacent spatial candidates of a virtual block in the *i*th search round (as shown in FIG. 9). The width and height of the virtual block for the *i*th search round are calculated by:  $\text{newWidth} = i \times 2 \times \text{gridX} + W$ ,  $\text{newHeight} = i \times 2 \times \text{gridY} + H$ . Obviously, the virtual block is the current block if the search round *i* is 0.

**[0133]** In the following, a BV predictor also is a BV candidate. The skip mode also is the merge mode.

**[0134]** The BV candidates can be divided into several groups according to some criterions. Each group is called a subgroup.

**[0135]** For example, we can take adjacent spatial and temporal BV candidates as a first subgroup and take the remaining BV candidates as a second subgroup; In another example, we can also take the first *N* ( $N \geq 2$ ) BV candidates as a first subgroup, take the following *M* ( $M \geq 2$ ) BV candidates as a second subgroup, and take the remaining BV candidates as a third subgroup.

#### 2.6. Merge mode with MVD (MMVD)

**[0136]** In addition to merge mode, where the implicitly derived motion information is directly used for prediction samples generation of the current CU, the merge mode with motion vector differences (MMVD) is introduced in VVC. A MMVD flag is signalled right after sending a regular merge flag to specify whether MMVD mode is used for a CU. In MMVD, after a merge candidate is selected, it is further refined by the signalled MVDs information. The further information includes a merge candidate flag, an index to specify motion magnitude, and an index for indication of motion direction. In MMVD mode, one for the first two candidates in the merge list is selected to be used as MV basis. The MMVD candidate flag is signalled to specify which one is used between the first and second merge candidates.

**[0137]** Distance index specifies motion magnitude information and indicate the pre-defined offset from the starting point. As shown in FIG. 8, an offset is added to either horizontal component or vertical component of starting MV. The relation of distance index and pre-defined offset is specified in Table 2-2.

TABLE 2-2

| The relation of distance index and pre-defined offset |              |     |   |   |   |   |    |    |
|-------------------------------------------------------|--------------|-----|---|---|---|---|----|----|
|                                                       | Distance IDX |     |   |   |   |   |    |    |
|                                                       | 0            | 1   | 2 | 3 | 4 | 5 | 6  | 7  |
| Offset (in unit of luma sample)                       | 1/4          | 1/2 | 1 | 2 | 4 | 8 | 16 | 32 |

**[0138]** Direction index represents the direction of the MVD relative to the starting point. The direction index can represent of the four directions as shown in Table 2-3. It's noted that the meaning of MVD sign could be variant according to the information of starting MVs. When the starting MVs is an un-prediction MV or bi-prediction MVs with both lists point to the same side of the current picture

(i.e. POCs of two references are both larger than the POC of the current picture, or are both smaller than the POC of the current picture), the sign in Table 2-3 specifies the sign of MV offset added to the starting MV. When the starting MVs is bi-prediction MVs with the two MVs point to the different sides of the current picture (i.e. the POC of one reference is larger than the POC of the current picture, and the POC of the other reference is smaller than the POC of the current picture), and the difference of POC in list **0** is greater than the one in list **1**, the sign in Table 2-3 specifies the sign of MV offset added to the list0 MV component of starting MV and the sign for the list1 MV has opposite value. Otherwise, if the difference of POC in list **1** is greater than list **0**, the sign in Table 2-3 specifies the sign of MV offset added to the list1 MV component of starting MV and the sign for the list0 MV has opposite value.

[0139] The MVD is scaled according to the difference of POCs in each direction. If the differences of POCs in both lists are the same, no scaling is needed. Otherwise, if the difference of POC in list **0** is larger than the one of list **1**, the MVD for list **1** is scaled, by defining the POC difference of L0 as td and POC difference of L1 as tb, described in FIG. 26. If the POC difference of L1 is greater than L0, the MVD for list **0** is scaled in the same way. If the starting MV is uni-predicted, the MVD is added to the available MV.

TABLE 2-3

| Sign of MV offset specified by direction index |     |     |     |     |
|------------------------------------------------|-----|-----|-----|-----|
| Direction IDX                                  | 00  | 01  | 10  | 11  |
| x-axis                                         | +   | −   | N/A | N/A |
| y-axis                                         | N/A | N/A | +   | −   |

### 2.7. Symmetric MVD coding

[0140] In VVC, besides the normal unidirectional prediction and bi-directional prediction mode MVD signalling, symmetric MVD mode for bi-predictional MVD signalling is applied. In the symmetric MVD mode, motion information including reference picture indices of both list-0 and list-1 and MVD of list-1 are not signaled but derived.

[0141] The decoding process of the symmetric MVD mode is as follows:

[0142] 1) At slice level, variables BiDirPredFlag, RefIdxSymL0 and RefIdxSymL1 are derived as follows:

[0143] If mvd\_11\_zero\_flag is 1, BiDirPredFlag is set equal to 0.

[0144] Otherwise, if the nearest reference picture in list-0 and the nearest reference picture in list-1 form a forward and backward pair of reference pictures or a backward and forward pair of reference pictures, BiDirPredFlag is set to 1, and both list-0 and list-1 reference pictures are short-term reference pictures. Otherwise BiDirPredFlag is set to 0.

[0145] 2) At CU level, a symmetrical mode flag indicating whether symmetrical mode is used or not is explicitly signaled if the CU is bi-prediction coded and BiDirPredFlag is equal to 1.

[0146] When the symmetrical mode flag is true, only mvp\_10\_flag, mvp\_11\_flag and MVD0 are explicitly signaled. The reference indices for list-0 and list-1 are set equal

to the pair of reference pictures, respectively. MVD1 is set equal to (−MVD0). The final motion vectors are shown in below formula.

$$\begin{cases} (mvx_0, mvy_0) = (mvp_{x_0} + mvd_{x_0}, mvp_{y_0} + mvd_{y_0}) \\ (mvx_1, mvy_1) = (mvp_{x_1} - mvd_{x_0}, mvp_{y_1} - mvd_{y_0}) \end{cases} \quad (2-1)$$

[0147] In the encoder, symmetric MVD motion estimation starts with initial MV evaluation. A set of initial MV candidates comprising of the MV obtained from uni-prediction search, the MV obtained from bi-prediction search and the MVs from the AMVP list. The one with the lowest rate-distortion cost is chosen to be the initial MV for the symmetric MVD motion search.

### 2.8. Bi-Directional Optical Flow (BDOF)

[0148] The bi-directional optical flow (BDOF) tool is included in VVC. BDOF, previously referred to as BIO, was included in the JEM. Compared to the JEM version, the BDOF in VVC is a simpler version that requires much less computation, especially in terms of number of multiplications and the size of the multiplier.

[0149] BDOF is used to refine the bi-prediction signal of a CU at the 4×4 subblock level. BDOF is applied to a CU if it satisfies all the following conditions:

[0150] The CU is coded using “true” bi-prediction mode, i.e., one of the two reference pictures is prior to the current picture in display order and the other is after the current picture in display order;

[0151] The distances (i.e. POC difference) from two reference pictures to the current picture are same;

[0152] Both reference pictures are short-term reference pictures;

[0153] The CU is not coded using affine mode or the SbTMVP merge mode;

[0154] CU has more than 64 luma samples;

[0155] Both CU height and CU width are larger than or equal to 8 luma samples;

[0156] BCW weight index indicates equal weight;

[0157] WP is not enabled for the current CU;

[0158] CIIP mode is not used for the current CU.

[0159] BDOF is only applied to the luma component. As its name indicates, the BDOF mode is based on the optical flow concept, which assumes that the motion of an object is smooth. For each 4×4 subblock, a motion refinement ( $v_x, v_y$ ) is calculated by minimizing the difference between the L0 and L1 prediction samples. The motion refinement is then used to adjust the bi-predicted sample values in the 4×4 subblock. The following steps are applied in the BDOF process.

[0160] First, the horizontal and vertical gradients,

$$\frac{\partial I^{(k)}}{\partial x}(i, j) \text{ and } \frac{\partial I^{(k)}}{\partial y}(i, j),$$

k=0,1, of the two prediction signals are computed by directly calculating the difference between two neighboring samples, i.e.,

$$\frac{\partial I^{(k)}}{\partial x}(i, j) = ((I^{(k)}(i+1, j) \gg \text{shift1}) - (I^{(k)}(i-1, j) \gg \text{shift1})) \quad (2-2)$$

$$\frac{\partial I^{(k)}}{\partial y}(i, j) = ((I^{(k)}(i, j+1) \gg \text{shift1}) - (I^{(k)}(i, j-1) \gg \text{shift1}))$$

where  $I^{(k)}(i, j)$  are the sample value at coordinate  $(i, j)$  of the prediction signal in list  $k$ ,  $k=0,1$ , and  $\text{shift1}$  is calculated based on the luma bit depth,  $\text{bitDepth}$ , as  $\text{shift1} = \max(6, \text{bitDepth}-6)$ .

**[0161]** Then, the auto- and cross-correlation of the gradients,  $S_1$ ,  $S_2$ ,  $S_3$ ,  $S_5$  and  $S_6$ , are calculated as

$$\begin{aligned} S_1 &= \sum_{(i, j) \in \Omega} \text{Abs}(\psi_x(i, j)), \\ S_3 &= \sum_{(i, j) \in \Omega} \theta(i, j) \cdot \text{Sign}(\psi_x(i, j)) \\ S_2 &= \sum_{(i, j) \in \Omega} \psi(i, j) \cdot \text{Sign}(\psi_y(i, j)) \\ S_5 &= \sum_{(i, j) \in \Omega} \text{Abs}(\psi_y(i, j)), \\ S_6 &= \sum_{(i, j) \in \Omega} \theta(i, j) \cdot \text{Sign}(\psi_y(i, j)) \end{aligned} \quad (2-3)$$

where

$$\begin{aligned} \psi_x(i, j) &= \left( \frac{\partial I^{(1)}}{\partial x}(i, j) + \frac{\partial I^{(0)}}{\partial x}(i, j) \right) \gg n_a \\ \psi_y(i, j) &= \left( \frac{\partial I^{(1)}}{\partial y}(i, j) + \frac{\partial I^{(0)}}{\partial y}(i, j) \right) \gg n_a \\ \theta(i, j) &= (I^{(1)}(i, j) \gg n_b) - (I^{(0)}(i, j) \gg n_b) \end{aligned} \quad (2-4)$$

where  $\Omega$  is a  $6 \times 6$  window around the  $4 \times 4$  subblock, and the values of  $n_a$  and  $n_b$  are set equal to  $\min(1, \text{bitDepth}-11)$  and  $\min(4, \text{bitDepth}-8)$ , respectively.

**[0162]** The motion refinement  $(v_x, v_y)$  is then derived using the cross- and auto-correlation terms using the following:

$$\begin{aligned} v_x &= S_1 > 0 ? \text{clip3}(-th'_{BIO}, th'_{BIO}, -((S_3 \cdot 2^{n_b-n_a}) \gg \lfloor \log_2 S_1 \rfloor)) : 0 \\ v_y &= S_5 > 0 ? \text{clip3}(-th_{BIO}, th_{BIO}, -((S_6 \cdot 2^{n_b-n_a} - (v_x S_{2,m}) \ll n_{S_2} + v_x S_{2,s})/2) \gg \lfloor \log_2 S_5 \rfloor)) : 0 \end{aligned} \quad (2-5)$$

where  $S_{2,m} = S_2 \gg n_{S_2}$ ,  $S_{2,s} = S_2 \& (2^{n_{S_2}} - 1)$ ,  $th_{BIO} = 2^{\max(S, BD-7)}$ ,  $\lfloor \cdot \rfloor$  is the floor function, and  $n_{S_2} = 12$ . Based on the motion refinement and the gradients, the following adjustment is calculated for each sample in the  $4 \times 4$  subblock:

$$b(x, y) = \text{rnd} \left( \left( \left( v_x \left( \frac{\partial I^{(1)}(x, y)}{\partial x} - \frac{\partial I^{(0)}(x, y)}{\partial x} \right) + v_y \left( \frac{\partial I^{(1)}(x, y)}{\partial y} - \frac{\partial I^{(0)}(x, y)}{\partial y} \right) + 1 \right) / 2 \right) \right) \quad (2-6)$$

**[0163]** Finally, the BDOF samples of the CU are calculated by adjusting the bi-prediction samples as follows:

$$\text{pred}_{BDOF}(x, y) = (I^{(0)}(x, y) + I^{(1)}(x, y) + b(x, y) + o_{offset}) \gg \text{shift} \quad (2-7)$$

**[0164]** These values are selected such that the multipliers in the BDOF process do not exceed 15-bit, and the maximum bit-width of the intermediate parameters in the BDOF process is kept within 32-bit.

**[0165]** In order to derive the gradient values, some prediction samples  $I^{(k)}(i, j)$  in list  $k$  ( $k=0,1$ ) outside of the current CU boundaries need to be generated. As depicted in FIG. 10, the BDOF in VVC uses one extended row/column around the CU's boundaries. In order to control the computational complexity of generating the out-of-boundary prediction samples, prediction samples in the extended area (white positions) are generated by taking the reference samples at the nearby integer positions (using floor ( ) operation on the coordinates) directly without interpolation, and the normal 8-tap motion compensation interpolation filter is used to generate prediction samples within the CU (gray positions). These extended sample values are used in gradient calculation only. For the remaining steps in the BDOF process, if any sample and gradient values outside of the CU boundaries are needed, they are padded (i.e. repeated) from their nearest neighbors.

**[0166]** When the width and/or height of a CU are larger than 16 luma samples, it will be split into subblocks with width and/or height equal to 16 luma samples, and the subblock boundaries are treated as the CU boundaries in the BDOF process. The maximum unit size for BDOF process is limited to  $16 \times 16$ . For each subblock, the BDOF process could be skipped. When the SAD of between the initial L0 and L1 prediction samples is smaller than a threshold, the BDOF process is not applied to the subblock. The threshold is set equal to  $(8 \cdot W \cdot (H > 1))$ , where  $W$  indicates the subblock width, and  $H$  indicates subblock height. To avoid the additional complexity of SAD calculation, the SAD between the initial L0 and L1 prediction samples calculated in DVMR process is re-used here.

**[0167]** If BCW is enabled for the current block, i.e., the BCW weight index indicates unequal weight, then bi-directional optical flow is disabled. Similarly, if WP is enabled for the current block, i.e., the luma\_weight\_lx\_flag is 1 for either of the two reference pictures, then BDOF is also disabled. When a CU is coded with symmetric MVD mode or CIIP mode, BDOF is also disabled.

## 2.9. Combined Inter and Intra Prediction (CIIP)

### 2.10. Affine Motion Compensated Prediction

**[0168]** In HEVC, only translation motion model is applied for motion compensation prediction (MCP). While in the real world, there are many kinds of motion, e.g., zoom in/out, rotation, perspective motions and the other irregular motions. In VVC, a block-based affine transform motion compensation prediction is applied. As shown FIG. 11, the affine motion field of the block is described by motion information of two control point (4-parameter) or three control point motion vectors (6-parameter).

[0169] For 4-parameter affine motion model, motion vector at sample location(x,y) in a block is derived as:

$$\begin{cases} mv_x = \frac{mv_{1x} - mv_{0x}}{W}x + \frac{mv_{1y} - mv_{0y}}{W}y + mv_{0x} \\ mv_y = \frac{mv_{1y} - mv_{0y}}{W}x + \frac{mv_{1x} - mv_{0x}}{W}y + mv_{0y} \end{cases} \quad (2-8)$$

[0170] For 6-parameter affine motion model, motion vector at sample location(x,y) in a block is derived as:

$$\begin{cases} mv_x = \frac{mv_{1x} - mv_{0x}}{W}x + \frac{mv_{2x} - mv_{0x}}{H}y + mv_{0x} \\ mv_y = \frac{mv_{1y} - mv_{0y}}{W}x + \frac{mv_{2y} - mv_{0y}}{H}y + mv_{0y} \end{cases} \quad (2-9)$$

[0171] Where (mv<sub>0x</sub>, mv<sub>0y</sub>) is motion vector of the top-left corner control point, (mv<sub>1x</sub>, mv<sub>1y</sub>) is motion vector of the top-right corner control point, and (mv<sub>2x</sub>, mv<sub>2y</sub>) is motion vector of the bottom-left corner control point.

[0172] In order to simplify the motion compensation prediction, block based affine transform prediction is applied. To derive motion vector of each 4×4 luma subblock, the motion vector of the center sample of each subblock, as shown in FIG. 12, is calculated according to above equations, and rounded to 1/16 fraction accuracy. Then the motion compensation interpolation filters are applied to generate the prediction of each subblock with derived motion vector. The subblock size of chroma-components is also set to be 4×4. The MV of a 4×4 chroma subblock is calculated as the average of the MVs of the four corresponding 4×4 luma subblocks.

[0173] As done for translational motion inter prediction, there are also two affine motion inter prediction modes: affine merge mode and affine AMVP mode.

#### 2.10.1. Affine Merge Prediction

[0174] AF\_MERGE mode can be applied for CUs with both width and height larger than or equal to 8. In this mode the CPMVs of the current CU is generated based on the motion information of the spatial neighbouring CUs. There can be up to five CPMVP candidates and an index is signalled to indicate the one to be used for the current CU. The following three types of CPMVP candidate are used to form the affine merge candidate list:

[0175] Inherited affine merge candidates that extrapolated from the CPMVs of the neighbour CUs

[0176] Constructed affine merge candidates CPMVPs that are derived using the translational MVs of the neighbour CUs

[0177] Zero MVs

[0178] In VVC, there are maximum two inherited affine candidates, which are derived from affine motion model of the neighbouring blocks, one from left neighbouring CUs and one from above neighbouring CUs. The candidate blocks are shown in FIG. 13. For the left predictor, the scan order is A0->A1, and for the above predictor, the scan order is B0->B1->B2. Only the first inherited candidate from each side is selected. No pruning check is performed between two inherited candidates. When a neighbouring affine CU is identified, its control point motion vectors are used to derive the CPMVP candidate in the affine merge list of the current CU. As shown in, if the neighbour left bottom block A is

coded in affine mode, the motion vectors v<sub>2</sub>, v<sub>3</sub> and v<sub>4</sub> of the top left corner, above right corner and left bottom corner of the CU which contains the block A are attained. When block A is coded with 4-parameter affine model, the two CPMVs of the current CU are calculated according to v<sub>2</sub>, and v<sub>3</sub>. In case that block A is coded with 6-parameter affine model, the three CPMVs of the current CU are calculated according to v<sub>2</sub>, v<sub>3</sub> and v<sub>4</sub>. Constructed affine candidate means the candidate is constructed by combining the neighbour translational motion information of each control point. The motion information for the control points is derived from the specified spatial neighbours and temporal neighbour shown in FIG. 15. CPMV<sub>k</sub> (k=1, 2, 3, 4) represents the k-th control point. For CPMV<sub>1</sub>, the B2->B3->A2 blocks are checked and the MV of the first available block is used. For CPMV<sub>2</sub>, the B1->B0 blocks are checked and for CPMV<sub>3</sub>, the A1->A0 blocks are checked. For TMVP is used as CPMV<sub>4</sub> if it's available.

[0179] After MVs of four control points are attained, affine merge candidates are constructed based on that motion information. The following combinations of control point MVs are used to construct in order: {CPMV<sub>1</sub>, CPMV<sub>2</sub>, CPMV<sub>3</sub>}, {CPMV<sub>1</sub>, CPMV<sub>2</sub>, CPMV<sub>4</sub>}, {CPMV<sub>1</sub>, CPMV<sub>3</sub>, CPMV<sub>4</sub>}, {CPMV<sub>2</sub>, CPMV<sub>3</sub>, CPMV<sub>4</sub>}, {CPMV<sub>1</sub>, CPMV<sub>2</sub>}, {CPMV<sub>1</sub>, CPMV<sub>3</sub>}

[0180] The combination of 3 CPMVs constructs a 6-parameter affine merge candidate and the combination of 2 CPMVs constructs a 4-parameter affine merge candidate. To avoid motion scaling process, if the reference indices of control points are different, the related combination of control point MVs is discarded.

[0181] After inherited affine merge candidates and constructed affine merge candidate are checked, if the list is still not full, zero MVs are inserted to the end of the list.

#### 2.10.2. Affine AMVP Prediction

[0182] Affine AMVP mode can be applied for CUs with both width and height larger than or equal to 16. An affine flag in CU level is signalled in the bitstream to indicate whether affine AMVP mode is used and then another flag is signalled to indicate whether 4-parameter affine or 6-parameter affine. In this mode, the difference of the CPMVs of current CU and their predictors CPMVPs is signalled in the bitstream. The affine AMVP candidate list size is 2 and it is generated by using the following four types of CPMVP candidate in order:

[0183] Inherited affine AMVP candidates that extrapolated from the CPMVs of the neighbour CUs

[0184] Constructed affine AMVP candidates CPMVPs that are derived using the translational MVs of the neighbour CUs

[0185] Translational MVs from neighbouring CUs

[0186] Zero MVs

[0187] The checking order of inherited affine AMVP candidates is same to the checking order of inherited affine merge candidates. The only difference is that, for AMVP candidate, only the affine CU that has the same reference picture as in current block is considered. No pruning process is applied when inserting an inherited affine motion predictor into the candidate list.

[0188] Constructed AMVP candidate is derived from the specified spatial neighbours shown in FIG. 15. The same checking order is used as done in affine merge candidate construction. In addition, reference picture index of the

neighbouring block is also checked. The first block in the checking order that is inter coded and has the same reference picture as in current CUs is used. There is only one When the current CU is coded with 4-parameter affine mode, and  $mv_0$  and  $mv_1$  are both available, they are added as one candidate in the affine AMVP list. When the current CU is coded with 6-parameter affine mode, and all three CPMVs are available, they are added as one candidate in the affine AMVP list. Otherwise, constructed AMVP candidate is set as unavailable.

**[0189]** If affine AMVP list candidates is still less than 2 after inherited affine AMVP candidates and Constructed AMVP candidate are checked,  $mv_0$ ,  $mv_1$ , and  $mv_2$  will be added, in order, as the translational MVs to predict all control point MVs of the current CU, when available. Finally, zero MVs are used to fill the affine AMVP list if it is still not full.

### 2.10.3. Affine Motion Information Storage

**[0190]** In VVC, the CPMVs of affine CUs are stored in a separate buffer. The stored CPMVs are only used to generate the inherited CPMVs in affine merge mode and affine AMVP mode for the lately coded CUs. The subblock MVs derived from CPMVs are used for motion compensation, MV derivation of merge/AMVP list of translational MVs and de-blocking.

**[0191]** To avoid the picture line buffer for the additional CPMVs, affine motion data inheritance from the CUs from above CTU is treated differently to the inheritance from the normal neighbouring CUs. If the candidate CU for affine motion data inheritance is in the above CTU line, the bottom-left and bottom-right subblock MVs in the line buffer instead of the CPMVs are used for the affine MVP derivation. In this way, the CPMVs are only stored in local buffer. If the candidate CU is 6-parameter affine coded, the affine model is degraded to 4-parameter model. As shown in FIG. 16, along the top CTU boundary, the bottom-left and bottom right subblock motion vectors of a CU are used for affine inheritance of the CUs in bottom CTUs.

### 2.10.4. Prediction Refinement with Optical Flow for Affine Mode

**[0192]** Subblock based affine motion compensation can save memory access bandwidth and reduce computation complexity compared to pixel-based motion compensation, at the cost of prediction accuracy penalty. To achieve a finer granularity of motion compensation, prediction refinement with optical flow (PROF) is used to refine the subblock based affine motion compensated prediction without increasing the memory access bandwidth for motion compensation. In VVC, after the subblock based affine motion compensation is performed, luma prediction sample is refined by adding a difference derived by the optical flow equation. The PROF is described as following four steps:

**[0193]** Step 1) The subblock-based affine motion compensation is performed to generate subblock prediction  $I(i,j)$ .

**[0194]** Step 2) The spatial gradients  $g_x(i,j)$  and  $g_y(i,j)$  of the subblock prediction are calculated at each sample location using a 3-tap filter  $[-1, 0, 1]$ . The gradient calculation is exactly the same as gradient calculation in BDOF.

$$g_x(i, j) = (I(i+1, j) \gg \text{shift1}) - (I(i-1, j) \gg \text{shift1}) \quad (2-10)$$

$$g_y(i, j) = (I(i, j+1) \gg \text{shift1}) - (I(i, j-1) \gg \text{shift1}) \quad (2-11)$$

shift1 is used to control the gradient's precision. The subblock (i.e. 4x4) prediction is extended by one sample on each side for the gradient calculation. To avoid additional memory bandwidth and additional interpolation computation, those extended samples on the extended borders are copied from the nearest integer pixel position in the reference picture.

**[0195]** Step 3) The luma prediction refinement is calculated by the following optical flow equation.

$$\Delta I(i, j) = g_x(i, j) * \Delta v_x(i, j) + g_y(i, j) * \Delta v_y(i, j) \quad (2-12)$$

where the  $\Delta v(i,j)$  is the difference between sample MV computed for sample location  $(i,j)$ , denoted by  $v(i,j)$ , and the subblock MV of the subblock to which sample  $(i,j)$  belongs, as shown in FIG. 17. The  $\Delta v(i,j)$  is quantized in the unit of 1/32 luma sample precision.

**[0196]** Since the affine model parameters and the sample location relative to the subblock center are not changed from subblock to subblock,  $\Delta v(i,j)$  can be calculated for the first subblock, and reused for other subblocks in the same CU. Let  $d_x(i,j)$  and  $d_y(i,j)$  be the horizontal and vertical offset from the sample location  $(i,j)$  to the center of the subblock  $(x_{SB}, y_{SB})$ ,  $\Delta v(x,y)$  can be derived by the following equation,

$$\begin{cases} dx(i, j) = i - x_{SB} \\ dy(i, j) = j - y_{SB} \end{cases} \quad (2-13)$$

$$\begin{cases} \Delta v_x(i, j) = C * dx(i, j) + D * dy(i, j) \\ \Delta v_y(i, j) = E * dx(i, j) + F * dy(i, j) \end{cases} \quad (2-14)$$

**[0197]** In order to keep accuracy, the enter of the subblock  $(x_{SB}, y_{SB})$  is calculated as  $((W_{SB}-1)/2, (H_{SB}-1)/2)$ , where  $W_{SB}$  and  $H_{SB}$  are the subblock width and height, respectively.

**[0198]** For 4-parameter affine model,

$$\begin{cases} C = F = \frac{v_{1x} - v_{0x}}{w} \\ E = -D = \frac{v_{1y} - v_{0y}}{w} \end{cases} \quad (2-15)$$

**[0199]** For 6-parameter affine model,

$$\begin{cases} C = \frac{v_{1x} - v_{0x}}{w} \\ D = \frac{v_{2x} - v_{0x}}{h} \\ E = \frac{v_{1y} - v_{0y}}{w} \\ F = \frac{v_{2y} - v_{0y}}{h} \end{cases} \quad (2-16)$$

where  $(v_{0x}, v_{0y})$ ,  $(v_{1x}, v_{1y})$ ,  $(v_{2x}, v_{2y})$  are the top-left, top-right and bottom-left control point motion vectors, w and h are the width and height of the CU.

**[0200]** Step 4) Finally, the luma prediction refinement  $\Delta I(i,j)$  is added to the subblock prediction  $I(i,j)$ . The final prediction  $I'$  is generated as the following equation.

$$I'(i, j) = I(i, j) + \Delta I(i, j) \quad (2-17)$$

**[0201]** PROF is not be applied in two cases for an affine coded CU: 1) all control point MVs are the same, which indicates the CU only has translational motion; 2) the affine motion parameters are greater than a specified limit because the subblock based affine MC is degraded to CU based MC to avoid large memory access bandwidth requirement. A fast encoding method is applied to reduce the encoding complexity of affine motion estimation with PROF. PROF is not applied at affine motion estimation stage in following two situations: a) if this CU is not the root block and its parent block does not select the affine mode as its best mode, PROF is not applied since the possibility for current CU to select the affine mode as best mode is low; b) if the magnitude of four affine parameters (C, D, E, F) are all smaller than a predefined threshold and the current picture is not a low delay picture, PROF is not applied because the improvement introduced by PROF is small for this case. In this way, the affine motion estimation with PROF can be accelerated.

#### 2.11. Subblock-Based Temporal Motion Vector Prediction (SbTMVP)

**[0202]** VVC supports the subblock-based temporal motion vector prediction (SbTMVP) method. Similar to the temporal motion vector prediction (TMVP) in HEVC, SbTMVP uses the motion field in the collocated picture to improve motion vector prediction and merge mode for CUs in the current picture. The same collocated picture used by TMVP is used for SbTMVP. SbTMVP differs from TMVP in the following two main aspects:

**[0203]** TMVP predicts motion at CU level, but SbTMVP predicts motion at sub-CU level;

**[0204]** Whereas TMVP fetches the temporal motion vectors from the collocated block in the collocated picture (the collocated block is the bottom-right or center block relative to the current CU), SbTMVP applies a motion shift before fetching the temporal motion information from the collocated picture, where the motion shift is obtained from the motion vector from one of the spatial neighboring blocks of the current CU.

**[0205]** The SbTMVP process is illustrated in FIG. 18A and FIG. 18B. SbTMVP predicts the motion vectors of the sub-CUs within the current CU in two steps. In the first step, the spatial neighbor A1 in FIG. 18A is examined. If A1 has a motion vector that uses the collocated picture as its reference picture, this motion vector is selected to be the motion shift to be applied. If no such motion is identified, then the motion shift is set to (0, 0).

**[0206]** In the second step, the motion shift identified in Step 1 is applied (i.e., added to the current block's coordinates) to obtain sub-CU-level motion information (motion vectors and reference indices) from the collocated picture as shown in FIG. 18B. The example in FIG. 18B assumes the motion shift is set to block A1's motion. Then, for each sub-CU, the motion information of its corresponding block (the smallest motion grid that covers the center sample) in the collocated picture is used to derive the motion information for the sub-CU. After the motion information of the collocated sub-CU is identified, it is converted to the motion vectors and reference indices of the current sub-CU in a similar way as the TMVP process of HEVC, where temporal motion scaling is applied to align the reference pictures of the temporal motion vectors to those of the current CU.

**[0207]** In VVC, a combined subblock based merge list which contains both SbTMVP candidate and affine merge candidates is used for the signalling of subblock based merge mode. The SbTMVP mode is enabled/disabled by a sequence parameter set (SPS) flag. If the SbTMVP mode is enabled, the SbTMVP predictor is added as the first entry of the list of subblock based merge candidates, and followed by the affine merge candidates. The size of subblock based merge list is signalled in SPS and the maximum allowed size of the subblock based merge list is 5 in VVC.

**[0208]** The sub-CU size used in SbTMVP is fixed to be 8x8, and as done for affine merge mode, SbTMVP mode is only applicable to the CU with both width and height are larger than or equal to 8.

**[0209]** The encoding logic of the additional SbTMVP merge candidate is the same as for the other merge candidates, that is, for each CU in P or B slice, an additional RD check is performed to decide whether to use the SbTMVP candidate.

#### 2.12. Adaptive Motion Vector Resolution (AMVR)

**[0210]** In HEVC, motion vector differences (MVDs) (between the motion vector and predicted motion vector of a CU) are signalled in units of quarter-luma-sample when use\_integer\_mv\_flag is equal to 0 in the slice header. In VVC, a CU-level adaptive motion vector resolution (AMVR) scheme is introduced. AMVR allows MVD of the CU to be coded in different precision. Dependent on the mode (normal AMVP mode or affine AMVP mode) for the current CU, the MVDs of the current CU can be adaptively selected as follows:

**[0211]** Normal AMVP mode: quarter-luma-sample, half-luma-sample, integer-luma-sample or four-luma-sample.

**[0212]** Affine AMVP mode: quarter-luma-sample, integer-luma-sample or 1/16 luma-sample.

**[0213]** The CU-level MVD resolution indication is conditionally signalled if the current CU has at least one non-zero MVD component. If all MVD components (that is, both horizontal and vertical MVDs for reference list L0 and reference list L1) are zero, quarter-luma-sample MVD resolution is inferred.

**[0214]** For a CU that has at least one non-zero MVD component, a first flag is signalled to indicate whether quarter-luma-sample MVD precision is used for the CU. If the first flag is 0, no further signaling is needed and quarter-luma-sample MVD precision is used for the current CU. Otherwise, a second flag is signalled to indicate half-luma-sample or other MVD precisions (integer or four-luma-sample) is used for normal AMVP CU. In the case of half-luma-sample, a 6-tap interpolation filter instead of the default 8-tap interpolation filter is used for the half-luma sample position. Otherwise, a third flag is signalled to indicate whether integer-luma-sample or four-luma-sample MVD precision is used for normal AMVP CU. In the case of affine AMVP CU, the second flag is used to indicate whether integer-luma-sample or 1/16 luma-sample MVD precision is used. In order to ensure the reconstructed MV has the intended precision (quarter-luma-sample, half-luma-sample, integer-luma-sample or four-luma-sample), the motion vector predictors for the CU will be rounded to the same precision as that of the MVD before being added together with the MVD. The motion vector predictors are rounded toward zero (that is, a negative motion vector predictor is rounded toward positive infinity and a positive motion vector predictor is rounded toward negative infinity). The encoder determines the motion vector resolution for the



current CU using RD check. To avoid always performing CU-level RD check four times for each MVD resolution, in VTM11, the RD check of MVD precisions other than quarter-luma-sample is only invoked conditionally. For normal AMVP mode, the RD cost of quarter-luma-sample MVD precision and integer-luma sample MV precision is computed first. Then, the RD cost of integer-luma-sample MVD precision is compared to that of quarter-luma-sample MVD precision to decide whether it is necessary to further check the RD cost of four-luma-sample MVD precision. When the RD cost for quarter-luma-sample MVD precision is much smaller than that of the integer-luma-sample MVD precision, the RD check of four-luma-sample MVD precision is skipped. Then, the check of half-luma-sample MVD precision is skipped if the RD cost of integer-luma-sample MVD precision is significantly larger than the best RD cost of previously tested MVD precisions. For affine AMVP mode, if affine inter mode is not selected after checking rate-distortion costs of affine merge/skip mode, merge/skip mode, quarter-luma-sample MVD precision normal AMVP mode and quarter-luma-sample MVD precision affine AMVP mode, then 1/16 luma-sample MV precision and 1-pel MV precision affine inter modes are not checked. Furthermore, affine parameters obtained in quarter-luma-sample MV precision affine inter mode is used as starting search point in 1/16 luma-sample and quarter-luma-sample MV precision affine inter modes.

#### 2.13. Bi-Prediction with CU-Level Weight (BCW)

[0215] In HEVC, the bi-prediction signal is generated by averaging two prediction signals obtained from two different reference pictures and/or using two different motion vectors. In VVC, the bi-prediction mode is extended beyond simple averaging to allow weighted averaging of the two prediction signals.

$$P_{bi-pred} = ((8 - w) * P_0 + w * P_1 + 4) \gg 3 \quad (2-18)$$

[0216] Five weights are allowed in the weighted averaging bi-prediction,  $w \in \{-2, 3, 4, 5, 10\}$ . For each bi-predicted CU, the weight  $w$  is determined in one of two ways: 1) for a non-merge CU, the weight index is signalled after the motion vector difference; 2) for a merge CU, the weight index is inferred from neighbouring blocks based on the merge candidate index. BCW is only applied to CUs with 256 or more luma samples (i.e., CU width times CU height is greater than or equal to 256). For low-delay pictures, all 5 weights are used. For non-low-delay pictures, only 3 weights ( $w \in \{3, 4, 5\}$ ) are used.

[0217] At the encoder, fast search algorithms are applied to find the weight index without significantly increasing the encoder complexity. These algorithms are summarized as follows. For further details readers are referred to the VTM software. When combined with AMVR, unequal weights are only conditionally checked for 1-pel and 4-pel motion vector precisions if the current picture is a low-delay picture.

[0218] When combined with affine, affine ME will be performed for unequal weights if and only if the affine mode is selected as the current best mode.

[0219] When the two reference pictures in bi-prediction are the same, unequal weights are only conditionally checked.

[0220] Unequal weights are not searched when certain conditions are met, depending on the POC distance

between current picture and its reference pictures, the coding QP, and the temporal level.

[0221] The BCW weight index is coded using one context coded bin followed by bypass coded bins. The first context coded bin indicates if equal weight is used; and if unequal weight is used, additional bins are signalled using bypass coding to indicate which unequal weight is used.

[0222] Weighted prediction (WP) is a coding tool supported by the H.264/AVC and HEVC standards to efficiently code video content with fading. Support for WP was also added into the VVC standard. WP allows weighting parameters (weight and offset) to be signalled for each reference picture in each of the reference picture lists L0 and L1. Then, during motion compensation, the weight(s) and offset(s) of the corresponding reference picture(s) are applied. WP and BCW are designed for different types of video content. In order to avoid interactions between WP and BCW, which will complicate VVC decoder design, if a CU uses WP, then the BCW weight index is not signalled, and  $w$  is inferred to be 4 (i.e. equal weight is applied). For a merge CU, the weight index is inferred from neighbouring blocks based on the merge candidate index. This can be applied to both normal merge mode and inherited affine merge mode. For constructed affine merge mode, the affine motion information is constructed based on the motion information of up to 3 blocks. The BCW index for a CU using the constructed affine merge mode is simply set equal to the BCW index of the first control point MV.

[0223] In VVC, CIIP and BCW cannot be jointly applied for a CU. When a CU is coded with CIIP mode, the BCW index of the current CU is set to 2, e.g., equal weight.

#### 2.14. Local Illumination Compensation (LIC)

[0224] Local illumination compensation (LIC) is a coding tool to address the issue of local illumination changes between current picture and its temporal reference pictures. The LIC is based on a linear model where a scaling factor and an offset are applied to the reference samples to obtain the prediction samples of a current block. Specifically, the LIC can be mathematically modeled by the following equation:

$$P(x, y) = \alpha \cdot P_r(x + v_x, y + v_y) + \beta$$

where  $P(x, y)$  is the prediction signal of the current block at the coordinate  $(x, y)$ ;  $P_r(x + v_x, y + v_y)$  is the reference block pointed by the motion vector  $(v_x, v_y)$ ;  $\alpha$  and  $\beta$  are the corresponding scaling factor and offset that are applied to the reference block. FIG. 19 illustrates the LIC process. In FIG. 19, when the LIC is applied for a block, a least mean square error (LMSE) method is employed to derive the values of the LIC parameters (i.e.,  $\alpha$  and  $\beta$ ) by minimizing the difference between the neighboring samples of the current block (i.e., the template  $T$  in FIG. 19) and their corresponding reference samples in the temporal reference pictures (i.e., either  $T_0$  or  $T_1$  in FIG. 19). Additionally, to reduce the computational complexity, both the template samples and the reference template samples are subsampled (adaptive subsampling) to derive the LIC parameters, i.e., only the shaded samples in FIG. 19 are used to derive  $\alpha$  and  $\beta$ .

[0225] To improve the coding performance, no subsampling for the short side is performed as shown in FIG. 20.

### 2.15. Decoder Side Motion Vector Refinement (DMVR)

[0226] In order to increase the accuracy of the MVs of the merge mode, a bilateral-matching (BM) based decoder side motion vector refinement is applied in VVC. In bi-prediction operation, a refined MV is searched around the initial MVs in the reference picture list L0 and reference picture list L1. The BM method calculates the distortion between the two candidate blocks in the reference picture list L0 and list L1. As illustrated in FIG. 21, the SAD between the two blocks based on each MV candidate (e.g., MV0' and MV1') around the initial MV is calculated. The MV candidate with the lowest SAD becomes the refined MV and used to generate the bi-predicted signal.

[0227] In VVC, the application of DMVR is restricted and is only applied for the CUs which are coded with following modes and features:

[0228] CU level merge mode with bi-prediction MV

[0229] One reference picture is in the past and another reference picture is in the future with respect to the current picture

[0230] The distances (i.e. POC difference) from two reference pictures to the current picture are same

[0231] Both reference pictures are short-term reference pictures

[0232] CU has more than 64 luma samples

[0233] Both CU height and CU width are larger than or equal to 8 luma samples

[0234] BCW weight index indicates equal weight

[0235] WP is not enabled for the current block

[0236] CIIP mode is not used for the current block

[0237] The refined MV derived by DMVR process is used to generate the inter prediction samples and also used in temporal motion vector prediction for future pictures coding. While the original MV is used in deblocking process and also used in spatial motion vector prediction for future CU coding.

[0238] The additional features of DMVR are mentioned in the following sub-clauses.

#### 2.15.1. Searching Scheme

[0239] In DMVR, the search points are surrounding the initial MV and the MV offset obey the MV difference mirroring rule. In other words, any points that are checked by DMVR, denoted by candidate MV pair (MV0, MV1) obey the following two equations:

$$MV0' = MV0 + MV\_offset \quad (2-19)$$

$$MV1' = MV1 - MV\_offset \quad (2-20)$$

[0240] Where MV\_offset represents the refinement offset between the initial MV and the refined MV in one of the reference pictures. The refinement search range is two integer luma samples from the initial MV. The searching includes the integer sample offset search stage and fractional sample refinement stage.

[0241] 25 points full search is applied for integer sample offset searching. The SAD of the initial MV pair is first calculated. If the SAD of the initial MV pair is smaller than a threshold, the integer sample stage of DMVR is terminated. Otherwise SADs of the remaining 24 points are calculated and checked in raster scanning order. The point with the smallest SAD is selected as the output of integer sample offset searching stage. To reduce the penalty of the

uncertainty of DMVR refinement, it is proposed to favor the original MV during the DMVR process. The SAD between the reference blocks referred by the initial MV candidates is decreased by 1/4 of the SAD value.

[0242] The integer sample search is followed by fractional sample refinement. To save the calculational complexity, the fractional sample refinement is derived by using parametric error surface equation, instead of additional search with SAD comparison. The fractional sample refinement is conditionally invoked based on the output of the integer sample search stage. When the integer sample search stage is terminated with center having the smallest SAD in either the first iteration or the second iteration search, the fractional sample refinement is further applied.

[0243] In parametric error surface based sub-pixel offsets estimation, the center position cost and the costs at four neighboring positions from the center are used to fit a 2-D parabolic error surface equation of the following form

$$E(x, y) = A(x - x_{min})^2 + B(y - y_{min})^2 + C \quad (2-21)$$

where  $(x_{min}, y_{min})$  corresponds to the fractional position with the least cost and C corresponds to the minimum cost value. By solving the above equations by using the cost value of the five search points, the  $(x_{min}, y_{min})$  is computed as:

$$x_{min} = (E(-1, 0) - E(1, 0)) / (2(E(-1, 0) + E(1, 0) - 2E(0, 0))) \quad (2-22)$$

$$y_{min} = (E(0, -1) - E(0, 1)) / (2(E(0, -1) + E(0, 1) - 2E(0, 0))) \quad (2-23)$$

[0244] The value of  $x_{min}$  and  $y_{min}$  are automatically constrained to be between -8 and 8 since all cost values are positive and the smallest value is  $E(0,0)$ . This corresponds to half pel offset with 1/16th-pel MV accuracy in VVC. The computed fractional  $(x_{min}, y_{min})$  are added to the integer distance refinement MV to get the sub-pixel accurate refinement delta MV.

#### 2.15.2. Bilinear-Interpolation and Sample Padding

[0245] In VVC, the resolution of the MVs is 1/16 luma samples. The samples at the fractional position are interpolated using an 8-tap interpolation filter. In DMVR, the search points are surrounding the initial fractional-pel MV with integer sample offset, therefore the samples of those fractional position need to be interpolated for DMVR search process. To reduce the calculation complexity, the bi-linear interpolation filter is used to generate the fractional samples for the searching process in DMVR. Another important effect is that by using bi-linear filter is that with 2-sample search range, the DMVR does not access more reference samples compared to the normal motion compensation process. After the refined MV is attained with DMVR search process, the normal 8-tap interpolation filter is applied to generate the final prediction. In order to not access more reference samples to normal MC process, the samples, which is not needed for the interpolation process based on the original MV but is needed for the interpolation process based on the refined MV, will be padded from those available samples.

#### 2.15.3. Maximum DMVR Processing Unit

[0246] When the width and/or height of a CU are larger than 16 luma samples, it will be further split into subblocks

with width and/or height equal to 16 luma samples. The maximum unit size for DMVR searching process is limit to 16×16.

## 2.16. Multi-Pass Decoder-Side Motion Vector Refinement

**[0247]** In this contribution, a multi-pass decoder-side motion vector refinement is applied instead of DMVR. In the first pass, bilateral matching (BM) is applied to a coding block. In the second pass, BM is applied to each 16×16 subblock within the coding block. In the third pass, MV in each 8×8 subblock is refined by applying bi-directional optical flow (BDOF). The refined MVs are stored for both spatial and temporal motion vector prediction.

### 2.16.1. First Pass-Block Based Bilateral Matching MV Refinement

**[0248]** In the first pass, a refined MV is derived by applying BM to a coding block. Similar to decoder-side motion vector refinement (DMVR), the refined MV is searched around the two initial MVs (MV0 and MV1) in the reference picture lists L0 and L1. The refined MVs (MV0\_pass1 and MV1\_pass1) are derived around the initiate MVs based on the minimum bilateral matching cost between the two reference blocks in L0 and L1.

**[0249]** BM performs local search to derive integer sample precision intDeltaMV and half-pel sample precision half-DeltaMv. The local search applies a 3×3 square search pattern to loop through the search range [−sHor, sHor] in a horizontal direction and [−sVer, sVer] in a vertical direction, wherein, the values of sHor and sVer are determined by the block dimension, and the maximum value of sHor and sVer is 8.

**[0250]** The bilateral matching cost is calculated as:  $\text{bilCost} = \text{mvDistanceCost} + \text{sadCost}$ . When the block size  $\text{cbW} * \text{cbH}$  is greater than 64, MRSAD cost function is applied to remove the DC effect of the distortion between the reference blocks. When the bilCost at the center point of the 3×3 search pattern has the minimum cost, the intDeltaMV or halfDeltaMV local search is terminated. Otherwise, the current minimum cost search point becomes the new center point of the 3×3 search pattern and the search for the minimum cost continues, until it reaches the end of the search range.

**[0251]** The existing fractional sample refinement is further applied to derive the final deltaMV. The refined MVs after the first pass are then derived as:

$$\text{MV0\_pass1} = \text{MV0} + \text{deltaMV}$$

$$\text{MV1\_pass1} = \text{MV1} - \text{deltaMV}$$

**2.16.2. Second Pass-Subblock Based Bilateral Matching MV Refinement** In the second pass, a refined MV is derived by applying BM to a 16×16 grid subblock. For each subblock, the refined MV is searched around the two MVs (MV0\_pass1 and MV1\_pass1), obtained on the first pass for the reference picture list L0 and L1. The refined MVs (MV0\_pass2 (sblIdx2) and MV1\_pass2 (sblIdx2)) are derived based on the minimum bilateral matching cost between the two reference subblocks in L0 and L1.

**[0252]** For each subblock, BM performs full search to derive integer sample precision intDeltaMV. The full search has a search range [−sHor, sHor] in a horizontal direction and [−sVer, sVer] in a vertical direction, wherein, the values of sHor and sVer are determined by the block dimension,

and the maximum value of sHor and sVer is 8. The bilateral matching cost is calculated by applying a cost factor to the SATD cost between the two reference subblocks, as:  $\text{bilCost} = \text{satdCost} * \text{costFactor}$ . The search area  $(2 * \text{sHor} + 1) * (2 * \text{sVer} + 1)$  is divided up to 5 diamond shape search regions shown on FIG. 22. Each search region is assigned a cost-Factor, which is determined by the distance (intDeltaMV) between each search point and the starting MV, and each diamond region is processed in the order starting from the center of the search area. In each region, the search points are processed in the raster scan order starting from the top left going to the bottom right corner of the region. When the minimum bilCost within the current search region is less than a threshold equal to  $\text{sbW} * \text{sbH}$ , the int-pel full search is terminated, otherwise, the int-pel full search continues to the next search region until all search points are examined.

**[0253]** BM performs local search to derive half sample precision halfDeltaMv. The search pattern and cost function are the same as defined in 2.9.1.

**[0254]** The existing VVC DMVR fractional sample refinement is further applied to derive the final deltaMV (sblIdx2). The refined MVs at second pass is then derived as:

$$\text{MV0\_pass2}(\text{sblIdx2}) = \text{MV0\_pass1} + \text{deltaMV}(\text{sblIdx2})$$

$$\text{MV1\_pass2}(\text{sblIdx2}) = \text{MV1\_pass1} - \text{deltaMV}(\text{sblIdx2})$$

### 2.16.3. Third Pass-Subblock Based Bi-Directional Optical Flow MV Refinement

**[0255]** In the third pass, a refined MV is derived by applying BDOF to an 8×8 grid subblock. For each 8×8 subblock, BDOF refinement is applied to derive scaled Vx and Vy without clipping starting from the refined MV of the parent subblock of the second pass. The derived bioMv(Vx, Vy) is rounded to 1/16 sample precision and clipped between −32 And 32.

**[0256]** The refined MVs (MV0\_pass3(sblIdx3) and MV1\_pass3(sblIdx3)) at third pass are derived as:

$$\text{MV0\_pass3}(\text{sblIdx3}) = \text{MV0\_pass2}(\text{sblIdx2}) + \text{bioMv}$$

$$\text{MV1\_pass3}(\text{sblIdx3}) = \text{MV0\_pass2}(\text{sblIdx2}) - \text{bioMv}$$

## 2.17. Sample-Based BDOF

**[0257]** In the sample-based BDOF, instead of deriving motion refinement (Vx, vy) on a block basis, it is performed per sample.

**[0258]** The coding block is divided into 8×8 subblocks. For each subblock, whether to apply BDOF or not is determined by checking the SAD between the two reference subblocks against a threshold. If decided to apply BDOF to a subblock, for every sample in the subblock, a sliding 5×5 window is used and the existing BDOF process is applied for every sliding window to derive Vx and vy. The derived motion refinement (Vx, vy) is applied to adjust the bi-predicted sample value for the center sample of the window.

## 2.18. Extended Merge Prediction

**[0259]** In VVC, the merge candidate list is constructed by including the following five types of candidates in order:

- [0260] (1) Spatial MVP from spatial neighbour CUs
- [0261] (2) Temporal MVP from collocated CUs
- [0262] (3) History-based MVP from a FIFO table
- [0263] (4) Pairwise average MVP
- [0264] (5) Zero MVs.

[0265] The size of merge list is signalled in sequence parameter set header and the maximum allowed size of merge list is 6. For each CU code in merge mode, an index of best merge candidate is encoded using truncated unary binarization (TU). The first bin of the merge index is coded with context and bypass coding is used for other bins. The derivation process of each category of merge candidates is provided in this session. As done in HEVC, VVC also supports parallel derivation of the merging candidate lists for all CUs within a certain size of area.

#### 2.18.1. Spatial Candidates Derivation

[0266] The derivation of spatial merge candidates in VVC is same to that in HEVC except the positions of first two merge candidates are swapped. A maximum of four merge candidates are selected among candidates located in the positions depicted in. The order of derivation is B0, A0, B1, A1 and B2. Position B2 is considered only when one or more than one CUs of position B0, A0, B1, A1 are not available (e.g. because it belongs to another slice or tile) or is intra coded. After candidate at position A1 is added, the addition of the remaining candidates is subject to a redundancy check which ensures that candidates with same motion information are excluded from the list so that coding efficiency is improved. To reduce computational complexity, not all possible candidate pairs are considered in the mentioned redundancy check. Instead only the pairs linked with an arrow in FIG. 24 are considered and a candidate is only added to the list if the corresponding candidate used for redundancy check has not the same motion information.

#### 2.18.2. Temporal Candidates Derivation

[0267] In this step, only one candidate is added to the list. Particularly, in the derivation of this temporal merge candidate, a scaled motion vector is derived based on co-located CU belonging to the collocated reference picture. The reference picture list to be used for derivation of the co-located CU is explicitly signalled in the slice header. The scaled motion vector for temporal merge candidate is obtained as illustrated by the dotted line in FIG. 25, which is scaled from the motion vector of the co-located CU using the POC distances,  $tb$  and  $td$ , where  $tb$  is defined to be the POC difference between the reference picture of the current picture and the current picture and  $td$  is defined to be the POC difference between the reference picture of the co-located picture and the co-located picture. The reference picture index of temporal merge candidate is set equal to zero.

[0268] The position for the temporal candidate is selected between candidates C0 and C1, as depicted in FIG. 26. If CU at position C0 is not available, is intra coded, or is outside of the current row of CTUs, position C1 is used. Otherwise, position C0 is used in the derivation of the temporal merge candidate.

#### 2.18.3. History-Based Merge Candidates Derivation

[0269] The history-based MVP(HMVP) merge candidates are added to merge list after the spatial MVP and TMVP. In this method, the motion information of a previously coded block is stored in a table and used as MVP for the current CU. The table with multiple HMVP candidates is main-

tained during the encoding/decoding process. The table is reset (emptied) when a new CTU row is encountered. Whenever there is a non-subblock inter-coded CU, the associated motion information is added to the last entry of the table as a new HMVP candidate.

[0270] The HMVP table size  $S$  is set to be 6, which indicates up to 6 History-based MVP(HMVP) candidates may be added to the table. When inserting a new motion candidate to the table, a constrained first-in-first-out (FIFO) rule is utilized wherein redundancy check is firstly applied to find whether there is an identical HMVP in the table. If found, the identical HMVP is removed from the table and all the HMVP candidates afterwards are moved forward, HMVP candidates could be used in the merge candidate list construction process. The latest several HMVP candidates in the table are checked in order and inserted to the candidate list after the TMVP candidate. Redundancy check is applied on the HMVP candidates to the spatial or temporal merge candidate.

[0271] To reduce the number of redundancy check operations, the following simplifications are introduced: Number of HMPV candidates is used for merge list generation is set as  $(N \leq 4)? M: (8-N)$ , wherein  $N$  indicates number of existing candidates in the merge list and  $M$  indicates number of available HMVP candidates in the table.

[0272] Once the total number of available merge candidates reaches the maximally allowed merge candidates minus 1, the merge candidate list construction process from HMVP is terminated.

#### 2.18.4. Pair-Wise Average Merge Candidates Derivation

[0273] Pairwise average candidates are generated by averaging predefined pairs of candidates in the existing merge candidate list, and the predefined pairs are defined as  $\{(0, 1), (0, 2), (1, 2), (0, 3), (1, 3), (2, 3)\}$ , where the numbers denote the merge indices to the merge candidate list. The averaged motion vectors are calculated separately for each reference list. If both motion vectors are available in one list, these two motion vectors are averaged even when they point to different reference pictures; if only one motion vector is available, use the one directly; if no motion vector is available, keep this list invalid.

[0274] When the merge list is not full after pair-wise average merge candidates are added, the zero MVPs are inserted in the end until the maximum merge candidate number is encountered.

#### 2.18.5. Merge Estimation Region

[0275] Merge estimation region (MER) allows independent derivation of merge candidate list for the CUs in the same merge estimation region (MER). A candidate block that is within the same MER to the current CU is not included for the generation of the merge candidate list of the current CU. In addition, the updating process for the history-based motion vector predictor candidate list is updated only if  $(xCb+cbWidth) \gg \text{Log2ParMrgLevel}$  is greater than  $xCb \gg \text{Log2ParMrgLevel}$  and  $(yCb+cbHeight) \gg \text{Log2ParMrgLevel}$  is greater than  $yCb \gg \text{Log2ParMrgLevel}$  and where  $(xCb, yCb)$  is the top-left luma sample position of the current CU in the picture and  $(cbWidth, cbHeight)$  is the CU size. The MER size is selected at encoder side and signalled as  $\text{log\_2\_parallel\_merge\_level\_minus2}$  in the sequence parameter set.

## 2.19. New Merge Candidates

### 2.19.1. Non-Adjacent Merge Candidates Derivation

[0276] In VVC, five spatially neighboring blocks shown in FIG. 27 as well as one temporal neighbor are used to derive merge candidates.

[0277] It is proposed to derive the additional merge candidates from the positions non-adjacent to the current block using the same pattern as that in VVC. To achieve this, for each search round  $i$ , a virtual block is generated based on the current block as follows:

[0278] First, the relative position of the virtual block to the current block is calculated by:

[0279]  $\text{Offset}_x = -i \times \text{grid}_X$ ,  $\text{Offset}_y = -i \times \text{grid}_Y$

[0280] where the  $\text{Offset}_x$  and  $\text{Offset}_y$  denote the offset of the top-left corner of the virtual block relative to the top-left corner of the current block,  $\text{grid}_X$  and  $\text{grid}_Y$  are the width and height of the search grid.

[0281] Second, the width and height of the virtual block are calculated by:

[0282]  $\text{newWidth} = i \times 2 \times \text{grid}_X + \text{currWidth}$

$\text{newHeight} = i \times 2 \times \text{grid}_Y + \text{currHeight}$ .

[0283] where the  $\text{currWidth}$  and  $\text{currHeight}$  are the width and height of current block. The new  $\text{Width}$  and  $\text{newHeight}$  are the width and height of new virtual block.

[0284]  $\text{grid}_X$  and  $\text{grid}_Y$  are currently set to  $\text{currWidth}$  and  $\text{currHeight}$ , respectively.

[0285] FIG. 28 illustrates the relationship between the virtual block and the current block.

[0286] After generating the virtual block, the blocks  $A_i$ ,  $B_i$ ,  $C_i$ ,  $D_i$  and  $E_i$  can be regarded as the VVC spatial neighboring blocks of the virtual block and their positions are obtained with the same pattern as that in VVC. Obviously, the virtual block is the current block if the search round  $i$  is 0. In this case, the blocks  $A_i$ ,  $B_i$ ,  $C_i$ ,  $D_i$  and  $E_i$  are the spatially neighboring blocks that are used in VVC merge mode.

[0287] When constructing the merge candidate list, the pruning is performed to guarantee each element in merge candidate list to be unique. The maximum search round is set to 1, which means that five non-adjacent spatial neighbor blocks are utilized.

[0288] Non-adjacent spatial merge candidates are inserted into the merge list after the temporal merge candidate in the order of  $B_1 \rightarrow A_1 \rightarrow C_1 \rightarrow D_1 \rightarrow E_1$ .

### 2.19.2. STMVP

[0289] It is proposed to derive an averaging candidate as STMVP candidate using three spatial merge candidates and one temporal merge candidate.

[0290] STMVP is inserted before the above-left spatial merge candidate.

[0291] The STMVP candidate is pruned with all the previous merge candidates in the merge list.

[0292] For the spatial candidates, the first three candidates in the current merge candidate list are used.

[0293] For the temporal candidate, the same position as VTM/HEVC collocated position is used.

[0294] For the spatial candidates, the first, second, and third candidates inserted in the current merge candidate list before STMVP are denoted as  $F$ ,  $S$ , and  $T$ .

[0295] The temporal candidate with the same position as VTM/HEVC collocated position used in TMVP is denoted as  $\text{Col}$ .

[0296] The motion vector of the STMVP candidate in prediction direction  $X$  (denoted as  $\text{mvLX}$ ) is derived as follows:

[0297] 1) If the reference indices of the four merge candidates are all valid and are all equal to zero in prediction direction  $X$  ( $X=0$  or  $1$ ),

$$\text{mvLX} = (\text{mvLX}_F + \text{mvLX}_S + \text{mvLX}_T + \text{mvLX}_{\text{Col}}) \gg 2$$

[0298] 2) If reference indices of three of the four merge candidates are valid and are equal to zero in prediction direction  $X$  ( $X=0$  or  $1$ ),

$$\text{mvLX} = (\text{mvLX}_F \times 3 + \text{mvLX}_S \times 3 + \text{mvLX}_{\text{Col}} \times 2) \gg 3 \text{ or}$$

$$\text{mvLX} = (\text{mvLX}_F \times 3 + \text{mvLX}_T \times 3 + \text{mvLX}_{\text{Col}} \times 2) \gg 3 \text{ or}$$

$$\text{mvLX} = (\text{mvLX}_S \times 3 + \text{mvLX}_T \times 3 + \text{mvLX}_{\text{Col}} \times 2) \gg 3$$

[0299] 3) If reference indices of two of the four merge candidates are valid and are equal to zero in prediction direction  $X$  ( $X=0$  or  $1$ ),

$$\text{mvLX} = (\text{mvLX}_F + \text{mvLX}_{\text{Col}}) \gg 1 \text{ or}$$

$$\text{mvLX} = (\text{mvLX}_S + \text{mvLX}_{\text{Col}}) \gg 1 \text{ or}$$

$$\text{mvLX} = (\text{mvLX}_T + \text{mvLX}_{\text{Col}}) \gg 1$$

Note: If the temporal candidate is unavailable, the STMVP mode is off.

### 2.19.3. Merge List Size

[0300] If considering both non-adjacent and STMVP merge candidates, the size of merge list is signalled in sequence parameter set header and the maximum allowed size of merge list is 8.

## 2.20. Geometric Partitioning Mode (GPM)

[0301] In VVC, a geometric partitioning mode is supported for inter prediction. The geometric partitioning mode is signalled using a CU-level flag as one kind of merge mode, with other merge modes including the regular merge mode, the MMVD mode, the CIIP mode and the subblock merge mode. In total 64 partitions are supported by geometric partitioning mode for each possible CU size  $w \times h = 2^m \times 2^n$  with  $m, n \in \{3 \dots 6\}$  excluding  $8 \times 64$  and  $64 \times 8$ .

[0302] When this mode is used, a CU is split into two parts by a geometrically located straight line (FIG. 29). The location of the splitting line is mathematically derived from the angle and offset parameters of a specific partition. Each part of a geometric partition in the CU is inter-predicted using its own motion; only uni-prediction is allowed for each partition, that is, each part has one motion vector and one reference index. The uni-prediction motion constraint is applied to ensure that same as the conventional bi-prediction, only two motion compensated prediction are needed for each CU. The uni-prediction motion for each partition is derived using the process described in 2.20.1. If geometric partitioning mode is used for the current CU, then a geometric partition index indicating the partition mode of the geometric partition (angle and offset), and two merge indices

(one for each partition) are further signalled. The number of maximum GPM candidate size is signalled explicitly in SPS and specifies syntax binarization for GPM merge indices. After predicting each of part of the geometric partition, the sample values along the geometric partition edge are adjusted using a blending processing with adaptive weights as in 2.20.2. This is the prediction signal for the whole CU, and transform and quantization process will be applied to the whole CU as in other prediction modes. Finally, the motion field of a CU predicted using the geometric partition modes is stored as in 2.20.3.

#### 2.20.1. Uni-Prediction Candidate List Construction

**[0303]** The uni-prediction candidate list is derived directly from the merge candidate list constructed according to the extended merge prediction process in 2.18. Denote  $n$  as the index of the uni-prediction motion in the geometric uni-prediction candidate list. The LX motion vector of the  $n$ -th extended merge candidate, with  $X$  equal to the parity of  $n$ , is used as the  $n$ -th uni-prediction motion vector for geometric partitioning mode. These motion vectors are marked with “x” in FIG. 30. In case a corresponding LX motion vector of the  $n$ —the extended merge candidate does not exist, the  $L(1-X)$  motion vector of the same candidate is used instead as the uni-prediction motion vector for geometric partitioning mode.

#### 2.20.2. Blending Along the Geometric Partitioning Edge

**[0304]** After predicting each part of a geometric partition using its own motion, blending is applied to the two prediction signals to derive samples around geometric partition edge. The blending weight for each position of the CU are derived based on the distance between individual position and the partition edge.

**[0305]** The distance for a position( $x,y$ ) to the partition edge are derived as:

$$d(x, y) = (2x + 1 - w)\cos(\varphi_i) + (2y + 1 - h)\sin(\varphi_i) - \rho_j \quad (2-24)$$

$$\rho_j = \rho_{x,j}\cos(\varphi_i) + \rho_{y,j}\sin(\varphi_i) \quad (2-25)$$

$$\rho_{x,j} = \begin{cases} 0 & i \% 16 = 8 \text{ or } (i \% 16 \neq 0 \text{ and } h \geq w) \\ \pm(j \times w) \gg 2 & \text{otherwise} \end{cases} \quad (2-26)$$

$$\rho_{y,j} = \begin{cases} \pm(j \times h) \gg 2 & i \% 16 = 8 \text{ or } (i \% 16 \neq 0 \text{ and } h \geq w) \\ 0 & \text{otherwise} \end{cases} \quad (2-27)$$

where  $i,j$  are the indices for angle and offset of a geometric partition, which depend on the signaled geometric partition index. The sign of  $\rho_{x,j}$  and  $\rho_{y,j}$  depend on angle index  $i$ .

**[0306]** The weights for each part of a geometric partition are derived as following:

$$wIdxL(x, y) = partIdx \cdot 32 + d(x, y) : 32 - d(x, y) \quad (2-28)$$

$$w_0(x, y) = \frac{Clip3(0, 8, (wIdxL(x, y) + 4) \gg 3)}{8} \quad (2-29)$$

$$w_1(x, y) = 1 - w_0(x, y) \quad (2-30)$$

**[0307]** The  $partIdx$  depends on the angle index  $i$ . One example of weigh  $w_0$  is illustrated in FIG. 31.

#### 2.20.3. Motion Field Storage for Geometric Partitioning Mode

**[0308]** Mv1 from the first part of the geometric partition, Mv2 from the second part of the geometric partition and a combined Mv of Mv1 and Mv2 are stored in the motion filed of a geometric partitioning mode coded CU. The stored motion vector type for each individual position in the motion filed are determined as:

$$sType = \quad (2-31)$$

$$abs(motionIdx) < 32 \cdot 2 : (motionIdx \leq 0 ? (1 - partIdx) : partIdx)$$

where  $motionIdx$  is equal to  $d(4x+2, 4y+2)$ , which is recalculated from equation (2-18). The  $partIdx$  depends on the angle index  $i$ .

**[0309]** If  $sType$  is equal to 0 or 1, Mv0 or Mv1 are stored in the corresponding motion field, otherwise if  $sType$  is equal to 2, a combined Mv from Mv0 and Mv2 are stored. The combined Mv are generated using the following process:

**[0310]** 1) If Mv1 and Mv2 are from different reference picture lists (one from L0 and the other from L1), then Mv1 and Mv2 are simply combined to form the bi-prediction motion vectors.

**[0311]** Otherwise, if Mv1 and Mv2 are from the same list, only uni-prediction motion Mv2 is stored.

#### 2.21. Multi-Hypothesis Prediction

**[0312]** In multi-hypothesis prediction (MHP), up to two additional predictors are signalled on top of inter AMVP mode, regular merge mode, affine merge and MMVD mode. The resulting overall prediction signal is accumulated iteratively with each additional prediction signal.

$$p_{n+1} = (1 - \alpha_{n+1})p_n + \alpha_{n+1}h_{n+1}$$

**[0313]** The weighting factor  $\alpha$  is specified according to the following Table 2-4:

TABLE 2-4

| weighting factor for MHP |          |
|--------------------------|----------|
| add_hyp_weight_idx       | $\alpha$ |
| 0                        | $1/4$    |
| 1                        | $1/8$    |

**[0314]** For inter AMVP mode, MHP is only applied if non-equal weight in BCW is selected in bi-prediction mode. The additional hypothesis can be either merge or AMVP mode. In the case of merge mode, the motion information is indicated by a merge index, and the merge candidate list is the same as in the Geometric Partition Mode. In the case of AMVP mode, the reference index, MVP index, and MVD are signaled.

#### 2.22. Non-Adjacent Spatial Candidate

**[0315]** The non-adjacent spatial merge candidates are inserted after the TMVP in the regular merge candidate list. The pattern of the spatial merge candidates is shown on FIG.

**32.** The distances between the non-adjacent spatial candidates and the current coding block are based on the width and height of the current coding block.

### 2.23. Template Matching (TM)

**[0316]** Template matching (TM) is a decoder-side MV derivation method to refine the motion information of the current CU by finding the closest match between a template (i.e., top and/or left neighbouring blocks of the current CU) in the current picture and a block (i.e., same size to the template) in a reference picture. As illustrated in FIG. 33, a better MV is to be searched around the initial motion of the current CU within a  $[-8, +8]$ -pel search range. The template matching that was proposed two modifications: search step size is determined based on AMVR mode and TM can be cascaded with bilateral matching process in merge modes. **[0317]** In AMVP mode, an MVP candidate is determined based on template matching error to pick up the one which reaches the minimum difference between current block template and reference block template, and then TM performs only for this particular MVP candidate for MV refinement. TM refines this MVP candidate, starting from full-pel MVD precision (or 4-pel for 4-pel AMVR mode) within a  $[-8, +8]$ -pel search range by using iterative diamond search. The MVP candidate may be further refined by using cross search with full-pel MVD precision (or 4-pel for 4-pel AMVR mode), followed sequentially by half-pel and quarter-pel ones depending on AMVR mode as specified in Table 2-5. This search process ensures that the MVP candidate still keeps the same MV precision as indicated by AMVR mode after TM process.

TABLE 2-5

| Search patterns of AMVR and merge mode with AMVR. |            |          |             |   |            |           |
|---------------------------------------------------|------------|----------|-------------|---|------------|-----------|
| Search pattern                                    | AMVR mode  |          |             |   | Merge mode |           |
|                                                   | Full-4-pel | Half-pel | Quarter-pel |   | AltIF = 0  | AltIF = 1 |
| 4-pel diamond                                     | v          |          |             |   |            |           |
| 4-pel cross                                       | v          |          |             |   |            |           |
| Full-pel diamond                                  |            | v        | v           | v | v          | v         |
| Full-pel cross                                    |            | v        | v           | v | v          | v         |
| Half-pel cross                                    |            |          | v           | v | v          | v         |
| Quarter-pel cross                                 |            |          |             | v | v          |           |
| 1/8-pel cross                                     |            |          |             |   | v          |           |

**[0318]** In merge mode, similar search method is applied to the merge candidate indicated by the merge index. As Table 2-5 shows, TM may perform all the way down to 1/8-pel MVD precision or skipping those beyond half-pel MVD precision, depending on whether the alternative interpolation filter (that is used when AMVR is of half-pel mode) is used according to merged motion information. Besides, when TM mode is enabled, template matching may work as an independent process or an extra MV refinement process between block-based and subblock-based bilateral matching (BM) methods, depending on whether BM can be enabled or not according to its enabling condition check.

### 2.24. Overlapped Block Motion Compensation (OBMC)

**[0319]** Overlapped Block Motion Compensation (OBMC) has previously been used in H.263. In the JEM, unlike in H.263, OBMC can be switched on and off using syntax at

the CU level. When OBMC is used in the JEM, the OBMC is performed for all motion compensation (MC) block boundaries except the right and bottom boundaries of a CU. Moreover, it is applied for both the luma and chroma components. In the JEM, a MC block is corresponding to a coding block. When a CU is coded with sub-CU mode (includes sub-CU merge, affine and FRUC mode), each sub-block of the CU is a MC block. To process CU boundaries in a uniform fashion, OBMC is performed at sub-block level for all MC block boundaries, where sub-block size is set equal to  $4 \times 4$ , as illustrated in FIG. 34.

**[0320]** When OBMC applies to the current sub-block, besides current motion vectors, motion vectors of four connected neighbouring sub-blocks, if available and are not identical to the current motion vector, are also used to derive prediction block for the current sub-block. These multiple prediction blocks based on multiple motion vectors are combined to generate the final prediction signal of the current sub-block.

**[0321]** Prediction block based on motion vectors of a neighbouring sub-block is denoted as  $P_N$ , with N indicating an index for the neighbouring above, below, left and right sub-blocks and prediction block based on motion vectors of the current sub-block is denoted as  $P_C$ . When  $P_N$  is based on the motion information of a neighbouring sub-block that contains the same motion information to the current sub-block, the OBMC is not performed from PN. Otherwise, every sample of  $P_N$  is added to the same sample in  $P_C$ , i.e., four rows/columns of  $P_N$  are added to  $P_C$ . The weighting factors  $\{1/4, 1/8, 1/16, 1/32\}$  are used for  $P_N$  and the weighting factors  $\{3/4, 7/8, 15/16, 31/32\}$  are used for  $P_C$ . The exception are small MC blocks, (i.e., when height or width of the coding block is equal to 4 or a CU is coded with sub-CU mode), for which only two rows/columns of  $P_N$  are added to  $P_C$ . In this case weighting factors  $\{1/4, 1/8\}$  are used for  $P_N$  and weighting factors  $\{3/4, 7/8\}$  are used for  $P_C$ . For  $P_N$  generated based on motion vectors of vertically (horizontally) neighbouring sub-block, samples in the same row (column) of  $P_N$  are added to  $P_C$  with a same weighting factor.

**[0322]** In the JEM, for a CU with size less than or equal to 256 luma samples, a CU level flag is signalled to indicate whether OBMC is applied or not for the current CU. For the CUs with size larger than 256 luma samples or not coded with AMVP mode, OBMC is applied by default. At the encoder, when OBMC is applied for a CU, its impact is taken into account during the motion estimation stage. The prediction signal formed by OBMC using motion information of the top neighbouring block and the left neighbouring block is used to compensate the top and left boundaries of the original signal of the current CU, and then the normal motion estimation process is applied.

### 2.25. Multiple Transform Selection (MTS) for Core Transform

**[0323]** In addition to DCT-II which has been employed in HEVC, a Multiple Transform Selection (MTS) scheme is used for residual coding both inter and intra coded blocks. It uses multiple selected transforms from the DCT8/DST7. The newly introduced transform matrices are DST-VII and DCT-VIII. Table 2-6 shows the basis functions of the selected DST/DCT.

TABLE 2-6

| Transform basis functions of DCT-II/VIII and DST-VII for N-point input |                                                                                                                                                                                                          |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transform Type                                                         | Basis function $T_i(j)$ , $i, j = 0, 1, \dots, N-1$                                                                                                                                                      |
| DCT-II                                                                 | $T_i(j) = \omega_0 \cdot \sqrt{\frac{2}{N}} \cdot \cos\left(\frac{\pi \cdot i \cdot (2j+1)}{2N}\right)$ $\text{where, } \omega_0 = \begin{cases} \sqrt{\frac{2}{N}} & i = 0 \\ 1 & i \neq 0 \end{cases}$ |
| DCT-VIII                                                               | $T_i(j) = \sqrt{\frac{4}{2N+1}} \cdot \cos\left(\frac{\pi \cdot (2i+1) \cdot (2j+1)}{4N+2}\right)$                                                                                                       |
| DST-VII                                                                | $T_i(j) = \sqrt{\frac{4}{2N+1}} \cdot \sin\left(\frac{\pi \cdot (2i+1) \cdot (j+1)}{2N+1}\right)$                                                                                                        |

**[0324]** In order to keep the orthogonality of the transform matrix, the transform matrices are quantized more accurately than the transform matrices in HEVC. To keep the intermediate values of the transformed coefficients within the 16-bit range, after horizontal and after vertical transform, all the coefficients are to have 10-bit.

**[0325]** In order to control MTS scheme, separate enabling flags are specified at SPS level for intra and inter, respectively. When MTS is enabled at SPS, a CU level flag is signalled to indicate whether MTS is applied or not. Here, MTS is applied only for luma. The MTS signaling is skipped when one of the below conditions is applied.

**[0326]** The position of the last significant coefficient for the luma TB is less than 1 (i.e., DC only)

**[0327]** The last significant coefficient of the luma TB is located inside the MTS zero-out region

**[0328]** If MTS CU flag is equal to zero, then DCT2 is applied in both directions. However, if MTS CU flag is equal to one, then two other flags are additionally signalled to indicate the transform type for the horizontal and vertical directions, respectively. Transform and signalling mapping table as shown in Table 2-7. Unified the transform selection for ISP and implicit MTS is used by removing the intra-mode and block-shape dependencies. If current block is ISP mode or if the current block is intra block and both intra and inter explicit MTS is on, then only DST7 is used for both horizontal and vertical transform cores. When it comes to transform matrix precision, 8-bit primary transform cores are used. Therefore, all the transform cores used in HEVC are kept as the same, including 4-point DCT-2 and DST-7, 8-point, 16-point and 32-point DCT-2. Also, other transform cores including 64-point DCT-2, 4-point DCT-8, 8-point, 16-point, 32-point DST-7 and DCT-8, use 8-bit primary transform cores.

TABLE 2-7

| Transform and signalling mapping table |              |              |             |          |
|----------------------------------------|--------------|--------------|-------------|----------|
| MTS_CU_flag                            | MTS_Hor_flag | MTS_Ver_flag | Intra/inter |          |
|                                        |              |              | Horizontal  | Vertical |
| 0                                      |              |              | DCT2        |          |
| 1                                      | 0            | 0            | DST7        | DST7     |
|                                        | 0            | 1            | DCT8        | DST7     |
|                                        | 1            | 0            | DST7        | DCT8     |
|                                        | 1            | 1            | DCT8        | DCT8     |

**[0329]** To reduce the complexity of large size DST-7 and DCT-8, High frequency transform coefficients are zeroed out for the DST-7 and DCT-8 blocks with size (width or height, or both width and height) equal to 32. Only the coefficients within the 16×16 lower-frequency region are retained.

**[0330]** As in HEVC, the residual of a block can be coded with transform skip mode. To avoid the redundancy of syntax coding, the transform skip flag is not signalled when the CU level MTS\_CU\_flag is not equal to zero. Note that implicit MTS transform is set to DCT2 when LFNST or MIP is activated for the current CU. Also the implicit MTS can be still enabled when MTS is enabled for inter coded blocks.

## 2.26. Subblock Transform (SBT)

**[0331]** In VTM, subblock transform is introduced for an inter-predicted CU. In this transform mode, only a sub-part of the residual block is coded for the CU. When inter-predicted CU with cu\_cbf equal to 1, cu\_sbt\_flag may be signaled to indicate whether the whole residual block or a sub-part of the residual block is coded. In the former case, inter MTS information is further parsed to determine the transform type of the CU. In the latter case, a part of the residual block is coded with inferred adaptive transform and the other part of the residual block is zeroed out.

**[0332]** When SBT is used for an inter-coded CU, SBT type and SBT position information are signaled in the bitstream. There are two SBT types and two SBT positions, as indicated in FIG. 35. For SBT-V (or SBT-H), the TU width (or height) may equal to half of the CU width (or height) or 1/4 of the CU width (or height), resulting in 2:2 split or 1:3/3:1 split. The 2:2 split is like a binary tree (BT) split while the 1:3/3:1 split is like an asymmetric binary tree (ABT) split. In ABT splitting, only the small region contains the non-zero residual. If one dimension of a CU is 8 in luma samples, the 1:3/3:1 split along that dimension is disallowed. There are at most 8 SBT modes for a CU.

**[0333]** Position-dependent transform core selection is applied on luma transform blocks in SBT-V and SBT-H (chroma TB always using DCT-2). The two positions of SBT-H and SBT-V are associated with different core transforms. More specifically, the horizontal and vertical transforms for each SBT position is specified in FIG. 35. For example, the horizontal and vertical transforms for SBT-V position 0 is DCT-8 and DST-7, respectively. When one side of the residual TU is greater than 32, the transform for both dimensions is set as DCT-2. Therefore, the subblock transform jointly specifies the TU tiling, cbf, and horizontal and vertical core transform type of a residual block. The SBT is not applied to the CU coded with combined inter-intra mode.



### 2.27. Template Matching Based Adaptive Merge Candidate Reorder

[0334] To improve the coding efficiency, after the merge candidate list is constructed, the order of each merge candidate is adjusted according to the template matching cost. The merge candidates are arranged in the list in accordance with the template matching cost of ascending order. It is operated in the form of sub-group.

[0335] The template matching cost is measured by the SAD (Sum of absolute differences) between the neighbouring samples of the current CU and their corresponding reference samples. If a merge candidate includes bi-predictive motion information, the corresponding reference samples are the average of the corresponding reference samples in reference list0 and the corresponding reference samples in reference list1, as illustrated in FIG. 36. If a merge candidate includes sub-CU level motion information, the corresponding reference samples consist of the neighbouring samples of the corresponding reference sub-blocks, as illustrated in FIG. 37.

[0336] The sorting process is operated in the form of sub-group, as illustrated in FIG. 38. The first three merge candidates are sorted together. The following three merge candidates are sorted together.

[0337] The template size (width of the left template or height of the above template) is 1. The sub-group size is 3.

### 2.28. Adaptive Merge Candidate List

[0338] We can assume the number of the merge candidates is 8. We take the first 5 merge candidates as a first subgroup and take the following 3 merge candidates as a second subgroup (i.e., the last subgroup).

[0339] For the encoder, after the merge candidate list is constructed, some merge candidates are adaptively reordered in an ascending order of costs of merge candidates as shown in FIG. 39.

[0340] More specifically, the template matching costs for the merge candidates in all subgroups except the last subgroup are computed; then reorder the merge candidates in their own subgroups except the last subgroup; finally, the final merge candidate list will be got.

[0341] For the decoder, after the merge candidate list is constructed, some/no merge candidates are adaptively reordered in ascending order of costs of merge candidates as shown in FIG. 40. In FIG. 40, the subgroup the selected (signaled) merge candidate located in is called the selected subgroup.

[0342] More specifically, if the selected merge candidate is located in the last subgroup, the merge candidate list construction process is terminated after the selected merge candidate is derived, no reorder is performed and the merge candidate list is not changed; otherwise, the execution process is as follows: The merge candidate list construction process is terminated after all the merge candidates in the selected subgroup are derived; compute the template matching costs for the merge candidates in the selected subgroup; reorder the merge candidates in the selected subgroup; finally, a new merge candidate list will be got.

[0343] For both encoder and decoder,

[0344] A template matching cost is derived as a function of T and RT, wherein T is a set of samples in the template and RT is a set of reference samples for the template.

[0345] When deriving the reference samples of the template for a merge candidate, the motion vectors of the merge candidate are rounded to the integer pixel accuracy.

[0346] The reference samples of the template (RT) for bi-directional prediction are derived by weighted averaging of the reference samples of the template in reference list0 (RT<sub>0</sub>) and the reference samples of the template in reference list1 (RT<sub>1</sub>) as follows.

$$RT = ((8 - w) * RT_0 + w * RT_1 + 4) \gg 3 \quad (2-32)$$

where the weight of the reference template in reference list0 (8-w) and the weight of the reference template in reference list1 (w) are decided by the BCW index of the merge candidate. BCW index equal to {0,1,2,3,4} corresponds to w equal to {-2,3,4,5,10}, respectively.

[0347] If the Local Illumination Compensation (LIC) flag of the merge candidate is true, the reference samples of the template are derived with LIC method.

[0348] The template matching cost is calculated based on the sum of absolute differences (SAD) of T and RT. The template size is 1. That means the width of the left template and/or the height of the above template is 1.

[0349] If the coding mode is MMVD, the merge candidates to derive the base merge candidates are not reordered. If the coding mode is GPM, the merge candidates to derive the uni-prediction candidate list are not reordered.

### 2.29. IBC with Extended Reference Area

[0350] An IBC reference area design is proposed that does not increase the current memory area required by ECM-3 and tests the performance.

[0351] FIG. 41 illustrates the design. In the figure, the blue square denotes the current CTU and the green ones denote CTUs that may be used by IBC reference. Specifically, assume that W denotes the maximum horizontal CTU index and the current CTU index is (m, n), for coding units in the current CTU, CTUs with index (0, n) . . . (m, n) and (m-1, n) . . . (W, n) defines the reference area that can be used by IBC.

[0352] One reason to have such a design is that in the current ECM, the left, above and upper-left CTUs are being used and thus need to be saved. To achieve this, all CTUs to the right of the above CTU in the above CTU row (for CTUs to be coded in the current CTU row) and all CTUs to the left of the current CTU in the current CTU row (for CTUs to be coded in the next CTU row) must be kept. It means that such a design does not increase the buffer size required by the current ECM.

### 2.30. IBC with Template Matching

[0353] It is proposed to also use Template Matching with IBC for both IBC merge mode and IBC AMVP mode. The IBC-TM merge list has been modified compared to the one used by regular IBC merge mode such that the candidates are selected according to a pruning method with a motion distance between the candidates as in the regular TM merge mode. The ending zero motion fulfillment (which is a nonsense regarding Intra coding) has been replaced by motion vectors to the left (-W, 0), top (0, -H) and top-left (-W, -H) CUs, then, if necessary, the list is fulfilled with the left one without pruning.

[0354] In the IBC-TM merge mode, the selected candidates are refined with the Template Matching method prior to the RDO or decoding process. The IBC-TM merge mode has been put in competition with the regular IBC merge mode and a TM-merge flag is signaled.

[0355] In the IBC-TM AMVP mode, up to 3 candidates are selected from the IBC merge list. Each of those 3 selected candidates are refined using the Template Matching

method and sorted according to their resulting Template Matching cost. Only the 2 first ones are then considered in the motion estimation process as usual. The Template Matching refinement for both IBC-TM merge and AMVP modes is quite simple since IBC motion vectors are constrained to be integer and within a reference region as shown in FIG. 42. So, in IBC-TM merge mode, all refinements are performed at integer precision, and in IBC-TM AMVP mode, they are performed either at integer or 4-pel precision. In both cases, the refined motion vectors in each refinement step must respect the constraint of the reference region.

### 2.31. Reconstruction-Reordered IBC (RR-IBC)

[0356] Screen content coding tools like Intra Block Copy (IBC) generate a prediction block by directly copying a prior coded reference region in the same picture. Symmetry is often observed in video content, especially in text character regions and computer-generated graphics in screen content sequences, as shown in FIG. 43. Therefore, a specific screen content coding tool considering the symmetry would be efficient to compress such kinds of video contents. A Reconstruction-Reordered IBC (RR-IBC) mode is proposed for screen content video coding. When it is applied, the samples in a reconstruction block are flipped according to a flip type of the current block. At the encoder side, the original block is flipped before motion search and residual calculation, while the prediction block is derived without flipping. At the decoder side, the reconstruction block is flipped back to restore the original block.

[0357] Two flip methods, horizontal flip and vertical flip, are supported for RR-IBC coded blocks. A syntax flag is firstly signalled for an IBC AMVP coded block, indicating whether the reconstruction is flipped, and if it is flipped, another flag is further signaled specifying the flip type. For IBC merge, the flip type is inherited from neighbouring blocks, without syntax signalling. Considering the horizontal or vertical symmetry, the current block and the reference block are normally aligned horizontally or vertically. Therefore, when a horizontal flip is applied, the vertical component of the BV is not signaled and inferred to be equal to 0. Similarly, the horizontal component of the BV is not signaled and inferred to be equal to 0 when a vertical flip is applied.

[0358] To better utilize the symmetry property, a flip-aware BV adjustment approach is applied to refine the block vector candidate. For example, as shown in FIG. 44,  $(x_{nbr}, y_{nbr})$  and  $(x_{cur}, y_{cur})$  represent the coordinates of the center sample of the neighboring block and the current block, respectively,  $BV^{nbr}$  and  $BV^{cur}$  denotes the BV of the neighboring block and the current block, respectively. Instead of directly inheriting the BV from a neighbouring block, the horizontal component of  $BV^{cur}$  is calculated by adding a motion shift to the horizontal component of  $BV^{nbr}$  (denoted as  $BV_h^{nbr}$ ) in case that the neighbouring block is coded with a horizontal flip, i.e.,  $BV_h^{cur} = 2(x_{nbr} - x_{cur}) + BV_h^{nbr}$ . Similarly, the vertical component of  $BV^{cur}$  is calculated by adding a motion shift to the vertical component of  $BV^{nbr}$  (denoted as  $BV_v^{nbr}$ ) in case that the neighbouring block is coded with a vertical flip, i.e.,  $BV_v^{cur} = 2(y_{nbr} - y_{cur}) + BV_v^{nbr}$ .

### 2.32. Intra Template Matching

[0359] Intra template matching prediction (Intra TMP) is a special intra prediction mode that copies the best prediction block from the reconstructed part of the current frame, whose L-shaped template matches the current template. For a predefined search range, the encoder searches for the most

similar template to the current template in a reconstructed part of the current frame and uses the corresponding block as a prediction block. The encoder then signals the usage of this mode, and the same prediction operation is performed at the decoder side.

[0360] The prediction signal is generated by matching the L-shaped causal neighbor of the current block with another block in a predefined search area in FIG. 45 consisting of:

[0361] R1: current CTU

[0362] R2: top-left CTU

[0363] R3: above CTU

[0364] R4: left CTU

[0365] SAD is used as a cost function.

[0366] Within each region, the decoder searches for the template that has least SAD with respect to the current one and uses its corresponding block as a prediction block.

[0367] The dimensions of all regions (SearchRange\_w, SearchRange\_h) are set proportional to the block dimension (BlkW, BlkH) to have a fixed number of SAD comparisons per pixel. That is:

$$\text{SearchRange\_w} = a * \text{BlkW}$$

$$\text{SearchRange\_h} = a * \text{BlkH}$$

[0368] Where 'a' is a constant that controls the gain/complexity trade-off. In practice, 'a' is equal to 5. The Intra template matching tool is enabled for CUs with size less than or equal to 64 in width and height. This maximum CU size for Intra template matching is configurable.

[0369] The Intra template matching prediction mode is signaled at CU level through a dedicated flag when DIMD is not used for current CU.

## 3. Problems

[0370] In current design of IBC, the prediction of current block is obtained with the samples in current picture which is indicated by a block vector. The coding performance of IBC is quite good for screen content videos in which repeated contents exist. However, for natural content videos, the coding gain of IBC is much lower than that for screen content videos due to the different characteristics.

## 4. Detailed Solutions

[0371] The embodiments below should be considered as examples to explain general concepts. These embodiments should not be interpreted in a narrow way. Furthermore, these embodiments can be combined in any manner. In the present disclosure, intra block copy (IBC) may not be limited to the current IBC technology but may be interpreted as the technology that reference (or prediction) block is obtained with samples in the current slice/tile/subpicture/picture/other video unit (e.g., CTU row) excluding the conventional intra prediction methods. A reference line may refer to a row and/or a column reconstructed samples adjacent to or non-adjacent to the current block, which is used to derive the intra prediction of current video unit via an interpolation filter along a certain direction, and the certain direction is determined by an intra prediction mode (e.g., conventional intra prediction with intra prediction modes), or derive the intra prediction of current video unit via weighting the reference samples of the reference line with a matrix or vector (e.g., MIP).

## Combination of Intra Block Copy and Intra Prediction

[0372] 1. It is proposed to use the combination of intra block copy (IBC) and intra prediction (CIBCIP) to derive the prediction/reconstruction of a video unit, which is obtained by fusing the IBC predicted signal and the intra predicted signal.

[0373] a. In one example,  $P(x,y) = W_{ip1} * IP_1(x,y) + W_{ip2} * IP_2(x,y) + \dots + W_{ipn} * IP_n(x,y) + W_{ibc1} * IBC_1(x,y) + W_{ibc2} * IBC_2(x,y) + \dots + W_{ibcm} * IBC_m(x,y)$ , where  $P(x,y)$  is the generated prediction value,  $IP_k$  is the prediction generated by the  $k$ th intra prediction,  $IBC_j$  is the prediction generated by the  $j$ th IBC and  $w_{ipk}$  and  $w_{ibcj}$  are corresponding weighting values.

[0374] b. In one example, the intra prediction may be generated by angular intra-prediction, DC, planar, cross-component prediction (CCLM), multi-model CCLM, left CCLM, above CCLM, etc.

[0375] i. The intra prediction mode may be coded using MPM or TIMD or DIMD or any other methods to signal the intra prediction mode.

[0376] ii. A specific set of intra prediction modes may be allowed to be used in CIBCIP.

[0377] c. In one example, IBC merge mode may be used.

[0378] i. In one example, one or more BV candidates in the IBC merge list may be allowed to be used in CIBCIP.

[0379] ii. In one example, one or more BV offsets may be used for CIBCIP.

[0380] 1) In one example, the BV offsets may be added to the BV candidate before it is used to obtain the IBC prediction.

[0381] 2) In one example, the BV offsets may be signalled or derived.

[0382] iii. In one example, the IBC merge mode may be at least one of regular IBC merge mode and IBC-MBVD merge mode and IBC-TM merge mode.

[0383] d. In one example, IBC AMVP mode may be used.

[0384] i. In one example, one or more BV predictors in the IBC AMVP list may be allowed to be used in CIBCIP.

[0385] ii. In one example, the block vector difference (BVD) used in CIBCIP may be signalled using the same way as IBC mode.

[0386] 1) Alternatively, the BVD may be not signalled but pre-defined.

[0387] iii. In one example, the BVD may be derived using the coding information.

[0388] e. In one example, a merge index (mergIdx) indicating the BV candidate in the IBC merge list and/or a BVP index (bvpIdx) indicating the BV predictor in the IBC AMVP list used to obtain the IBC predicted signal may be signalled.

[0389] 1) In one example, the binarization or signalling method of the merge index or the BVP index may be same as that in IBC mode.

[0390] 2) Alternatively, the merge index or the BVP index may be pre-defined, such as  $\text{mergIdx} = 0$  or  $\text{mergIdx} = 1$ ;  $\text{bvpIdx} = 0$  or  $\text{bvpIdx} = 1$ .

[0391] 3) Alternatively, the merge index or the BVP index may be derived using the coding information.

[0392] 4) Alternatively, the merge index or the BVP index may be derived using the template matching (e.g., with the smallest template matching cost).

[0393] f. In one example, the construction of IBC merge list or IBC AMVP list used in CIBCIP mode may be same as or different from that used in IBC mode.

[0394] g. In one example, the number of BV candidates (N) in IBC merge (or AMVP) list that can be used for CIBCIP is less than or equal to the number of BV candidates (M) in the IBC merge (or AMVP) list that can be used for IBC. N is an integer that larger than 0 and less than or equal to M.

[0395] i. In one example,  $N=1$ , or  $N=2$ , or  $N=3$ , or  $N=4$ , or  $N=5$ , or  $N=6$ .

[0396] ii. In one example, the first N BV candidates of the IBC merge (or AMVP) list may be used for CIBCIP.

[0397] h. In one example, template matching may be used to derive/refine the BV, which is used to obtain the IBC predicted signal.

[0398] i. In one example, BV offset may be derived using template matching, which is added to the BV candidate in the IBC merge list.

[0399] ii. In one example, the BVD may be derived using template matching based method.

[0400] iii. In one example, the BVD sign may be derived using template matching based method.

[0401] iv. In one example, the intra prediction mode or the intra prediction method used to obtain the intra predicted signal may be used in the template matching to derive/refine the BV.

[0402] i. In one example, the BV list may be reordered before being used for CIBCIP.

[0403] i. In one example, template matching or bilateral matching cost may be used for the reordering.

[0404] ii. In one example, template matching or bilateral matching may be used during the construction of the BV list used for CIBCIP.

[0405] iii. In one example, the BV list may refer to the IBC merge list or the IBC AMVP list.

[0406] iv. In one example, the reordering method for the BV list used for CIBCIP may be same as that for IBC.

[0407] v. Alternatively, the reordering method for the BV list used for CIBCIP may be different from that for IBC.

[0408] 1) In one example, the number of BV candidates ( $N_1$ ) in the BV list used for the reordering for the CIBCIP may be less than or equal to the number of BV candidates ( $M_1$ ) used for the reordering for IBC mode.

[0409] a) In one example,  $N_1=1$ , or 2, or 3, or 4 when IBC merge mode is used for CIBCIP.

[0410] b) In one example,  $N_1=1$ , or 2, or 3 when IBC AMVP is used for CIBCIP.

[0411] j. In one example, intra prediction may refer to conventional intra prediction method (e.g., intra prediction using 35 intra prediction modes in HEVC or 67 intra prediction modes in VVC), or other intra prediction method which obtains the prediction block with samples in the current slice/tile/subpicture/picture/other video unit (e.g., CU, PU, TU, CTU, CTU row) excluding IBC.

[0412] i. In one example, the intra predicted signal may be obtained using one or more pre-defined intra prediction modes.

[0413] 1) In one example, the pre-defined intra prediction modes may refer to Planar mode, DC mode, Horizontal mode, Vertical mode.

- [0414] ii. In one example, the intra predicted signal may be obtained using one or more of most probable modes (MPMs)
- [0415] iii. In one example, the intra predicted signal may be obtained using an intra prediction mode which is derived using the block vector that is used to obtain the IBC predicted signal.
- [0416] iv. In one example, the intra predicted signal may be obtained using an intra prediction mode which is derived using template base method, such as TIMD.
- [0417] v. In one example, the intra predicted signal may be obtained using an intra prediction mode which is derived using neighboring samples or the gradients of the neighboring samples, such as DIMD.
- [0418] vi. In one example, the intra predicted signal may be obtained using ISP.
- [0419] vii. In one example, the intra predicted signal may be obtained using MIP.
- [0420] viii. In one example, the intra predicted signal may be obtained using MRL.
- [0421] ix. In one example, the intra predicted signal may be intra template matching prediction (IntraTMP).
- [0422] 2. In one example, the weighting parameters used to fuse the IBC predicted signal and intra predicted signal may be signalled or derived.
  - [0423] a. In one example, the weighting parameters may be signalled.
  - [0424] i. In one example, a set of weighting parameters is constructed and an index indicating the weighting parameters may be signalled.
  - [0425] b. In one example, the weighting parameters may be derived using the coding information.
  - [0426] i. In one example, the coding information may refer to the coding mode of neighboring units.
  - [0427] 1) In one example, the weighting parameters may be dependent on whether one or more neighboring units are coded with intra prediction or IBC mode.
  - [0428] ii. In one example, the coding information may refer to the intra prediction mode used to obtain the intra predicted signal.
  - [0429] iii. In one example, the coding information may refer to the block sizes or block dimensions of the current video unit and/or the neighboring video units.
  - [0430] iv. In one example, the weighting parameters may be derived using template matching method (e.g., with the smallest template matching cost).
  - [0431] c. In one example, the weighting parameters may be pre-defined.
- [0432] 3. In one example, the reference area of CIBCIP may be smaller than or equal to the reference area of IBC.
  - [0433] a. In one example, the reference area of CIBCIP may be dependent on the coding information of intra prediction.
  - [0434] i. In one example, the reference area of CIBCIP may be dependent on the intra prediction modes.
  - [0435] b. Alternatively, the reference area of CIBCIP may be different from the reference area of IBC.
- [0436] 4. Whether to and/or how to apply the CIBCIP mode for the video unit may depend on coding information, the coding information may refer to:
  - [0437] a. whether IBC or the intra prediction method is allowed,

- [0438] b. block dimensions and/or block size,
- [0439] c. block depth,
- [0440] d. slice/picture type and/or partition tree type (single, or dual tree, or local dual tree),
- [0441] e. temporal layer identification,
- [0442] f. block location,
- [0443] g. colour component.

Signalling of Combination of Intra Block Copy and Intra Prediction

- [0444] 5. Indication of the CIBCIP mode may be derived on-the-fly.
  - [0445] 6. Indication of the CIBCIP mode may be conditionally signalled wherein the condition may include:
    - [0446] a. whether IBC or the intra prediction method is allowed,
    - [0447] b. block dimensions and/or block size,
    - [0448] c. block depth,
    - [0449] d. slice/picture type and/or partition tree type (single, or dual tree, or local dual tree),
    - [0450] e. temporal layer identification,
    - [0451] f. block location,
    - [0452] g. colour component.
  - [0453] 7. Whether current block is coded with CIBCIP mode may be signalled using one or more syntax elements.
    - [0454] a. In one example, the syntax element may be binarized with fixed length coding, or truncated unary coding, or unary coding, or EG coding, or coded a flag.
    - [0455] b. In one example, the syntax element may be bypass coded or context coded.
      - [0456] i. The context may depend on coded information, such as block dimensions, and/or block size, and/or slice/picture types, and/or the information of neighbouring blocks (adjacent or non-adjacent), and/or the information of other coding tools used for current block, and/or the information of temporal layer.
    - [0457] c. In one example, the indication of CIBCIP mode may be signalled when current video unit is IBC coded.
    - [0458] d. In one example, the syntax element may be signalled before or after the indication of IBC-TM mode, or IBC-MBVD mode.
    - [0459] i. In one example, whether to signal and/or how to the syntax element may be dependent on whether IBC mode, or IBC-TM mode, or IBC-MBVD mode is enabled for the video unit.
    - [0460] e. In one example, the one or more syntax elements may be signalled at sequence header/picture header/SPS/VPS/DPS/DCI/PPS/APS/slice header/tile group header.
    - [0461] f. In one example, the syntax element may be coded in a predictive way.
    - [0462] g. For example, the syntax element of the current block may be predicted by that of a neighboring block.
  - [0463] 8. In one example, for above bullets, the RR-IBC or symmetric IBC method may be used in CIBCIP.
  - [0464] 9. In one example, for above bullets, the RR-IBC or symmetric IBC method may be disabled in CIBCIP.
  - [0465] 10. In one example, the flip type of the IBC predicted part may be set to NO\_FLIP(e.g., 0).
- Intra Prediction with Fused Reference Lines
- [0466] 11. It is proposed to fuse more than one reference lines before being used to derive the intra prediction of a video unit.
    - [0467] a. In one example, the number of reference lines (N) and which reference lines to be fused may be

pre-defined, signalled in the bitstream, or derived on-the-fly, wherein N is an integer larger than 1.

[0468] i. In one example, N may be pre-defined, such as N=2, or N=3.

[0469] ii. In one example, N may be signalled at sequence header/picture header/SPS/VPS/DPS/DCI/PPS/APS/slice header/tile group header.

[0470] iii. In one example, N may be determined based on coding information.

[0471] 1) In one example, the coding information may refer to the block size, block dimensions, or block positions, or coding mode, or intra prediction modes.

[0472] iv. In one example, which reference lines are used in the fusion may be indicated by reference line index.

[0473] 1) The reference line index may be pre-defined, signalled in the bitstream, or derived on-the-fly.

[0474] 2) In one example, one of the reference line indices may be pre-defined and other remaining N-1 reference line indices are signalled.

[0475] 3) In one example, one of the reference line indices may be signalled and other remaining N-1 reference line indices are pre-defined or derived on-the-fly.

[0476] b. In one example, the N reference lines may be fused using the weighting parameters.

[0477] i. In one example,  $L = W_1 * L_1 + W_2 * L_2 + \dots + W_{N-1} * L_N$ , wherein  $L_i$  and  $W_i$  denotes the i-th reference line used in fusion and the corresponding weight, and L denotes the fused reference line used for intra prediction.

[0478] 1) In another example,  $L = (W_1 * L_1 + W_2 * L_2 + \dots + W_{N-1} * L_N) \gg \text{Shift1}$ , wherein  $W_1' + W_2' + \dots + W_{N-1}' = 2^{\text{Shift1}}$ .

[0479] ii. In one example, the weighting parameters may be pre-defined, or signalled in the bitstream, or derived on-the-fly.

[0480] iii. In one example, when the reference line  $L_a$  is closer to current video unit than the reference line  $L_b$ , the corresponding weighting parameter  $W_a$  or  $W_a'$  of  $L_a$  may be equal to or larger than  $W_b$  or  $W_b'$  of  $L_b$ .

[0481] iv. In one example, when N=2,  $W_1=3/4$  and  $W_2=1/4$ , or  $W_1=5/8$  and  $W_2=3/8$ , or  $W_1=1/2$  and  $W_2=1/2$ .

[0482] v. In one example, when N=3,  $W_1=1/2$  and  $W_2=1/4$  and  $W_3=1/4$ , or  $W_1=5/8$  and  $W_2=1/4$  and  $W_3=1/8$ .

[0483] c. In one example, the number of samples in one reference line may be same as the number of samples in another reference line. Denote the fusion for a sample as follows:  $P(x, y) = W_1 * P(x_1, y_1) + W_2 * P(x_2, y_2) + \dots + W_{N-1} * P(x_{N-1}, y_{N-1})$ , wherein  $P(x_i, y_i)$  denotes one sample in the i-th reference line.

[0484] i. In one example, when fusing the above part of the reference lines, the samples in different reference lines with the same horizontal position may be fused.

[0485] 1) In one example,  $x_1 = x_2 = \dots = x_{N-1}$ .

[0486] ii. In one example, when fusing the left part of the reference lines, the samples in different reference lines with the same vertical position may be fused.

[0487] 1) In one example,  $y_1 = y_2 = \dots = y_{N-1}$ .

[0488] d. In one example, the number of samples in one reference line may be different from the number of samples in another reference line. Denote the number

of samples in the reference line  $L_m$  and  $L_n$  as  $S_m$  and  $S_n$ . An example is shown as FIG. 46.

[0489] i. In one example, the number of samples in the fused reference line may be same as that of reference line which has the least number of samples.

[0490] 1) In one example, to fuse a sample of the fused reference line, more samples may be used in the reference line  $L_m$  than the reference line  $L_n$ , wherein the total number of samples in  $L_m$  is larger than  $L_n$ .

[0491] a) In one example, two or more samples may be used in  $L_m$ , and one sample may be used in  $L_n$ .

[0492] 2) In one example,  $S_n$  samples in reference line  $L_m$  may be used in fusion with  $S_n$  samples in reference line  $L_n$ . An example is depicted as FIG. 47.

[0493] ii. In one example, the number of samples in the fused reference line may be same as that of reference line which has the largest number of samples.

[0494] 1) In one example, when  $S_n$  is less than  $S_m$ ,  $(S_m - S_n)$  samples may be padded or derived using the samples in the reference line  $S_n$  and used for fusion. An example is depicted as FIG. 48.

[0495] e. In one example, fusion of the reference lines may be performed after derivation of each reference line.

[0496] i. Alternatively, fusion of the reference lines may be performed during the derivation of the reference lines.

[0497] f. In one example, the derivation of reference samples in the reference lines for the fusion may be same as the derivation of reference samples not used for fusion of the reference lines.

[0498] i. Alternatively, the derivations may be different.

[0499] 1) In one example, how to handle the unavailable reference samples may be different.

[0500] g. In one example, the reference sample filtering may be performed after the fusion of reference line.

[0501] i. Alternatively, the reference sample filtering may be performed before the fusion of reference line.

[0502] 1) In one example, the reference sample filtering may be different for different reference lines.

[0503] 12. Whether to and how to use the fused reference line to derive the intra prediction of current video unit may depend on coding information.

[0504] a. In one example, the coding information may refer to one or more intra prediction methods.

[0505] i. In one example, the fused reference line may be used in conventional intra prediction.

[0506] ii. In one example, the fused reference line may be used in MRL/ISP/MIP/DIMD/TIMD.

[0507] iii. In one example, the fused reference line may be used in conventional chroma intra prediction.

[0508] iv. In one example, the fused reference line may be used in fusion of LM and angular for chroma.

[0509] v. In one example, the fused reference line may be used as an additional method or to replace current intra prediction method.

[0510] b. In one example, the coding information may refer to colour component.

[0511] i. In one example, the fused reference line may be used in intra prediction of luma component.

[0512] ii. In one example, the fused reference line may be used in intra prediction of chroma components.

- [0513] c. In one example, the coding information may refer to intra prediction mode.
- [0514] i. In one example, the fused reference line may be used when DC mode is used.
- [0515] ii. In one example, the fused reference line may be used when Planar mode is used.
- [0516] iii. In one example, the fused reference line may be used when angular intra prediction mode is used.
- [0517] iv. In one example, the fused reference line may be used when angular intra prediction mode which has non-integer slope.
- [0518] v. In one example, the fused reference line may be used for more than one intra prediction modes.
- [0519] d. In one example, the coding information may refer to block size/dimensions of current block and/or neighboring blocks.
- [0520] i. In one example, the fused reference line may be used when the block size of current block is larger than and equal to T1.
- [0521] ii. In another example, the fused reference line may be used when the block size of current block is less than T2.
- [0522] e. In one example, the coding information may refer to slice types, and/or temporal layer, and/or QPs.
- [0523] f. In one example, the fused reference line may be not allowed to be used for video units in a different CTU.
- [0524] 13. In one example, how to fuse the reference lines, and whether to and how to use the fused reference line to derive the intra prediction of current video unit may be signalled in the bitstream.

#### General Aspects

- [0525] 14. In above examples, the video unit may refer to the video unit may refer to colour component/sub-picture/slice/tile/coding tree unit (CTU)/CTU row/groups of CTU/coding unit (CU)/prediction unit (PU)/transform unit (TU)/coding tree block (CTB)/coding block (CB)/prediction block (PB)/transform block (TB)/a block/sub-block of a block/sub-region within a block/any other region that contains more than one sample or pixel.
- [0526] 15. Whether to and/or how to apply the disclosed methods above may be signalled at sequence level/group of pictures level/picture level/slice level/tile group level, such as in sequence header/picture header/SPS/VPS/DPS/DCI/PPS/APS/slice header/tile group header.
- [0527] 16. Whether and/or how to apply the above methods may depend on the following information:
- [0528] a. A message signalled in the DPS/SPS/VPS/PPS/APS/picture header/slice header/tile group header/Largest coding unit (LCU)/Coding unit (CU)/LCU row/group of LCUs/TU/PU block/Video coding unit;
- [0529] b. Position of CU/PU/TU/block/Video coding unit;
- [0530] c. Block dimension of current block and/or its neighbouring blocks;
- [0531] d. Block shape of current block and/or its neighbouring blocks;
- [0532] e. coded mode of a block, e.g., IBC or non-IBC inter mode or non-IBC subblock mode;
- [0533] f. Indication of the colour format (such as 4:2:0, 4:4:4);
- [0534] g. Coding tree structure;
- [0535] h. Slice/tile group type and/or picture type;

- [0536] i. Colour component (e.g., may be only applied on chroma components or luma component);

- [0537] j. Temporal layer ID;

- [0538] k. Profiles/Levels/Tiers of a standard.

[0539] As used herein, the term “video unit” or “video block” may be a sequence, a picture, a slice, a tile, a brick, a subpicture, a coding tree unit (CTU)/coding tree block (CTB), a CTU/CTB row, one or multiple coding units (CUs)/coding blocks (CBs), one or multiple CTUs/CTBs, one or multiple Virtual Pipeline Data Unit (VPDU), a sub-region within a picture/slice/tile/brick. The term “reference line” may refer to a row and/or a column reconstructed samples adjacent to or non-adjacent to the current block, which is used to derive the intra prediction of current video unit via an interpolation filter along a certain direction, and the certain direction is determined by an intra prediction mode (e.g., conventional intra prediction with intra prediction modes), or derive the intra prediction of current video unit via weighting the reference samples of the reference line with a matrix or vector (e.g., MIP).

[0540] FIG. 49 illustrates a flowchart of a method 4900 for video processing in accordance with embodiments of the present disclosure. The method 4900 is implemented during a conversion between a video unit of a video and a bitstream of the video.

[0541] At block 4910, for a conversion between a video unit of a video and a bitstream of the video unit, a combination of intra block copy (IBC) and intra prediction (CIBCIP) is applied to the video unit.

[0542] At block 4920, a prediction of the video unit is derived by combining an IBC predicted signal and an intra predicted signal.

[0543] At block 4930, the conversion is performed based on the prediction of the video unit. In some embodiments, the conversion may include encoding the video unit into the bitstream. Alternatively, or in addition, the conversion may include decoding the video unit from the bitstream. In this way, coding efficiency and coding performance can be improved.

[0544] In some embodiments, the prediction of the video unit is obtained by:  $P(x,y) = w_{ip1} * IP_1(x,y) + w_{ip2} * IP_2(x,y) + \dots + w_{ipn} * IP_n(x,y) + w_{ibc1} * IBC_1(x,y) + w_{ibc2} * IBC_2(x,y) + \dots + w_{ibcm} * IBC_m(x,y)$ , and where  $P(x,y)$  represents the prediction of the video unit,  $IP_k$  represents a prediction generated by a k-th intra prediction,  $IBC_j$  is a prediction generated by a j-th IBC prediction, and  $w_{ipk}$  and  $w_{ibcj}$  represent corresponding weighting values, and k and j are integer numbers.

[0545] In some embodiments, the intra predicted signal is generated by at least one of: an angular intra-prediction mode, a direct currency (DC) mode, a planar mode, a cross-component prediction (CCLM) mode, a multi-model CCLM mode, a left CCLM mode, or an above CCLM mode. In some embodiments, an intra prediction mode of the intra predicted signal is coded using one of the followings to indicate the intra prediction mode: a most probable mode (MPM), a template-based intra mode derivation (TIMD), or a decoder-side intra mode derivation (DIMD). In some embodiments, a set of intra prediction modes is allowed to be used in the CIBCIP.

[0546] In some embodiments, an IBC merge mode is used in the CIBCIP. In some embodiments, one or more block vector (BV) candidates in an IBC merge list are allowed to be used in the CIBCIP. In some embodiments, one or more BV offsets are used for the CIBCIP. In some embodiments, the one or more BV offsets are added to a BV candidate before the BV candidate is used to obtain the IBC predicted signal. In some embodiments, the one or more BV offsets are

indicated or derived. In some embodiments, the IBC merge mode comprises at least one of: a regular IBC merge mode, an IBC merge mode with block vector differences (IBC-MBVD) merge mode, or an IBC-template matching (TM) merge mode.

**[0547]** In some embodiments, an IBC advanced motion vector prediction (AMVP) mode is used in the CIBCIP. In some embodiments, one or more BV predictors in an IBC AMVP list are allowed to be used in the CIBCIP. In some embodiments, a block vector difference (BVD) used in the CIBCIP is indicated using the same way as IBC mode. In some embodiments, a BVD used in the CIBCIP is pre-defined. In some embodiments, a BVD used in the CIBCIP is derived using coding information of the video unit.

**[0548]** In some embodiments, at least one of: a merge index indicating a BV candidate in an IBC merge list or a BVP index indicating a BV predictor in an IBC AMVP list used to obtain the IBC predicted signal is indicated. In some embodiments, a binarization or signaling approach of the merge index or the BVP index is same as that in IBC mode. In some embodiments, at least one of: the merge index or the BVP index is pre-defined. In some embodiments, the merge index is 0 or 1. Alternatively, the BVP index is 0 or 1. In some embodiments, at least one of the merge index or the BVP index is derived using coding information of the video unit. In some embodiments, at least one of the merge index or the BVP index is derived using template matching.

**[0549]** In some embodiments, a construction of IBC merge list or IBC AMVP list used in the CIBCIP is same as that used in IBC mode. Alternatively, the construction of IBC merge list or IBC AMVP list used in the CIBCIP is different from that used in the IBC mode.

**[0550]** In some embodiments, the first number of BV candidates in an IBC merge list that is used for the CIBCIP is less than or equal to the number of BV candidates in the IBC merge list that is used for IBC, and where the first number is an integer number that is larger than 0 and less than or equal to the second number. Alternatively, the first number of BV candidates in an AMVP list that is used for the CIBCIP is less than or equal to the number of BV candidates in the AMVP list that is used for IBC, and where the first number is an integer number that is larger than 0 and less than or equal to the second number. In some embodiments, the first number is one of: 1, 2, 3, 4, 5, or 6. In some embodiments, the first N BV candidates of the IBC merge list are used for the CIBCIP. Alternatively, the first N BV candidates of the AMVP list are used for the CIBCIP, and where N is an integer number.

**[0551]** In some embodiments, a template matching is used to derive/refine a BV that is used to obtain the IBC predicted signal. In some embodiments, a BV offset is derived using template matching, which is added to a BV candidate in an IBC merge list.

**[0552]** In some embodiments, a block vector difference (BVD) is derived using a template matching based approach. In some embodiments, a sign of the BVD is derived using a template matching based method.

**[0553]** In some embodiments, an intra prediction mode or an intra prediction method used to obtain the intra predicted signal is used in the template matching to derive/refine the BV. In some embodiments, a BV list is reordered before being used for the CIBCIP.

**[0554]** In some embodiments, a template matching or bilateral matching cost is used for the reordering of the BV list. In some embodiments, template matching or bilateral matching is used during a construction of the BV list used

for the CIBCIP. In some embodiments, the BV list comprises an IBC merge list or an IBC AMVP list.

**[0555]** In some embodiments, a reordering approach for the BV list used for the CIBCIP is same as that for IBC mode. Alternatively, the reordering approach for the BV list used for the CIBCIP is different from that for the IBC mode.

**[0556]** In some embodiments, the number of BV candidates in the BV list used for the reordering for the CIBCIP is less than or equal to the number of BV candidates used for the reordering for IBC mode. In some embodiments, if an IBC merge mode is used for the CIBCIP, the number of BV candidates is one of 1, 2, 3, or 4. Alternatively, if an IBC AMVP is used for the CIBCIP, the number of BV candidates is one of 1, 2, or 3.

**[0557]** In some embodiments, an intra prediction comprise a conventional intra prediction approach or other intra prediction approach which obtains a prediction block with samples in of the followings excluding IBC: a current slice, a current tile, a current subpicture, a current picture, or other video unit.

**[0558]** In some embodiments, the intra predicted signal is obtained using one of: one or more pre-defined intra prediction modes, one or more of MPMs, an intra prediction mode which is derived using a block vector that is used to obtain the IBC predicted signal, one or more intra prediction modes which are derived using template base approach, one or more intra prediction modes which are derived using neighboring samples or gradients of neighboring samples, an intra sub-partition (ISP), a matrix weighted intra prediction (MIP), a multiple reference line intra prediction (MRL), or an intra template matching prediction (IntraTMP).

**[0559]** In some embodiments, the one or more pre-defined intra prediction modes comprise at least one of: a planar mode, a DC mode, a horizontal mode, or a vertical mode. In some embodiments, the intra predicted signal is and intra template matching prediction (IntraTMP).

**[0560]** In some embodiments, weighting parameters used to combine the IBC predicted signal and the intra predicted signal are indicated or derived. In some embodiments, a set of weighting parameters is constructed and an index indicating the weighting parameters is indicated. In some embodiments, the weighting parameters are derived using the coding information.

**[0561]** In some embodiments, the coding information comprises a coding mode of neighboring units. In some embodiments, the weighting parameters are dependent on whether one or more neighboring units are coded with an intra prediction or IBC mode.

**[0562]** In some embodiments, the coding information comprises an intra prediction mode used to obtain the intra predicted signal. Alternatively, or in addition the coding information comprises at least one of: a block size or a block dimension of the video unit, or a block size or a block dimension of a neighboring video unit. In some embodiments, the weighting parameters are derived using template matching approach. In some embodiments, the weighting parameters are pre-defined.

**[0563]** In some embodiments, a reference area of the CIBCIP is smaller than or equal to a reference area of IBC. In some embodiments, the reference area of the CIBCIP is dependent on coding information of intra prediction. In some embodiments, the reference area of the CIBCIP is dependent on an intra prediction mode. In some embodiments, the reference area of the CIBCIP is different from the reference area of the IBC.

**[0564]** In some embodiments, whether to and/or an approach to apply the CIBCIP to the video unit depends on

coding information. The coding information comprise at least one of: whether IBC or intra prediction approach is allowed, a block dimensions and/or block size, a block depth, a slice type, a picture type, a partition tree type, a temporal layer identification, a block location, or a colour component.

**[0565]** In some embodiments, an indication of the CIBCIP is derived dynamically. In some embodiments, an indication of the CIBCIP is indicated based on a condition. The condition comprise at least one of: whether IBC or intra prediction approach is allowed, a block dimensions and/or block size, a block depth, a slice type, a picture type, a partition tree type, a temporal layer identification, a block location, or a colour component.

**[0566]** In some embodiments, whether the video unit is coded with the CIBCIP is indicated using one or more syntax elements. In some embodiments, the one or more syntax elements are binarized with one of: a fixed length coding, a truncated unary coding, a unary coding, an EG coding, or a coded flag.

**[0567]** In some embodiments, the one or more syntax elements are bypass coded or context coded. In some embodiments, the context depends on coded information. The coded information may include at least one of: block dimensions, a block size, a slice type, a picture type, information of neighboring blocks, information of other coding tools used for the video unit, or information of temporal layer.

**[0568]** In some embodiments, an indication of the CIBCIP is indicated when the video unit is IBC coded. In some embodiments, the one or more syntax elements are indicated before or after an indication of IBC-TM mode, or IBC-MBVD mode.

**[0569]** In some embodiments, whether to indicate and/or an approach to indicate the one or more syntax elements is dependent on whether at least one of: an IBC mode, an IBC-TM mode, an IBC-MBVD mode is enabled for the video unit.

**[0570]** In some embodiments, the one or more syntax elements are indicated at one of the followings: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header. In some embodiments, the one or more syntax elements are coded in a predictive way. In some embodiments, the one or more syntax elements of the video unit are predicted by that of a neighboring block.

**[0571]** In some embodiments, a reconstruction reordered IBC (RR-IBC) or a symmetric IBC approach is used in the CIBCIP. Alternatively, the RR-IBC or symmetric IBC approach are disabled in the CIBCIP. In some embodiments, a flip type of an IBC predicted part is set to NO\_FLIP.

**[0572]** In some embodiments, the video unit comprises at least one of: a color component, a prediction block (PB), a transform block (TB), a coding block (CB), a prediction unit (PU), a transform unit (TU), a coding tree block (CTB), a coding unit (CU), a coding tree unit (CTU), a CTU row, groups of CTU, a slice, a tile, a sub-picture, a block, a sub-region within a block, or a region containing more than one sample or pixel.

**[0573]** In some embodiments, an indication of whether to and/or how to derive the prediction of the video unit by combining the IBC predicted signal and the intra predicted

signal is indicated at one of the followings: sequence level, group of pictures level, picture level, slice level, or tile group level.

**[0574]** In some embodiments, an indication of whether to and/or how to derive the prediction of the video unit by combining the IBC predicted signal and the intra predicted signal is indicated in one of the following: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

**[0575]** In some embodiments, the method **4900** further comprises: determining whether to and/or how to derive the prediction of the video unit by combining the IBC predicted signal and the intra predicted signal based on at least one of the followings: a message indicated in one of: DPS, SPS, VPS, PPS, APS, picture header, slice header, tile group header, largest coding unit (LCU), coding unit (CU), LCU row, group of LCUs, TU, PU block, video coding unit, a position of one of: CU, PU, TU, block, video coding unit, a block dimension of current block and/or its neighbouring blocks, a block shape of current block and/or its neighbouring blocks, a coded mode of the video unit, an indication of colour format, a coding tree structure a slice type, a tile group type, a picture type, a colour component, a temporal layer identity, profiles or levels or Tiers of a standard.

**[0576]** According to further embodiments of the present disclosure, a non-transitory computer-readable recording medium is provided. The non-transitory computer-readable recording medium stores a bitstream of a video which is generated by a method performed by an apparatus for video processing. The method comprises: applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of the video; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and generating the bitstream based on the prediction of the video unit.

**[0577]** According to still further embodiments of the present disclosure, a method for storing bitstream of a video is provided. The method comprises: applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of the video; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; generating the bitstream based on the prediction of the video unit; and storing the bitstream in a non-transitory computer-readable recording medium.

**[0578]** FIG. 50 illustrates a flowchart of a method **5000** for video processing in accordance with embodiments of the present disclosure. The method **5000** is implemented during a conversion between a video unit of a video and a bitstream of the video.

**[0579]** At block **5010**, for a conversion between a video unit of a video and a bitstream of the video unit, a plurality of reference lines is combined.

**[0580]** At block **5020**, an intra prediction of the video unit is derived based on the combined plurality of reference lines.

**[0581]** At block **5030**, the conversion is performed based on the intra prediction. In some embodiments, the conversion may include encoding the video unit into the bitstream. Alternatively, or in addition, the conversion may include decoding the video unit from the bitstream. In this way, coding efficiency and coding performance can be improved.

**[0582]** In some embodiments, the number of reference lines and which reference lines to be combined are one of: pre-defined, indicated in the bitstream, or derived dynami-



cally. Alternatively, the number of reference lines is an integer number that is larger than 1.

[0583] In some embodiments, the number of reference lines is predefined. Alternatively, or in addition, the number of reference lines is 2 or 3.

[0584] In some embodiments, the number of reference lines is indicated in one of the following: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

[0585] In some embodiments, the number of reference lines is determined based on coding information. In some embodiments, the coding information comprise one of: a block size, block dimensions, block positions, a coding mode, or an intra prediction mode.

[0586] In some embodiments, which reference lines are used in the combination are indicated a reference line index. In some embodiments, the reference line index is predefined. Alternatively, the reference line index is indicated in the bitstream. Alternatively, the reference line index is derived dynamically.

[0587] In some embodiments, one of reference line indexes is pre-defined or other remaining reference line indexes are indicated. In some embodiments, one of reference line indexes is indicated and other remaining reference line indexes are pre-defined or derived dynamically.

[0588] In some embodiments, the plurality of reference lines is combined using weighting parameters. In some embodiments, the combined plurality of reference lines are obtained by:  $L = W_1 * L_1 + W_2 * L_2 + \dots + W_{N-1} * L_{N-1}$ , where  $L_i$  and  $W_i$  represent the  $i$ -th reference line used in the combination and a corresponding weight, and  $L$  represent the combined plurality of reference lines used for the intra prediction. In some embodiments,  $L = (W_1 * L_1 + W_2 * L_2 + \dots + W_{N-1} * L_{N-1}) \gg \text{Shift1}$ , where  $W_1 + W_2 + \dots + W_{N-1} = 2^{\text{Shift1}}$ .

[0589] In some embodiments, the weighting parameters are pre-defined. Alternatively, the weighting parameters are indicated in the bitstream. Alternatively, the weighting parameters are derived dynamically.

[0590] In some embodiments, when a reference line  $L_a$  is closer to the video unit than a reference line  $L_b$ , the corresponding weighting parameter  $W_a$  or  $W_a'$  of the reference line  $L_a$  are equal to or larger than  $W_b$  or  $W_b'$  of the reference line  $L_b$ . In some embodiments, if  $N=2$ ,  $W_1=3/4$  and  $W_2=1/4$ , or  $W_1=5/8$  and  $W_2=3/8$ , or  $W_1=1/2$  and  $W_2=1/2$ . In some embodiments, if  $N=3$ ,  $W_1=1/2$  and  $W_2=1/4$  and  $W_3=1/4$ , or  $W_1=5/8$  and  $W_2=1/4$  and  $W_3=1/8$ .

[0591] In some embodiments, the number of samples in one reference line is same as the number of samples in another reference line. In some embodiments, the combination for a sample is obtained as follows:  $P(x, y) = W_1 * P(x_1, y_1) + W_2 * P(x_2, y_2) + \dots + W_{N-1} * P(x_{N-1}, y_{N-1})$ , where  $P(x_i, y_i)$  represent one sample in the  $i$ -th reference line, and  $W_i$  represents a weighting parameter corresponding to the  $i$ -th reference line.

[0592] In some embodiments, when combining an above part of the reference lines, samples in different reference lines with the same horizontal position are combined. In some embodiments,  $x_1 = x_2 = \dots = x_{N-1}$ .

[0593] In some embodiments, when a left part of the reference lines, samples in different reference lines with the same vertical position are combined. In some embodiments,  $y_1 = y_2 = \dots = y_{N-1}$ .

[0594] In some embodiments, the number of samples in one reference line is different from the number of samples in another reference line. In some embodiments, the number of samples in the combined plurality of reference lines is same as that of reference line which has the least number of samples.

[0595] In some embodiments, the number of samples used in a reference line  $L_m$  is larger than the number of samples in a reference line  $L_n$  to combine a sample of combined reference lines, where the total number of samples in the reference line  $L_m$  is larger than that in the reference line  $L_n$ . In some embodiments, two or more samples are used in the reference line  $L_m$ , and one sample is used in the reference line  $L_n$ . In some embodiments,  $S_n$  samples in the reference line  $L_m$  are used in combination with  $S_n$  samples in the reference line  $L_n$ , where  $S_n$  is an integer number and represent the number of samples in the reference line  $L_n$ . In some embodiments, the number of samples in the combined plurality of reference lines is same as that of reference line which has the largest number of samples.

[0596] In some embodiments, if the number of samples in a reference line  $L_n$  is less than the number of samples in a reference line  $L_m$ , a difference between the number of samples in the reference line  $L_n$  and the number of samples in a reference line  $L_m$  is padded or derived using samples in the reference line  $L_n$  and used for the combination. In some embodiments, the combination of the plurality of reference lines are performed after derivation of the plurality of reference lines. Alternatively, the combination of the plurality of reference lines are performed during the derivation of the plurality of reference lines.

[0597] In some embodiments, a derivation of reference samples in the plurality of reference lines for the combination is same as a derivation of reference samples not used for the combination. Alternatively, the derivation of reference samples in the plurality of reference lines for the combination is different from the derivation of reference samples not used for the combination.

[0598] In some embodiments, unavailable reference samples are processed in different way in the derivation of reference samples in the plurality of reference lines for the combination and the derivation of reference samples not used for the combination.

[0599] In some embodiments, a reference sample filtering is performed after the combination of the plurality of reference lines. Alternatively, the reference sample filtering is performed before the combination of the plurality of reference lines. In some embodiments, the reference sample filtering is different for different reference lines.

[0600] In some embodiments, whether to and/or an approach to use the combined plurality of reference lines to derive the intra prediction of the video unit depends on coding information. In some embodiments, the coding information comprises one or more intra prediction methods.

[0601] In some embodiments, the combined plurality of reference line is used in at least one of the followings: a conventional intra prediction, an intra sub-partition (ISP), a matrix weighted intra prediction (MIP), a multiple reference line intra prediction (MRL), a template-based intra mode derivation (TIMD), or a decoder-side intra mode derivation (DIMD), a conventional chroma intra prediction, a combination of LM and angular for chroma, or an additional method or to replace a current intra prediction method.

[0602] In some embodiments, the coding information comprises a color component. In some embodiments, the combined plurality of reference lines is used in an intra

prediction of luma component. Alternatively, the combined plurality of reference lines is used in an intra prediction of chroma components.

**[0603]** In some embodiments, the coding information comprises intra prediction mode. In some embodiments, the combined plurality of reference lines is used when a DC mode is used. Alternatively, the combined plurality of reference lines is used when a planar mode is used. Alternatively, the combined plurality of reference lines is used when an angular intra prediction mode is used. Alternatively, the combined plurality of reference lines is used when an angular intra prediction mode which has non-integer slope is used. Alternatively, the combined plurality of reference lines is used for more than one intra prediction modes.

**[0604]** In some embodiments, the coding information comprises at least one of: a block size of the video unit, block dimensions of the video unit, a block size of neighboring blocks, or block dimensions of neighboring blocks. In some embodiments, the combined plurality of reference lines is used when the block size of the video unit is larger than and equal to a first threshold. Alternatively, the combined plurality of reference lines is used when the block size of the video unit block is less than a second threshold.

**[0605]** In some embodiments, the coding information comprises at least one of: a slice type, a temporal layer, or a quantization parameter (QP). In some embodiments, the combined plurality of reference lines is not allowed to be used for video units in a different CTU.

**[0606]** In some embodiments, an approach of combining the plurality of reference lines is indicated in the bitstream. Alternatively, or in addition, whether to and/or an approach to use the combined plurality of reference lines to derive the intra prediction of the video unit is indicated in the bitstream.

**[0607]** In some embodiments, the video unit comprises at least one of: a color component, a prediction block (PB), a transform block (TB), a coding block (CB), a prediction unit (PU), a transform unit (TU), a coding tree block (CTB), a coding unit (CU), a coding tree unit (CTU), a CTU row, groups of CTU, a slice, a tile, a sub-picture, a block, a sub-region within a block, or a region containing more than one sample or pixel. In some embodiments, an indication of whether to and/or how to derive the intra prediction of the video unit based on the combined plurality of reference lines is indicated at one of the followings: sequence level, group of pictures level, picture level, slice level, or tile group level.

**[0608]** In some embodiments, an indication of whether to and/or how to derive the intra prediction of the video unit based on the combined plurality of reference lines is indicated in one of the following: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

**[0609]** In some embodiments, the method 5000 further comprises: determining whether to and/or how to derive the intra prediction of the video unit based on the combined plurality of reference lines based on at least one of the followings: a message indicated in one of: DPS, SPS, VPS, PPS, APS, picture header, slice header, tile group header, largest coding unit (LCU), coding unit (CU), LCU row, group of LCUs, TU, PU block, video coding unit, a position of one of: CU, PU, TU, block, video coding unit, a block dimension of current block and/or its neighbouring blocks, a block shape of current block and/or its neighbouring blocks, a coded mode of the video unit, an indication of

colour format, a coding tree structure a slice type, a tile group type, a picture type, a colour component, a temporal layer identity, profiles or levels or Tiers of a standard.

**[0610]** According to further embodiments of the present disclosure, a non-transitory computer-readable recording medium is provided. The non-transitory computer-readable recording medium stores a bitstream of a video which is generated by a method performed by an apparatus for video processing. The method comprises: combining a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; and generating the bitstream based on the prediction of the video unit.

**[0611]** According to still further embodiments of the present disclosure, a method for storing bitstream of a video is provided. The method comprises: combining a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; generating the bitstream based on the prediction of the video unit; and storing the bitstream in a non-transitory computer-readable recording medium.

**[0612]** Implementations of the present disclosure can be described in view of the following clauses, the features of which can be combined in any reasonable manner.

**[0613]** Clause 1. A method of video processing, comprising: applying, for a conversion between a video unit of a video and a bitstream of the video unit, a combination of intra block copy (IBC) and intra prediction (CIBCIP) to the video unit; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and performing the conversion based on the prediction of the video unit.

**[0614]** Clause 2. The method of clause 1, wherein the prediction of the video unit is obtained by:  $P(x,y) = wip1 * IP1(x,y) + wip2 * IP2(x,y) + \dots + wipn * IPn(x,y) + wibc1 * IBC_1(x,y) + wibc2 * IBC_2(x,y) + \dots + wibcm * IBC_m(x,y)$ , and wherein  $P(x,y)$  represents the prediction of the video unit,  $IPk$  represents a prediction generated by a k-th intra prediction,  $IBC_j$  is a prediction generated by a j-th IBC prediction, and  $wipk$  and  $wibcj$  represent corresponding weighting values, and  $k$  and  $j$  are integer numbers.

**[0615]** Clause 3. The method of clause 1, wherein the intra predicted signal is generated by at least one of: an angular intra-prediction mode, a direct current (DC) mode, a planar mode, a cross-component prediction (CCLM) mode, a multi-model CCLM mode, a left CCLM mode, or an above CCLM mode.

**[0616]** Clause 4. The method of clause 1, wherein an intra prediction mode of the intra predicted signal is coded using one of the followings to indicate the intra prediction mode: a most probable mode (MPM), a template-based intra mode derivation (TIMD), or a decoder-side intra mode derivation (DIMD).

**[0617]** Clause 5. The method of clause 1, wherein a set of intra prediction modes is allowed to be used in the CIBCIP.

**[0618]** Clause 6. The method of clause 1, wherein an IBC merge mode is used in the CIBCIP.

**[0619]** Clause 7. The method of clause 6, wherein one or more block vector (BV) candidates in an IBC merge list are allowed to be used in the CIBCIP.

**[0620]** Clause 8. The method of clause 6, wherein one or more BV offsets are used for the CIBCIP.

**[0621]** Clause 9. The method of clause 8, wherein the one or more BV offsets are added to a BV candidate before the BV candidate is used to obtain the IBC predicted signal.

**[0622]** Clause 10. The method of clause 8, wherein the one or more BV offsets are indicated or derived.

[0623] Clause 11. The method of clause 6, wherein the IBC merge mode comprises at least one of: a regular IBC merge mode, an IBC merge mode with block vector differences (IBC-MBVD) merge mode, or an IBC-template matching (TM) merge mode.

[0624] Clause 12. The method of clause 1, wherein an IBC advanced motion vector prediction (AMVP) mode is used in the CIBCIP.

[0625] Clause 13. The method of clause 12, wherein one or more BV predictors in an IBC AMVP list are allowed to be used in the CIBCIP.

[0626] Clause 14. The method of clause 12, wherein a block vector difference (BVD) used in the CIBCIP is indicated using the same way as IBC mode.

[0627] Clause 15. The method of clause 12, wherein a BVD used in the CIBCIP is pre-defined.

[0628] Clause 16. The method of clause 12, wherein a BVD used in the CIBCIP is derived using coding information of the video unit.

[0629] Clause 17. The method of clause 1, wherein at least one of: a merge index indicating a BV candidate in an IBC merge list or a BVP index indicating a BV predictor in an IBC AMVP list used to obtain the IBC predicted signal is indicated.

[0630] Clause 18. The method of clause 17, wherein a binarization or signaling approach of the merge index or the BVP index is same as that in IBC mode.

[0631] Clause 19. The method of clause 17, wherein at least one of: the merge index or the BVP index is pre-defined.

[0632] Clause 20. The method of clause 19, wherein the merge index is 0 or 1, or wherein the BVP index is 0 or 1.

[0633] Clause 21. The method of clause 17, wherein at least one of the merge index or the BVP index is derived using coding information of the video unit.

[0634] Clause 22. The method of clause 17, wherein at least one of the merge index or the BVP index is derived using template matching.

[0635] Clause 23. The method of clause 1, wherein a construction of IBC merge list or IBC AMVP list used in the CIBCIP is same as that used in IBC mode, or wherein the construction of IBC merge list or IBC AMVP list used in the CIBCIP is different from that used in the IBC mode.

[0636] Clause 24. The method of clause 1, wherein the first number of BV candidates in an IBC merge list that is used for the CIBCIP is less than or equal to the number of BV candidates in the IBC merge list that is used for IBC, and wherein the first number is an integer number that is larger than 0 and less than or equal to the second number, and/or wherein the first number of BV candidates in an AMVP list that is used for the CIBCIP is less than or equal to the number of BV candidates in the AMVP list that is used for IBC, and wherein the first number is an integer number that is larger than 0 and less than or equal to the second number.

[0637] Clause 25. The method of clause 24, wherein the first number is one of: 1, 2, 3, 4, 5, or 6.

[0638] Clause 26. The method of clause 24, wherein the first N BV candidates of the IBC merge list are used for the CIBCIP, and/or wherein the first N BV candidates of the AMVP list are used for the CIBCIP, and wherein N is an integer number.

[0639] Clause 27. The method of clause 1, wherein a template matching is used to derive/refine a BV that is used to obtain the IBC predicted signal.

[0640] Clause 28. The method of clause 27, wherein a BV offset is derived using template matching, which is added to a BV candidate in an IBC merge list.

[0641] Clause 29. The method of clause 27, wherein a block vector difference (BVD) is derived using a template matching based approach.

[0642] Clause 30. The method of clause 27, wherein a sign of the BVD is derived using a template matching based method.

[0643] Clause 31. The method of clause 27, wherein an intra prediction mode or an intra prediction method used to obtain the intra predicted signal is used in the template matching to derive/refine the BV.

[0644] Clause 32. The method of clause 1, wherein a BV list is reordered before being used for the CIBCIP.

[0645] Clause 33. The method of clause 32, wherein a template matching or bilateral matching cost is used for the reordering of the BV list.

[0646] Clause 34. The method of clause 32, wherein template matching or bilateral matching is used during a construction of the BV list used for the CIBCIP.

[0647] Clause 35. The method of clause 32, wherein the BV list comprises an IBC merge list or an IBC AMVP list.

[0648] Clause 36. The method of clause 32, wherein a reordering approach for the BV list used for the CIBCIP is same as that for IBC mode, or wherein the reordering approach for the BV list used for the CIBCIP is different from that for the IBC mode.

[0649] Clause 37. The method of clause 36, wherein the number of BV candidates in the BV list used for the reordering for the CIBCIP is less than or equal to the number of BV candidates used for the reordering for IBC mode.

[0650] Clause 38. The method of clause 37, wherein if an IBC merge mode is used for the CIBCIP, the number of BV candidates is one of 1, 2, 3, or 4, or wherein if an IBC AMVP is used for the CIBCIP, the number of BV candidates is one of 1, 2, or 3.

[0651] Clause 39. The method of clause 1, wherein an intra prediction comprise a conventional intra prediction approach or other intra prediction approach which obtains a prediction block with samples in of the followings excluding IBC: a current slice, a current tile, a current subpicture, a current picture, or other video unit.

[0652] Clause 40. The method of clause 39, wherein the intra predicted signal is obtained using one of: one or more pre-defined intra prediction modes, one or more of MPMs, an intra prediction mode which is derived using a block vector that is used to obtain the IBC predicted signal, one or more intra prediction modes which are derived using template base approach, one or more intra prediction modes which are derived using neighboring samples or gradients of neighboring samples, an intra sub-partition (ISP), a matrix weighted intra prediction (MIP), a multiple reference line intra prediction (MRL), or an intra template matching prediction (IntraTMP).

[0653] Clause 41. The method of clause 40, wherein the one or more pre-defined intra prediction modes comprise at least one of: a planar mode, a DC mode, a horizontal mode, or a vertical mode.

[0654] Clause 42. The method of clause 1, wherein weighting parameters used to combine the IBC predicted signal and the intra predicted signal are indicated or derived.

[0655] Clause 43. The method of clause 42, wherein a set of weighting parameters is constructed and an index indicating the weighting parameters is indicated.

[0656] Clause 44. The method of clause 42, wherein the weighting parameters are derived using the coding information.

[0657] Clause 45. The method of clause 44, wherein the coding information comprises a coding mode of neighboring units.

[0658] Clause 46. The method of clause 45, wherein the weighting parameters are dependent on whether one or more neighboring units are coded with an intra prediction or IBC mode.

[0659] Clause 47. The method of clause 44, wherein the coding information comprises an intra prediction mode used to obtain the intra predicted signal.

[0660] Clause 48. The method of clause 44, wherein the coding information comprises at least one of: a block size or a block dimension of the video unit, or a block size or a block dimension of a neighboring video unit.

[0661] Clause 49. The method of clause 42, wherein the weighting parameters are derived using template matching approach.

[0662] Clause 50. The method of clause 42, wherein the weighting parameters are pre-defined.

[0663] Clause 51. The method of clause 1, wherein a reference area of the CIBCIP is smaller than or equal to a reference area of IBC.

[0664] Clause 52. The method of clause 51, wherein the reference area of the CIBCIP is dependent on coding information of intra prediction.

[0665] Clause 53. The method of clause 51, wherein the reference area of the CIBCIP is dependent on an intra prediction mode.

[0666] Clause 54. The method of clause 51, wherein the reference area of the CIBCIP is different from the reference area of the IBC.

[0667] Clause 55. The method of clause 1, wherein whether to and/or an approach to apply the CIBCIP to the video unit depends on coding information, wherein the coding information comprise at least one of: whether IBC or intra prediction approach is allowed, a block dimensions and/or block size, a block depth, a slice type, a picture type, a partition tree type, a temporal layer identification, a block location, or a colour component.

[0668] Clause 56. The method of clause 1, wherein an indication of the CIBCIP is derived dynamically.

[0669] Clause 57. The method of clause 1, wherein an indication of the CIBCIP is indicated based on a condition, wherein the condition comprise at least one of: whether IBC or intra prediction approach is allowed, a block dimensions and/or block size, a block depth, a slice type, a picture type, a partition tree type, a temporal layer identification, a block location, or a colour component.

[0670] Clause 58. The method of clause 1, wherein whether the video unit is coded with the CIBCIP is indicated using one or more syntax elements.

[0671] Clause 59. The method of clause 58, wherein the one or more syntax elements are binarized with one of: a fixed length coding, a truncated unary coding, a unary coding, an EG coding, or a coded flag.

[0672] Clause 60. The method of clause 58, wherein the one or more syntax elements are bypass coded or context coded.

[0673] Clause 61. The method of clause 60, wherein the context depends on coded information, and wherein the coded information comprises at least one of: block dimensions, a block size, a slice type, a picture type, information of neighboring blocks, information of other coding tools used for the video unit, or information of temporal layer.

[0674] Clause 62. The method of clause 58, wherein an indication of the CIBCIP is indicated when the video unit is IBC coded.

[0675] Clause 63. The method of clause 58, wherein the one or more syntax elements are indicated before or after an indication of IBC-TM mode, or IBC-MBVD mode.

[0676] Clause 64. The method of clause 63, wherein whether to indicate and/or an approach to indicate the one or more syntax elements is dependent on whether at least one of: an IBC mode, an IBC-TM mode, an IBC-MBVD mode is enabled for the video unit.

[0677] Clause 65. The method of clause 58, wherein the one or more syntax elements are indicated at one of the followings: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

[0678] Clause 66. The method of clause 58, wherein the one or more syntax elements are coded in a predictive way.

[0679] Clause 67. The method of clause 58, wherein the one or more syntax elements of the video unit are predicted by that of a neighboring block.

[0680] Clause 68. The method of any of clauses 1-67, wherein a reconstruction reordered IBC (RR-IBC) or a symmetric IBC approach is used in the CIBCIP, or wherein the RR-IBC or symmetric IBC approach are disabled in the CIBCIP.

[0681] Clause 69. The method of clause 1, wherein a flip type of an IBC predicted part is set to NO\_FLIP.

[0682] Clause 70. The method of any of clauses 1-69, wherein the video unit comprises at least one of: a color component, a prediction block (PB), a transform block (TB), a coding block (CB), a prediction unit (PU), a transform unit (TU), a coding tree block (CTB), a coding unit (CU), a coding tree unit (CTU), a CTU row, groups of CTU, a slice, a tile, a sub-picture, a block, a sub-region within a block, or a region containing more than one sample or pixel.

[0683] Clause 71. The method of any of clauses 1-69, wherein an indication of whether to and/or how to derive the prediction of the video unit by combining the IBC predicted signal and the intra predicted signal is indicated at one of the followings: sequence level, group of pictures level, picture level, slice level, or tile group level.

[0684] Clause 72. The method of any of clauses 1-69, wherein an indication of whether to and/or how to derive the prediction of the video unit by combining the IBC predicted signal and the intra predicted signal is indicated in one of the following: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

[0685] Clause 73. The method of any of clauses 1-69, further comprising: determining whether to and/or how to derive the prediction of the video unit by combining the IBC predicted signal and the intra predicted signal based on at least one of the followings: a message indicated in one of: DPS, SPS, VPS, PPS, APS, picture header, slice header, tile group header, largest coding unit (LCU), coding unit (CU), LCU row, group of LCUs, TU, PU block, video coding unit, a position of one of: CU, PU, TU, block, video coding unit, a block dimension of current block and/or its neighbouring blocks, a block shape of current block and/or its neighbouring blocks, a coded mode of the video unit, an indication of colour format, a coding tree structure a slice type, a tile group type, a picture type, a colour component, a temporal layer identity, profiles or levels or Tiers of a standard.

[0686] Clause 74. A method of video processing, comprising: combining, for a conversion between a video unit of a

video and a bitstream of the video unit, a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; and performing the conversion based on the intra prediction.

**[0687]** Clause 75. The method of clause 74, wherein the number of reference lines and which reference lines to be combined are one of: pre-defined, indicated in the bitstream, or derived dynamically, wherein the number of reference lines is an integer number that is larger than 1.

**[0688]** Clause 76. The method of clause 75, wherein the number of reference lines is predefined, and/or wherein the number of reference lines is 2 or 3.

**[0689]** Clause 77. The method of clause 75, wherein the number of reference lines is indicated in one of the following: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

**[0690]** Clause 78. The method of clause 75, wherein the number of reference lines is determined based on coding information.

**[0691]** Clause 79. The method of clause 78, wherein the coding information comprise one of: a block size, block dimensions, block positions, a coding mode, or an intra prediction mode.

**[0692]** Clause 80. The method of clause 75, wherein which reference lines are used in the combination are indicated a reference line index.

**[0693]** Clause 81. The method of clause 80, wherein the reference line index is pre-defined, or wherein the reference line index is indicated in the bitstream, or wherein the reference line index is derived dynamically.

**[0694]** Clause 82. The method of clause 75, wherein one of reference line indexes is pre-defined or other remaining reference line indexes are indicated.

**[0695]** Clause 83. The method of clause 75, wherein one of reference line indexes is indicated and other remaining reference line indexes are pre-defined or derived dynamically.

**[0696]** Clause 84. The method of clause 74, wherein the plurality of reference lines are combined using weighting parameters.

**[0697]** Clause 85. The method of clause 84, wherein the combined plurality of reference lines are obtained by:  $L=W1*L1+W2*L2+\dots+WN-1*LN$ , wherein  $L_i$  and  $W_i$  represent the  $i$ -th reference line used in the combination and a corresponding weight, and  $L$  represent the combined plurality of reference lines used for the intra prediction.

**[0698]** Clause 86. The method of clause 85, wherein  $L=(W1*L1+W2*L2+\dots+WN-1*LN)>>Shift1$ , wherein  $W1+W2+\dots+WN-1=2^{Shift1}$ .

**[0699]** Clause 87. The method of clause 84, wherein the weighting parameters are pre-defined, or wherein the weighting parameters are indicated in the bitstream, or wherein the weighting parameters are derived dynamically.

**[0700]** Clause 88. The method of clause 84, wherein when a reference line  $La$  is closer to the video unit than a reference line  $Lb$ , the corresponding weighting parameter  $Wa$  or  $Wa'$  of the reference line  $La$  are equal to or larger than  $Wb$  or  $Wb'$  of the reference line  $Lb$ .

**[0701]** Clause 89. The method of clause 85 or 86, wherein if  $N=2$ ,  $W1=3/4$  and  $W2=1/4$ , or  $W1=5/8$  and  $W2=3/8$ , or  $W1=1/2$  and  $W2=1/2$ .

**[0702]** Clause 90. The method of clause 85 or 86, wherein if  $N=3$ ,  $W1=1/2$  and  $W2=1/4$  and  $W3=1/4$ , or  $W1=5/8$  and  $W2=1/4$  and  $W3=1/8$ .

**[0703]** Clause 91. The method of clause 74, wherein the number of samples in one reference line is same as the number of samples in another reference line.

**[0704]** Clause 92. The method of clause 91, wherein the combination for a sample is obtained as follows:  $P(x, y)=W1*P(x1, y1)+W2*P(x2, y2)+\dots+WN-1*P(xN-1, yN-1)$ , wherein  $P(x_i, y_i)$  represent one sample in the  $i$ -th reference line, and  $W_i$  represents a weighting parameter corresponding to the  $i$ -th reference line.

**[0705]** Clause 93. The method of clause 92, wherein when combining an above part of the reference lines, samples in different reference lines with the same horizontal position are combined.

**[0706]** Clause 94. The method of clause 93, wherein  $x1=x2=\dots=xN-1$ .

**[0707]** Clause 95. The method of clause 92, wherein when a left part of the reference lines, samples in different reference lines with the same vertical position are combined.

**[0708]** Clause 96. The method of clause 95, wherein  $y1=y2=\dots=yN-1$ .

**[0709]** Clause 97. The method of clause 74, wherein the number of samples in one reference line is different from the number of samples in another reference line.

**[0710]** Clause 98. The method of clause 97, wherein the number of samples in the combined plurality of reference lines is same as that of reference line which has the least number of samples.

**[0711]** Clause 99. The method of clause 98, wherein the number of samples used in a reference line  $Lm$  is larger than the number of samples in a reference line  $Ln$  to combine a sample of combined reference lines, wherein the total number of samples in the reference line  $Lm$  is larger than that in the reference line  $Ln$ .

**[0712]** Clause 100. The method of clause 99, wherein two or more samples are used in the reference line  $Lm$ , and one sample is used in the reference line  $Ln$ .

**[0713]** Clause 101. The method of clause 98, wherein  $S_n$  samples in the reference line  $Lm$  are used in combination with  $S_n$  samples in the reference line  $Ln$ , wherein  $S_n$  is an integer number and represent the number of samples in the reference line  $Ln$ .

**[0714]** Clause 102. The method of clause 97, wherein the number of samples in the combined plurality of reference lines is same as that of reference line which has the largest number of samples.

**[0715]** Clause 103. The method of clause 102, wherein if the number of samples in a reference line  $Ln$  is less than the number of samples in a reference line  $Lm$ , a difference between the number of samples in the reference line  $Ln$  and the number of samples in a reference line  $Lm$  is padded or derived using samples in the reference line  $Ln$  and used for the combination.

**[0716]** Clause 104. The method of clause 74, wherein the combination of the plurality of reference lines are performed after derivation of the plurality of reference lines, or wherein the combination of the plurality of reference lines are performed during the derivation of the plurality of reference lines.

**[0717]** Clause 105. The method of clause 74, wherein a derivation of reference samples in the plurality of reference lines for the combination is same as a derivation of reference samples not used for the combination, or wherein the derivation of reference samples in the plurality of reference lines for the combination is different from the derivation of reference samples not used for the combination.

**[0718]** Clause 106. The method of clause 105, wherein unavailable reference samples are processed in different way

in the derivation of reference samples in the plurality of reference lines for the combination and the derivation of reference samples not used for the combination.

**[0719]** Clause 107. The method of clause 74, wherein a reference sample filtering is performed after the combination of the plurality of reference lines, or wherein the reference sample filtering is performed before the combination of the plurality of reference lines.

**[0720]** Clause 108. The method of clause 107, wherein the reference sample filtering is different for different reference lines.

**[0721]** Clause 109. The method of clause 74, wherein whether to and/or an approach to use the combined plurality of reference lines to derive the intra prediction of the video unit depends on coding information.

**[0722]** Clause 110. The method of clause 109, wherein the coding information comprises one or more intra prediction methods.

**[0723]** Clause 111. The method of clause 110, wherein the combined plurality of reference line is used in at least one of the followings: a conventional intra prediction, an intra sub-partition (ISP), a matrix weighted intra prediction (MIP), a multiple reference line intra prediction (MRL), a template-based intra mode derivation (TIMD), or a decoder-side intra mode derivation (DIMD), a conventional chroma intra prediction, a combination of LM and angular for chroma, or an additional method or to replace a current intra prediction method.

**[0724]** Clause 112. The method of clause 109, wherein the coding information comprises a color component.

**[0725]** Clause 113. The method of clause 112, wherein the combined plurality of reference lines is used in an intra prediction of luma component, or wherein the combined plurality of reference lines is used in an intra prediction of chroma components.

**[0726]** Clause 114. The method of clause 109, wherein the coding information comprises intra prediction mode.

**[0727]** Clause 115. The method of clause 114, wherein the combined plurality of reference lines is used when a DC mode is used, or wherein the combined plurality of reference lines is used when a planar mode is used, or wherein the combined plurality of reference lines is used when an angular intra prediction mode is used, or wherein the combined plurality of reference lines is used when an angular intra prediction mode which has non-integer slope is used, or wherein the combined plurality of reference lines is used for more than one intra prediction modes.

**[0728]** Clause 116. The method of clause 109, wherein the coding information comprises at least one of: a block size of the video unit, block dimensions of the video unit, a block size of neighboring blocks, or block dimensions of neighboring blocks.

**[0729]** Clause 117. The method of clause 116, wherein the combined plurality of reference lines is used when the block size of the video unit is larger than and equal to a first threshold, or wherein the combined plurality of reference lines is used when the block size of the video unit block is less than a second threshold.

**[0730]** Clause 118. The method of clause 109, wherein the coding information comprises at least one of: a slice type, a temporal layer, or a quantization parameter (QP).

**[0731]** Clause 119. The method of clause 109, wherein the combined plurality of reference lines is not allowed to be used for video units in a different CTU.

**[0732]** Clause 120. The method of clause 74, wherein an approach of combining the plurality of reference lines is indicated in the bitstream, and/or wherein whether to and/or

an approach to use the combined plurality of reference lines to derive the intra prediction of the video unit is indicated in the bitstream.

**[0733]** Clause 121. The method of any of clauses 74-120, wherein the video unit comprises at least one of: a color component, a prediction block (PB), a transform block (TB), a coding block (CB), a prediction unit (PU), a transform unit (TU), a coding tree block (CTB), a coding unit (CU), a coding tree unit (CTU), a CTU row, groups of CTU, a slice, a tile, a sub-picture, a block, a sub-region within a block, or a region containing more than one sample or pixel.

**[0734]** Clause 122. The method of any of clauses 74-120, wherein an indication of whether to and/or how to derive the intra prediction of the video unit based on the combined plurality of reference lines is indicated at one of the followings: sequence level, group of pictures level, picture level, slice level, or tile group level.

**[0735]** Clause 123. The method of any of clauses 74-120, wherein an indication of whether to and/or how to derive the intra prediction of the video unit based on the combined plurality of reference lines is indicated in one of the following: a sequence header, a picture header, a sequence parameter set (SPS), a video parameter set (VPS), a dependency parameter set (DPS), a decoding capability information (DCI), a picture parameter set (PPS), an adaptation parameter sets (APS), a slice header, or a tile group header.

**[0736]** Clause 124. The method of any of clauses 74-120, further comprising: determining whether to and/or how to derive the intra prediction of the video unit based on the combined plurality of reference lines based on at least one of the followings: a message indicated in one of: DPS, SPS, VPS, PPS, APS, picture header, slice header, tile group header, largest coding unit (LCU), coding unit (CU), LCU row, group of LCUs, TU, PU block, video coding unit, a position of one of: CU, PU, TU, block, video coding unit, a block dimension of current block and/or its neighbouring blocks, a block shape of current block and/or its neighbouring blocks, a coded mode of the video unit, an indication of colour format, a coding tree structure a slice type, a tile group type, a picture type, a colour component, a temporal layer identity, profiles or levels or Tiers of a standard.

**[0737]** Clause 125. The method of any of clauses 1-124, wherein the conversion includes encoding the video unit into the bitstream.

**[0738]** Clause 126. The method of any of clauses 1-124, wherein the conversion includes decoding the video unit from the bitstream.

**[0739]** Clause 127. An apparatus for video processing comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to perform a method in accordance with any of clauses 1-126.

**[0740]** Clause 128. A non-transitory computer-readable storage medium storing instructions that cause a processor to perform a method in accordance with any of clauses 1-126.

**[0741]** Clause 129. A non-transitory computer-readable recording medium storing a bitstream of a video which is generated by a method performed by an apparatus for video processing, wherein the method comprises: applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of the video; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and generating the bitstream based on the prediction of the video unit.

**[0742]** Clause 130. A method for storing a bitstream of a video, comprising: applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of

the video; deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; generating the bitstream based on the prediction of the video unit; and storing the bitstream in a non-transitory computer-readable recording medium.

[0743] Clause 131. A non-transitory computer-readable recording medium storing a bitstream of a video which is generated by a method performed by an apparatus for video processing, wherein the method comprises: combining a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines; and generating the bitstream based on the prediction of the video unit.

[0744] Clause 132. A method for storing a bitstream of a video, comprising: combining a plurality of reference lines; deriving an intra prediction of the video unit based on the combined plurality of reference lines;

[0745] generating the bitstream based on the prediction of the video unit; and storing the bitstream in a non-transitory computer-readable recording medium.

#### Example Device

[0746] FIG. 51 illustrates a block diagram of a computing device 5100 in which various embodiments of the present disclosure can be implemented. The computing device 5100 may be implemented as or included in the source device 110 (or the video encoder 114 or 200) or the destination device 120 (or the video decoder 124 or 300).

[0747] It would be appreciated that the computing device 5100 shown in FIG. 51 is merely for purpose of illustration, without suggesting any limitation to the functions and scopes of the embodiments of the present disclosure in any manner.

[0748] As shown in FIG. 51, the computing device 5100 includes a general-purpose computing device 5100. The computing device 5100 may at least comprise one or more processors or processing units 5110, a memory 5120, a storage unit 5130, one or more communication units 5140, one or more input devices 5150, and one or more output devices 5160.

[0749] In some embodiments, the computing device 5100 may be implemented as any user terminal or server terminal having the computing capability. The server terminal may be a server, a large-scale computing device or the like that is provided by a service provider. The user terminal may for example be any type of mobile terminal, fixed terminal, or portable terminal, including a mobile phone, station, unit, device, multimedia computer, multimedia tablet, Internet node, communicator, desktop computer, laptop computer, notebook computer, netbook computer, tablet computer, personal communication system (PCS) device, personal navigation device, personal digital assistant (PDA), audio/video player, digital camera/video camera, positioning device, television receiver, radio broadcast receiver, E-book device, gaming device, or any combination thereof, including the accessories and peripherals of these devices, or any combination thereof.

[0750] It would be contemplated that the computing device 5100 can support any type of interface to a user (such as “wearable” circuitry and the like).

[0751] The processing unit 5110 may be a physical or virtual processor and can implement various processes based on programs stored in the memory 5120. In a multi-processor system, multiple processing units execute computer executable instructions in parallel so as to improve the parallel processing capability of the computing device 5100.

The processing unit 5110 may also be referred to as a central processing unit (CPU), a microprocessor, a controller or a microcontroller.

[0752] The computing device 5100 typically includes various computer storage medium. Such medium can be any medium accessible by the computing device 5100, including, but not limited to, volatile and non-volatile medium, or detachable and non-detachable medium. The memory 5120 can be a volatile memory (for example, a register, cache, Random Access Memory (RAM)), a non-volatile memory (such as a Read-Only Memory (ROM), Electrically Erasable Programmable Read-Only Memory (EEPROM), or a flash memory), or any combination thereof. The storage unit 5130 may be any detachable or non-detachable medium and may include a machine-readable medium such as a memory, flash memory drive, magnetic disk or another other media, which can be used for storing information and/or data and can be accessed in the computing device 5100.

[0753] The computing device 5100 may further include additional detachable/non-detachable, volatile/non-volatile memory medium. Although not shown in FIG. 51, it is possible to provide a magnetic disk drive for reading from and/or writing into a detachable and non-volatile magnetic disk and an optical disk drive for reading from and/or writing into a detachable non-volatile optical disk. In such cases, each drive may be connected to a bus (not shown) via one or more data medium interfaces.

[0754] The communication unit 5140 communicates with a further computing device via the communication medium. In addition, the functions of the components in the computing device 5100 can be implemented by a single computing cluster or multiple computing machines that can communicate via communication connections. Therefore, the computing device 5100 can operate in a networked environment using a logical connection with one or more other servers, networked personal computers (PCs) or further general network nodes.

[0755] The input device 5150 may be one or more of a variety of input devices, such as a mouse, keyboard, tracking ball, voice-input device, and the like. The output device 5160 may be one or more of a variety of output devices, such as a display, loudspeaker, printer, and the like. By means of the communication unit 5140, the computing device 5100 can further communicate with one or more external devices (not shown) such as the storage devices and display device, with one or more devices enabling the user to interact with the computing device 5100, or any devices (such as a network card, a modem and the like) enabling the computing device 5100 to communicate with one or more other computing devices, if required. Such communication can be performed via input/output (I/O) interfaces (not shown).

[0756] In some embodiments, instead of being integrated in a single device, some or all components of the computing device 5100 may also be arranged in cloud computing architecture. In the cloud computing architecture, the components may be provided remotely and work together to implement the functionalities described in the present disclosure. In some embodiments, cloud computing provides computing, software, data access and storage service, which will not require end users to be aware of the physical locations or configurations of the systems or hardware providing these services. In various embodiments, the cloud computing provides the services via a wide area network (such as Internet) using suitable protocols. For example, a cloud computing provider provides applications over the wide area network, which can be accessed through a web browser or any other computing components. The software

or components of the cloud computing architecture and corresponding data may be stored on a server at a remote position. The computing resources in the cloud computing environment may be merged or distributed at locations in a remote data center. Cloud computing infrastructures may provide the services through a shared data center, though they behave as a single access point for the users. Therefore, the cloud computing architectures may be used to provide the components and functionalities described herein from a service provider at a remote location. Alternatively, they may be provided from a conventional server or installed directly or otherwise on a client device.

[0757] The computing device 5100 may be used to implement video encoding/decoding in embodiments of the present disclosure. The memory 5120 may include one or more video coding modules 5125 having one or more program instructions. These modules are accessible and executable by the processing unit 5110 to perform the functionalities of the various embodiments described herein.

[0758] In the example embodiments of performing video encoding, the input device 5150 may receive video data as an input 5170 to be encoded. The video data may be processed, for example, by the video coding module 5125, to generate an encoded bitstream. The encoded bitstream may be provided via the output device 5160 as an output 5180.

[0759] In the example embodiments of performing video decoding, the input device 5150 may receive an encoded bitstream as the input 5170. The encoded bitstream may be processed, for example, by the video coding module 5125, to generate decoded video data. The decoded video data may be provided via the output device 5160 as the output 5180.

[0760] While this disclosure has been particularly shown and described with references to preferred embodiments thereof, it will be understood by those skilled in the art that various changes in form and details may be made therein without departing from the spirit and scope of the present application as defined by the appended claims. Such variations are intended to be covered by the scope of this present application. As such, the foregoing description of embodiments of the present application is not intended to be limiting.

I/We claim:

1. A method of video processing, comprising:

applying, for a conversion between a video unit of a video and a bitstream of the video, a combination of intra block copy (IBC) and intra prediction (CIBCIP) to the video unit;

deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and performing the conversion based on the prediction of the video unit.

2. The method of claim 1, wherein the prediction of the video unit is obtained by:

$$P(x, y) = w_{ip1} * IP_1(x, y) + w_{ip2} * IP_2(x, y) + \dots + w_{ipn} * IP_n(x, y) +$$

$$w_{ibc1} * IBC_1(x, y) + w_{ibc2} * IBC_2(x, y) + \dots + w_{ibcm} * IBC_m(x, y),$$

and

wherein  $P(x, y)$  represents the prediction of the video unit,  $IP_k$  represents a prediction generated by a  $k$ -th intra prediction,  $IBC_j$  is a prediction generated by a  $j$ -th IBC prediction, and  $w_{ipk}$  and  $w_{ibcj}$  represent corresponding weighting values, and  $k$  and  $j$  are integer numbers.

3. The method of claim 1, wherein the intra predicted signal is generated by at least one of: an angular intra-prediction mode, a direct currency (DC) mode, a planar mode, a cross-component prediction (CCLM) mode, a multi-model CCLM mode, a left CCLM mode, or an above CCLM mode, and/or

wherein an intra prediction mode of the intra predicted signal is coded using one of the followings to indicate the intra prediction mode: a most probable mode (MPM), a template-based intra mode derivation (TIMD), or a decoder-side intra mode derivation (DIMD).

4. The method of claim 1, wherein a set of intra prediction modes is allowed to be used in the CIBCIP, and/or

wherein an IBC merge mode is used in the CIBCIP, and/or wherein an IBC advanced motion vector prediction (AMVP) mode is used in the CIBCIP.

5. The method of claim 4, wherein one or more block vector (BV) candidates in an IBC merge list are allowed to be used in the CIBCIP, and/or

wherein the IBC merge mode comprises at least one of: a regular IBC merge mode, an IBC merge mode with block vector differences (IBC-MBVD) merge mode, or an IBC-template matching (TM) merge mode, and/or wherein one or more BV predictors in an IBC AMVP list are allowed to be used in the CIBCIP, and/or

wherein a block vector difference (BVD) used in the CIBCIP is indicated using a same way as IBC mode, and/or

wherein the BVD used in the CIBCIP is derived using coding information of the video unit.

6. The method of claim 1, wherein at least one of: a merge index indicating a BV candidate in an IBC merge list or a BVP index indicating a BV predictor in an IBC AMVP list used to obtain the IBC predicted signal is indicated.

7. The method of claim 6, wherein a binarization or signaling approach of the merge index or the BVP index is same as that in IBC mode.

8. The method of claim 1, wherein a first number of BV candidates in an IBC merge list that is used for the CIBCIP is less than or equal to a second number of BV candidates in the IBC merge list that is used for IBC, and

wherein the first number is an integer number that is larger than 0 and less than or equal to the second number, and/or

wherein a first number of BV candidates in an AMVP list that is used for the CIBCIP is less than or equal to a second number of BV candidates in the AMVP list that is used for the IBC, and wherein the first number is an integer number that is larger than 0 and less than or equal to the second number, and/or

wherein a template matching is used to derive/refine a BV that is used to obtain the IBC predicted signal, and/or wherein a BV list is reordered before being used for the CIBCIP.

9. The method of claim 8, wherein the template matching or the bilateral matching cost is used for the reordering of the BV list, and/or

wherein the template matching or the bilateral matching is used during a construction of the BV list used for the CIBCIP, and/or

wherein the BV list comprises an IBC merge list or an IBC AMVP list, and/or

wherein a reordering approach for the BV list used for the CIBCIP is same as that for IBC mode, or

wherein the reordering approach for the BV list used for the CIBCIP is different from that for the IBC mode.



**10.** The method of claim **1**, wherein an intra prediction comprise a conventional intra prediction approach or other intra prediction approach which obtains a prediction block with samples in of the followings excluding the IBC: a current slice, a current tile, a current subpicture, a current picture, or other video unit.

**11.** The method of claim **10**, wherein the intra predicted signal is obtained using one of:

- one or more pre-defined intra prediction modes,
- one or more of MPMs,
- an intra prediction mode which is derived using a block vector that is used to obtain the IBC predicted signal,
- one or more intra prediction modes which are derived using template base approach,
- one or more intra prediction modes which are derived using neighboring samples or gradients of the neighboring samples,
- an intra sub-partition (ISP),
- a matrix weighted intra prediction (MIP),
- a multiple reference line intra prediction (MRL), or
- an intra template matching prediction (IntraTMP).

**12.** The method of claim **1**, wherein weighting parameters used to combine the IBC predicted signal and the intra predicted signal are derived using coding information, or

- wherein the weighting parameters used to combine the IBC predicted signal and the intra predicted signal are pre-defined, and the coding information comprises a coding mode of neighboring units, and/or wherein the weighting parameters are dependent on whether one or more neighboring units are coded with an intra prediction or IBC mode.

**13.** The method of claim **1**, wherein whether to and/or an approach to apply the CIBCIP to the video unit depends on coding information, wherein the coding information comprise at least one of: whether the IBC or an intra prediction approach is allowed, a block dimensions and/or block size, a block depth, a slice type, a picture type, a partition tree type, a temporal layer identification, a block location, or a colour component, and/or

- wherein an indication of the CIBCIP is indicated based on a condition, wherein the condition comprise at least one of: whether the IBC or the intra prediction approach is allowed, the block dimensions and/or the block size, the block depth, the slice type, the picture type, the partition tree type, the temporal layer identification, the block location, or the colour component, and/or

- wherein whether the video unit is coded with the CIBCIP is indicated using one or more syntax elements, and/or the one or more syntax elements are bypass coded or context coded.

**14.** The method of claim **1**, wherein a reconstruction reordered IBC (RR-IBC) or a symmetric IBC approach is used in the CIBCIP, or

- wherein the RR-IBC or symmetric IBC approach are disabled in the CIBCIP.

**15.** The method of claim **1**, further comprising:  
combining a plurality of reference lines to define a combined plurality of reference lines; and  
deriving an intra prediction of the video unit based on the combined plurality of reference lines.

**16.** The method of claim **15**, wherein a number of reference lines and which reference lines to be combined are one of: pre-defined, indicated in the bitstream, or derived dynamically, wherein the number of reference lines is an integer number that is larger than 1.

**17.** The method of claim **1**, wherein the conversion includes encoding the video unit into the bitstream, or wherein the conversion includes decoding the video unit from the bitstream.

**18.** An apparatus for video processing comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to:

- apply, for a conversion between a video unit of a video and a bitstream of the video, a combination of intra block copy (IBC) and intra prediction (CIBCIP) to the video unit;

- derive a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and
- perform the conversion based on the prediction of the video unit.

**19.** A non-transitory computer-readable storage medium storing instructions that cause a processor to:

- apply, for a conversion between a video unit of a video and a bitstream of the video, a combination of intra block copy (IBC) and intra prediction (CIBCIP) to the video unit;

- derive a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and
- perform the conversion based on the prediction of the video unit.

**20.** A non-transitory computer-readable recording medium storing a bitstream of a video which is generated by a method performed by an apparatus for video processing, wherein the method comprises:

- applying a combination of intra block copy (IBC) and intra prediction (CIBCIP) to a video unit of the video;
- deriving a prediction of the video unit by combining an IBC predicted signal and an intra predicted signal; and
- generating the bitstream based on the prediction of the video unit.

\* \* \* \* \*